

Artificial Intelligence

CS 431

Prof. Dr. Hesham El-Deeb

2023/2024

رؤية كلية الحاسبات والذكاء الاصطناعي

تسعى كلية الحاسبات والذكاء الاصطناعي لأن تكون صرحاً تعليمياً وبحثياً متميزاً في مجال علوم الحاسب وتكنولوجيا المعلومات يعمل وفقاً لمعايير الجودة العالمية بما يمكن من تحقيق المنافسة والتميز على كل من المستويات المحلية والإقليمية والعالمية.

The Faculty of Computers and Artificial Intelligence aims to be a pioneer educational institution and research center in the field of Computer Science and Information Technology. This makes it possible to achieve competition and uniqueness at the local, regional, and international levels.

رسالة كلية الحاسبات والذكاء الاصطناعي

تلتزم كلية الحاسبات والذكاء الاصطناعي بتقديم خدمات تعليمية أكاديمية تؤدي إلى خريج متميز قادر على المنافسة محلياً وإقليمياً وعالمياً وعلى كفاءة عالية في مجالات الحاسبات وتكنولوجيا المعلومات ويتمتع بمهارات وأخلاقيات مهنية رفيعة وقادر على التعلم الذاتي لمواكبة التطورات السريعة في مجال تخصصه وتقديم تعليم مستمر وخدمات تدريبية واستشارية لتحقيق التنمية القومية والارتقاء بالبيئة المحلية وإجراء البحوث العلمية في مختلف مجالات علوم الحاسب وتكنولوجيا المعلومات.

The Faculty of Computers and Artificial Intelligence is committed to:

Providing academic educational services that lead to a distinguished graduate capable of competing locally, regionally, internationally, and highly qualified in the field of computers and information technology. In addition, graduates will have high professional skills and ethics and will be capable of self-learning to cope with rapid developments in his/her field of specialization.

Providing continuous education, training and consulting services to achieve national development and improve the local environment.

Conducting scientific research in various fields of computer science and information technology.

Course Specification

CS 431: Artificial Intelligence

A-Affiliation:

Relevant Program: computer science /BSc program

Department offering the program: CS Department

Department offering the course: CS Department

Date of Specification Approval: 18 February 2018

B-Basic Information:

Title : Artificial Intelligence **Code:** CS 431

Level : 4

Semester: Spring

Pre-requisite: CS111 Introduction to Computers

Tutorial/Practical: 3 H

Lectures: 3H

Credit Hours:3H

C-Professional Information:

Definition and design of intelligent agents. Types and applications of Intelligent Agent. Search problem definition. Types of informed search techniques, types of uninformed search techniques and applicability of search techniques. Logic programming tools, logic & resolution proof and using of logic concept in theorem proving. Definition of planning problem, types of plans, planning algorithms and applicability of Planning. Definition of the concept of learning, types of learning, learning algorithms and applicability of Neural Networks in learning. Natural Language Processing in Search engines. XML, RDF, ontology for NLP interface. Semantic Web and NLP.

1- Course Learning Objectives:

By the end of this course, the students should understand the presented overview of the AI principles and practices used to solve complex real-world issues, the fundamental background in problem-solving, logic programming, fuzzy logic, knowledge representation, reasoning, AI learning methods, and robotics.

The main objective is to give the students a general understanding of the topic at hand without going too deeply into the specifics, whenever that's possible.

2- Intended Learning Outcomes (I LOs)

a- Knowledge and Understanding:

On successful completion of the course, the students should demonstrate knowledge and sound understanding of:

- a1 Essential facts, concepts, principles, and theories relating to AI and AI applications as appropriate to the program of study ((A1)
- a3 Tools, practices and methodologies used in the specification, design, implementation, and evaluation of AI systems. (A4)
- a4 Criteria and specifications appropriate to specific problems, and plan strategies for their AI solutions. (A1)
- a10 Current developments in AI research. (A4)

b- Intellectual Skills:

On successful completion of the course, the students should be able to:

- b1 Analyze AI problems and provide solutions related to the design and construction of AI systems.(B1)
- b2 Realize the concepts, principles, theories and practices behind AI as an academic discipline. (B2)

b4 Analyze, propose and evaluate alternative AI systems and processes considering limitations, and quality constraints.(B3)

c –Professional and Practical Skills:

On successful completion of the course, the students should be able to:

c2 Implement comprehensive AI knowledge and skills in projects and in deployment of AI applications to solve position practical problems.(C1)

c3 Deploy the equipment and tools used for the construction, maintenance and documentation of AI applications.(C6)

C5 Develop a range of fundamental research skills, through the use of online resources, technical repositories and AI library-based material.(C1)

d- General and Transferable Skills:

On successful completion of the course, the students should be able to

d4 Use an appropriate mix of tools and aids in preparing and presenting reports for a range of audiences, including management, technical, users, industry or the academic community. (D4)

d6 Expose communication skills, public speaking and presentation skills, delegation, writing skills, oral delivery, and effectively using various media for a variety of audience.(D1)

Course Contribution in the CS program ILO'S

ILO'S		program ILO'S
A	Knowledge and understanding	A1, A4
B	Intellectual skills	B1,B2,B3
C	Professional and practical skills	C1,C6
D	General and transferable skills	D1,D4

Contents:

Topic	Lecture	Tutorial/ Practical
Intelligent Agents Definition and design of intelligent agents. Types of Intelligent Agent Applications of Intelligent Agent.	5	5
Search Search problem definition. Types of informed search techniques, Types of uninformed search techniques Applicability of search techniques	5	5
Logic programming Logic programming tools, logic & resolution proof using of logic concept in theorem proving.	5	5
Planning Definition of planning problem, Types of plans Planning algorithms Applicability of Planning.	5	5
Learning Definition of the concept of learning,	5	5

Types of learning, Learning algorithms Applicability of Neural Networks in learning.		
Natural Language Processing Natural Language Processing in Search engines. XML, RDF, Ontology for NLP interface. Semantic Web and NLP.	5	5
Total	30	30

4-Teaching and Learning and Assessment methods:

Course IL O'S		Teaching Methods						Learning Methods		Assessment Method				
		Lecture	Presentations and Movies	Discussions and seminars	Tutorials	Problem solving	Laboratory & Experiments	Researching and Reports	Modeling and Simulation	Written Exam	Practical Exam	Quizzes	Term papers	Assignments
Knowledge	a1	✓		✓	✓	✓		✓		✓		✓		✓
	a3	✓			✓					✓		✓		✓
	a4	✓			✓	✓		✓		✓		✓		✓
	a10	✓			✓	✓				✓		✓		✓
Information	b1	✓		✓	✓	✓		✓	✓	✓		✓		✓

	b2	✓			✓				✓		✓		
	b4	✓			✓				✓		✓		✓
Applied	c2	✓		✓	✓	✓			✓		✓		
	c3	✓		✓	✓	✓			✓		✓		
	C5	✓		✓	✓	✓			✓		✓		
General	d4	✓			✓	✓							✓
	d6	✓			✓								

5- Assessment Timing and Grading:

Assessment method	Timing	Grade (Decreases)
Semester Work : seminars, quizzes assignment and reports	Bi-Weekly	20%
Mid-Term Exam	7-th Week	20%
Written Exam	16-th Week	60%
Total		100%

6- List of references

6.1 - Course notes:

- Available a CD softcopy from lecture notes to all students.

6.2 – Required Books:

- Ali Magine. (2022) Fundamental Intelligence, Volume I: AI as a Label: Rethinking Natural and Artificial Intelligence.

6.3- Recommended Books:

- Peter Norvig . (2022) *Artificial Intelligence: A Modern Approach Paperback*
- Plamen P. Angelov. (2022) *Handbook on Computer Learning and Intelligence: Explainable Ai and Supervised Learning / Deep Learning, Intelligent Control and Evolutionary Computation 2nd Edition*
- Human-Centered Artificial Intelligence: Research and Applications. (2022) Chang Nam, Jae-Yoon Jung, Sangwon Lee.

6.4- Websites:

- <https://www.kcl.ac.uk/study/undergraduate/courses/artificial-intelligence-msci>

7- Facilities required for teaching and learning:

7.1 – White board and markers.

7.2 – Overhead projector (transparencies).

7.3 – Data show.

Course Coordinator: Prof. Dr. Hesham El-Deeb

Head of Department: Prof. Dr. Hanafy Ismaeel

Contents

CHAPTER 1	15
Artificial Intelligence Basic Concepts	15
1.1 Introduction	15
1.1.1 AI Perspective	16
1.1.2 AI Specialized languages	20
1.2 Artificial Intelligence Definitions	22
1.2.1 Thinking humanly (Cognitive modeling)	23
1.2.2 Thinking rationally (Laws of thought)	23
1.2.3 Acting humanly (Turing test)	23
1.2.4 Acting rationally (Rational agent)	24
1.3 AI brief history	25
Philosophy	25
1.4 AI state of the art	43
1.5 The Future of AI	44
Revision Sheet # 1	46
Assignment #1	46
Assignment #2	46
Assignment #3	46
CHAPTER 2	47
Robotics Basic Concepts	47
2.1 Robotics Scope	47
2.2 Robotics Design Issues	59
2.2.1 Robot Technology	60
2.2.2 Sensors and Actuators	63
2.2.3 Speaker command: Beep	65
2.2.4 Motors	65
2.2.5 LED Lights commands – setLED()	66
2.2.6 Mobile Robot locomotion (Moving Robot Style)	66
2.2.7 Robot Control	69

2.2.8 Case Study: Line Following Robot.....	74
2.3 Robotics Applications.....	76
2.3.1 Early Robotics.....	76
2.3.2 Robot Generations.....	77
2.3.3 Robotics Today	77
2.3.4 Robotics Road Map	78
Revision Sheet # 2	79
Assignment # 4	79
Assignment # 5	80
CHAPTER 3	81
Intelligent Agents.....	81
3.1 Intelligent Agent Definition	81
3.2 Environment Types	88
3.3 Types of Intelligent Agent.....	89
3.3.1 Simple reflex agents.....	89
3.3.2 Model-based reflex agents.....	91
3.3.3 Utility-based agents	92
3.3.4 Goal-based agents.....	93
3.3.5 Learning agents	94
3.4 Applications of Intelligent Agents.....	97
3.5 More Examples on software Agents.....	98
3.6 Intelligent Agent Design.....	103
Revision Sheet # 3	107
Assignment # 6	107
Assignment # 7	107
CHAPTER 4	108
Searching.....	108
4.1 Problem Solving Agents.....	108
4.2 Search Problem Types	109
4.3 Search problem definition	112

4.4	Search Problem Examples	114
4.5	Search Strategies	115
4.6	Types of uninformed search techniques	116
4.6.1	Breadth-first search.....	116
4.6.2	Uniform-cost search	119
4.6.3	Depth-first search	120
4.6.4	Depth-limited search	124
4.6.5	Iterative deepening search (DLS by increasing limit)	126
4.7	Types of Informed Search Techniques	129
4.7.1	Best-first search	129
4.7.2	Greedy best-first search	132
4.7.3	A* search (Enhanced Greedy Best First Search).....	137
4.8	Constraint Satisfaction Problems (CSP)	142
4.8.1	Constraint Satisfaction Problems Basic Concepts (CSP).....	142
4.8.2	Solving of CSPs	151
4.8.2.1	Backtracking Search (without heuristics)	151
4.8.2.2	Backtracking Search with Heuristics	156
4.9	Applicability of search techniques	161
	Revision Sheet # 4	162
	Assignment # 8	162
	Assignment # 9	162
	Assignment # 10	162
	Assignment # 11	163
	Assignment # 12	163
CHAPTER 5		164
Learning		164
5.1	Definition of the concept of learning.....	164
5.1.1	Factors affecting learning	164
5.2	Types of Learning.....	165
5.2.1	Learning Algorithms	167

5.3 Learning Decision Trees	168
5.3.1 The decision tree representation	168
5.3.2 Expressiveness of decision trees	169
5.3.3 Making decision trees from examples	170
5.4 Artificial Neural Networks	174
5.4.1 Neural network structures	176
5.4.2 Single-layer feed-forward neural networks (perceptrons)	178
5.4.3 Multilayer feed-forward neural networks	179
5.4.4 Learning in multilayer networks	180
5.4.5 Applicability of Neural Networks in learning	183
5.5 Support Vector Machines	190
5.6 Deep Learning	192
5.6.1 Deep Learning & Natural Language Processing	194
5.6.2 Deep Learning & Commercial activity	195
Revision Sheet # 5	196
Assignment # 13	196
Assignment # 14	196
CHAPTER 6	198
Planning	198
6.1 Definition of Planning Problem	198
6.2 Types of plans	198
6.3 Applicability of Planning	198
6.4 Planning Algorithms	200
6.5 Planning Examples	202
Revision Sheet # 6	207
Assignment # 15	207
Assignment # 16	207
CHAPTER 7	208
Logic Programming	208
7.1 Logic Programming Tools	208

7.2 logic & Resolution proof	221
7.3 Using of logic concept in theorem proving	223
Revision Sheet # 7	225
CHAPTER 8	226
Natural Language Processing.....	226
8.1 Natural Language Processing in Search engines	229
8.2 XML.....	232
8.3 RDF.....	239
8.4 Ontology for NLP interface	241
8.5 Semantic Web and NLP	242
Revision Sheet # 8	245

CHAPTER 1

Artificial Intelligence Basic Concepts

We call ourselves *Homo sapiens*—man the wise—because our intelligence is so important to us. For thousands of years, we have tried to understand how we think; that is, how a mere handful of matter can perceive, understand, predict, and manipulate a world far larger and more complicated than itself. The field of artificial intelligence, or AI, goes further still: it Artificial Intelligence attempts not just to understand but also to build intelligent entities.

AI is one of the newest fields in science and engineering. Work started in earnest soon after World War II, and the name itself was coined in 1956. Along with molecular biology, AI is regularly cited as the “field I would most like to be in” by scientists in other disciplines. A student in physics might reasonably feel that all the good ideas have already been taken by Galileo, Newton, Einstein, and the rest. AI, on the other hand, still has openings for several full-time Einsteins and Edisons.

AI currently encompasses a huge variety of subfields, ranging from the general (learning and perception) to the specific, such as playing chess, proving mathematical theorems, writing poetry, driving a car on a crowded street, and diagnosing diseases. AI is relevant to any intellectual task; it is truly a universal field.

1.1 Introduction

In computer science, Artificial Intelligence (AI), sometimes called machine intelligence, is intelligence demonstrated by machines, in contrast to the natural intelligence displayed by humans. Leading AI textbooks define the field as the study of “intelligent agents”: any device that perceives its environment and takes actions that maximize its chance of successfully achieving its goals. Colloquially, the term “artificial intelligence” is often used to describe machines (or computers) that mimic

"cognitive" functions that humans associate with the human mind, such as "learning" and "problem solving".

1.1.1 AI Perspective

Humankind has given itself the scientific name *Homo sapiens*—man the wise—because our mental capacities are so important to our everyday lives and our sense of self. The field of artificial intelligence, or AI, attempts to understand intelligent entities. Thus, one reason to study it is to learn more about ourselves. But unlike philosophy and psychology, which are also concerned with intelligence, AI strives to build intelligent entities as well as understand them. Another reason to study AI is that these constructed intelligent entities are interesting and useful in their own right. AI has produced many significant and impressive products even at this early stage in its development. Although no one can predict the future in detail, it is clear that computers with human-level intelligence (or better) would have a huge impact on our everyday lives and on the future course of civilization.

AI addresses one of the ultimate puzzles. How is it possible for a slow, tiny brain, whether biological or electronic, to perceive, understand, predict, and manipulate a world far larger and more complicated than itself? How do we go about making something with those properties? These are hard questions, but unlike the search for faster-than-light travel or an antigravity device, the researcher in AI has solid evidence that the quest is possible.

All the researcher has to do, is to look in the mirror to see an example of an intelligent system. AI is one of the newest disciplines. It was formally initiated in 1956, when the name was coined, although at that point work had been under way for about five years. Along with modern genetics, it is regularly cited as the "field we would most like to be in" by scientists in other disciplines.

A student in physics might reasonably feel that all the good ideas have already been taken by Galileo, Newton, Einstein, and the rest, and that it takes many years of study

before one can contribute new ideas. AI, on the other hand, still has openings for a full-time Einstein. The study of intelligence is also one of the oldest disciplines.

For over 2000 years, philosophers have tried to understand how seeing, learning, remembering, and reasoning could, or should, Chapter Introduction be done.' The advent of usable computers in the early 1950s turned the learned but armchair speculation concerning these mental faculties into a real experimental and theoretical discipline. Many felt that the new "Electronic Super-Brains" had unlimited potential for intelligence.

"Faster than Einstein" was a typical headline. But as well as providing a vehicle for creating artificially intelligent entities, the computer provides a tool for testing theories of intelligence, and many theories failed to withstand the test—a case of "out of the armchair, into the fire." AI has turned out to be more difficult than many at first imagined, and modern ideas are much richer, more subtle, and more interesting as a result.

AI currently encompasses a huge variety of subfields, from general-purpose areas such as perception and logical reasoning, to specific tasks such as playing chess, proving mathematical theorems, writing poetry, and diagnosing diseases. Often, scientists in other fields move gradually into artificial intelligence, where they find the tools and vocabulary to systematize and automate the intellectual tasks on which they have been working all their lives. Similarly, workers in AI can choose to apply their methods to any area of human intellectual endeavor. In this sense, it is truly a universal field.

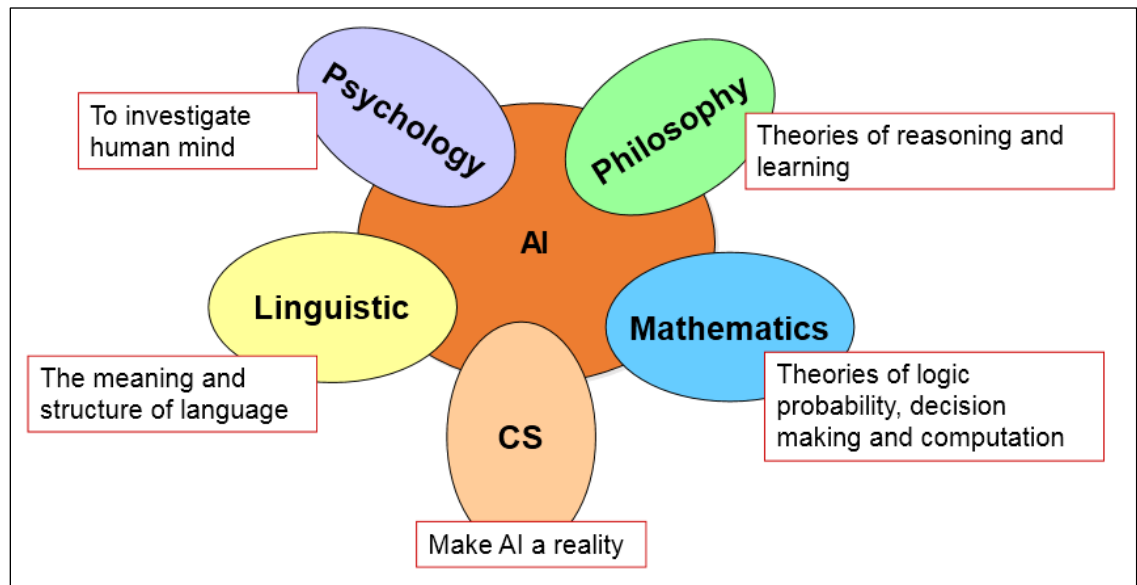
Artificial intelligence (AI) is the intelligence of machines and the branch of computer science which aims to create it. Again, major AI textbooks define artificial intelligence as "*the study and design of intelligent agents*," where an intelligent agent is a system that perceives its environment and takes actions which maximize its

chances of success. John McCarthy, who coined the term in 1956, defines it as "*the science and engineering of making intelligent machines*".

As we mention earlier, the most important topics in AI includes reasoning, knowledge, planning ,learning ,communication, perception and ability to move and manipulate object. These topics are grouped into two main branches. The first one is the Symbolic branch and the second one is the Sub-symbolic branch.

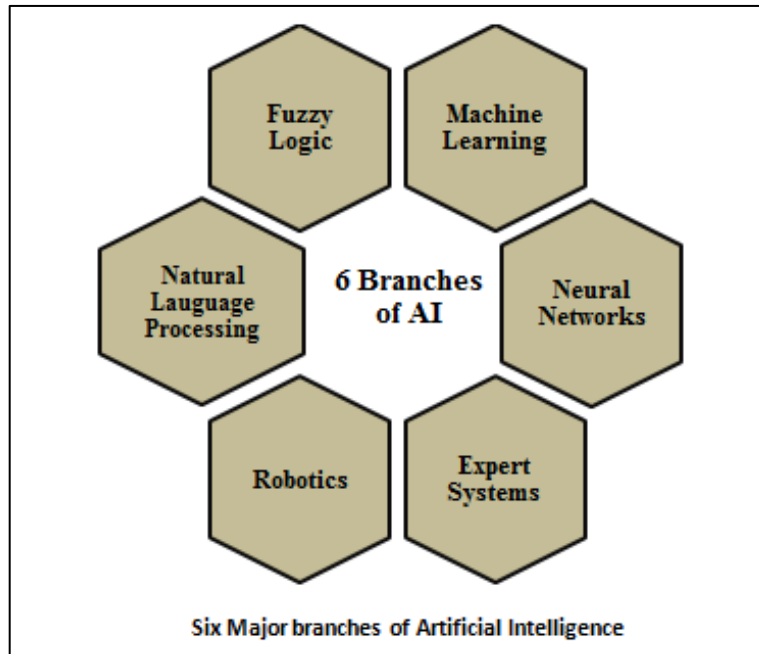
The later branch includes Early cybernetics and brain simulation, Nouvelle AI, Embodied AI, Situated AI, Behavior based AI; Embodied Embedded Cognition, Embodied cognitive science and Computational intelligence (CI) like Neural networks, Fuzzy systems and Evolutionary computation, containing Evolutionary algorithms and Swarm (Bees) intelligence.

AI inherited many ideas, viewpoints and techniques from other disciplines as shown in the following figure.



The large span of AI needs a huge solid and effective tools came from Computer science, Psychology, Philosophy, Neuroscience, Cognitive science, Linguistics, Ontology, Operations research, Economics, Control theory, Probability, Optimization and Logic.

These tools encourage the AI researchers to implement different overlap task Robotics, Control systems, Scheduling, Data mining, Logistics, Speech recognition and Facial recognition. These leads to the urgency of having the following branches of the AI field as shown in the following figure.



AI has many Expression Equivalence such as Computational Intelligence (CI), Synthetic Intelligence (SI), Intelligent Systems (IS) and Computational Rationality (CR).

Although AI is very rich and attractive area of scientific research area, some problems are still under extensive efforts to achieve its complete solution like Deduction, reasoning, problem solving, Knowledge representation ,Planning ,Learning ,Natural language processing ,Motion with manipulation ,Perception ,Social intelligence ,Creativity and General intelligence.

The high anticipation for solving such problems based on the Tools of AI research including Search and optimization , Logic , Probabilistic methods for uncertain reasoning, Classifiers and statistical learning methods , Neural networks , Control theory and specialized languages.

The applications of AI may include Medical diagnosis, Stock trading , Robot control, law and Scientific discovery and toys

1.1.2 AI Specialized languages

Now, the big question is raised: Do AI fields need special languages to cover all the previous topics, tasks and applications. The answer is loud and clear, yes and the major specialized AI languages will be presented as follows:

IPL (Information Processing Language)

It includes features intended to support programs that could perform general problem solving, including associations, lists, schemas (frames), dynamic memory allocation, data types, recursion, associative retrieval, functions as arguments, generators (streams), and cooperative multitasking

Lisp (list processing language)

It is a practical mathematical notation for computer programs based on lambda calculus (a formal system designed to investigate function definition, function application and recursion introduced by Alonzo Church and Stephen Cole Kleene in the 1930s as part of an investigation into the foundations of mathematics). Linked lists are one of Lisp languages' major data structures, and Lisp source code is itself made up of lists. As a result, Lisp programs can manipulate source code as a data structure, giving rise to the macro systems that allow programmers to create new syntax or even new domain-specific programming languages embedded in Lisp.

STRIPS (Stanford Research Institute Problem Solver)

It is an automated planner developed by Richard Fikes and Nils Nilsson in 1971). It is a language for expressing automated planning problem instances. It expresses an initial state, the goal states, and a set of actions. For each action pre-conditions (what must be established before the action is performed) and post-conditions (what is established after the action is performed) are specified.

Planner

Planner is a programming language designed by Carl Hewitt at MIT, and first published in 1969. First, subsets such as Micro-Planner and Pico-Planner were implemented, and then essentially the whole language was implemented as Popler by Julian Davies at the University of Edinburgh in the POP2 programming language. Derivations such as QA4, Conniver, and QLISP were important tools in artificial intelligence research in the 1970s, which influenced commercial developments such as KEE and ART.

It is a hybrid between procedural and logical languages. It gives a procedural interpretation to logical sentences where implications are interpreted with pattern-directed inference.

Prolog (Programming Logic Language)

Prolog has its roots in first-order logic, a formal logic, and unlike many other programming languages, Prolog is intended primarily as a declarative programming language: the program logic is expressed in terms of relations, represented as facts and rules. A computation is initiated by running a query over these relations. The language has been used for theorem proving, expert systems, term rewriting, type systems, and automated planning, as well as its original intended field of use, natural language processing. Modern Prolog environments support the creation of graphical user interfaces, as well as administrative and networked applications.

Other AI Language

AI applications are also often written in standard languages like C++ and languages designed for mathematics, such as Matlab and Lush (Lisp Universal Shell).

1.2 Artificial Intelligence Definitions

AI has different tasty alternative definitions that could give the readers the ability to have different angles to observe the AI field such as:

- “The exciting new effort to make computers think machine with minds” (Haugeland, 1985).
- “The study of mental faculties through the use of computational models” (Charniak and McDermott, 1985).
- "The art of creating machines that perform functions that require intelligence when performed by people" (Kurzweil, 1990).
- "The study of how to make computers do things at which, at the moment, people are better" (Rich and Knight, 1991).
- “The study of how to make computers do things at which, at the moment, people are better” (Rich and Knight, 1991).
- “The branch of computer science that is concerned with the automation of intelligent behavior” (Luger and Stubblefield, 1993).

Views of AI fall into four categories. The following table could include these four wings to fly with in order to have an AI application.

AI View	Description
Thinking humanly	“The exciting new effort to make computers think.... machines with minds, in the full literal sense”
Thinking rationally (correct behavior)	“The study of the computations that make it possible to perceive, reason, and act”
Acting humanly	“The art of creating machines that perform functions that require intelligence when performed by people”
Acting rationally	“computational intelligence is the study of the design of intelligence agents”

1.2.1 Thinking humanly (Cognitive modeling)

In the 1960s, the Harvard Center for Cognitive Studies and the Center for Human Information Processing at the University of California San Diego were influential in developing the academic study of cognitive science. Information-processing psychology replaced prevailing orthodoxy of behaviorism. By the early 1980s the cognitive revolution approach had become the dominant line of research inquiry across most branches in the field of psychology.

It requires scientific theories of internal activities of the brain such as what level of abstraction? “Knowledge” or “circuits”? in addition to how to validate the Predicting and testing behavior of human subjects (top-down) or Direct identification from neurological data (bottom-up). Both approaches (roughly, Cognitive Science and Cognitive Neuroscience) are now distinct from AI.

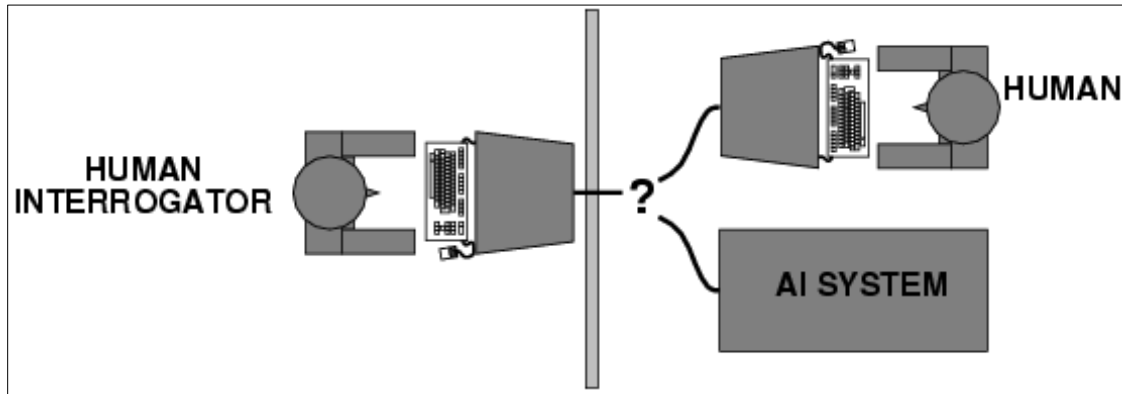
1.2.2 Thinking rationally (Laws of thought)

It is normative (or prescriptive) rather than descriptive. The famous Aristotle quotes:” what are correct arguments/thought processes”?

Several Greek schools developed various forms of logic: notation and rules of derivation for thoughts; may or may not have proceeded to the idea of mechanization. Direct line through mathematics and philosophy to modern AI. The Problems are: Not all intelligent behavior is mediated by logical discussion and what is the purpose of thinking? What thoughts should I have out of all the thoughts (logical or otherwise) that I could have?

1.2.3 Acting humanly (Turing test)

In 1950, Turing in his book "Computing machinery and intelligence" proposed to consider the question, ‘Can machines think?’ or “Can machines behave intelligently?”. Operational test for intelligent behavior: the Imitation Game.



Predicted that by 2000, a machine might have a 30% chance of fooling a lay person for 5 minutes. Anticipated all major arguments against AI in following 50 years. Suggested major components of AI: knowledge, reasoning, language understanding, learning

The problem is that Turing test is not reproducible, constructive, or amenable to mathematical analysis.

1.2.4 Acting rationally (Rational agent)

Rational behavior is doing the right thing, where the right thing which is expected to maximize goal achievement, given the available information. This doesn't necessarily involve thinking – e.g., blinking reflex – but thinking should be in the service of rational action Aristotle where *“Every art and every inquiry, and similarly every action and pursuit, is thought to aim at some good”*.

Rational Agents

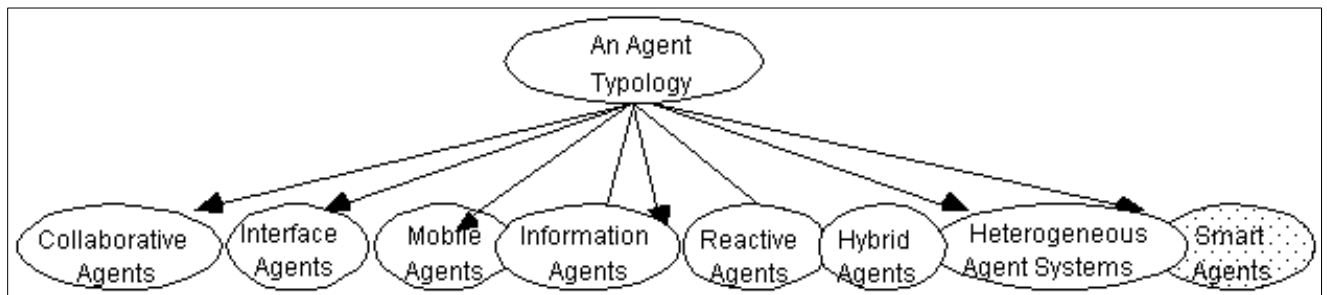
An agent is an entity that percept (sense) and act. Abstractly, an agent is a function from Percept histories to Actions:

$$[f: P^* \rightarrow A]$$

For any given class of environments and tasks, we seek the agent (or class of agents) with the best performance. Computational limitations make perfect rationality unachievable → design best program for given machine resources.

Software Intelligent Agents

A program which autonomously acts on behalf of its human (or organizational) principal while carrying out complex information and communication tasks which have been delegated to it. An intelligent agent operates in its environment by acting on information perceived from the environment and any existing applicable knowledge. They maybe: Collaborative agents, Interface Agents, Mobile Agents, Information Agents, Reactive Agents, Smart Agents, Multi-Agents, Hybrid Agents, Heterogeneous Agents.



1.3 AI brief history

In this section, we provide a brief history of the disciplines that contributed ideas, viewpoints, and techniques to AI. Like any history, this one is forced to concentrate on a small number of people, events, and ideas and to ignore others that also were important. We organize the history around a series of questions. We certainly would not wish to give the impression that these questions are the only ones the disciplines address or that the disciplines have all been working toward AI as their ultimate fruition.

Philosophy

- Can formal rules be used to draw valid conclusions?
- How does the mind arise from a physical brain?
- Where does knowledge come from?
- How does knowledge lead to action?

Aristotle (384–322 B.C.), was the first to formulate a precise set of laws governing the rational part of the mind. He developed an informal system of syllogisms for proper reasoning, which in principle allowed one to generate conclusions mechanically, given initial premises. Much later, Ramon Lull (d. 1315) had the idea that useful reasoning could actually be carried out by a mechanical artifact.

Thomas Hobbes (1588–1679) proposed that reasoning was like numerical computation that “we add and subtract in our silent thoughts.” The automation of computation itself was already well under way. Around 1500, Leonardo da Vinci (1452–1519) designed but did not build a mechanical calculator; recent reconstructions have shown the design to be functional.

The first known calculating machine was constructed around 1623 by the German scientist Wilhelm Schickard (1592–1635), although the Pascaline, built in 1642 by Blaise Pascal (1623–1662), is more famous. Pascal wrote that “the arithmetical machine produces effects which appear nearer to thought than all the actions of animals.” Gottfried Wilhelm Leibniz (1646–1716) built a mechanical device intended to carry out operations on concepts rather than numbers, but its scope was rather limited.

Leibniz did surpass Pascal by building a calculator that could add, subtract, multiply, and take roots, whereas the Pascaline could only add and subtract. Some speculated that machines might not just do calculations but actually be able to think and act on their own. In his 1651 book *Leviathan*,

Thomas Hobbes suggested the idea of an “artificial animal,” arguing “For what is the heart but a spring; and the nerves, but so many strings; and the joints, but so many wheels.” It’s one thing to say that the mind operates, at least in part, according to logical rules, and to build physical systems that emulate some of those rules; it’s another to say that the mind itself is such a physical system.

René Descartes (1596–1650) gave the first clear discussion of the distinction between mind and matter and of the problems that arise. One problem with a purely physical conception of the mind is that it seems to leave little room for free will: if

the mind is governed entirely by physical laws, then it has no more free will than a rock “deciding” to fall toward the center of the earth.

Descartes was a strong advocate of the power Rationalism of reasoning in understanding the world, a philosophy now called rationalism, and one that DUALISM counts Aristotle and Leibnitz as members. But Descartes was also a proponent of dualism. He held that there is a part of the human mind (or soul or spirit) that is outside of nature, exempt from physical laws. Animals, on the other hand, did not possess this dual quality; Materialism they could be treated as machines.

An alternative to dualism is materialism, which holds that the brain’s operation according to the laws of physics constitutes the mind. Free will is simply the way that the perception of available choices appears to the choosing entity. Given a physical mind that manipulates knowledge, the next problem is to establish Empiricism the source of knowledge.

The empiricism movement, starting with Francis Bacon’s (1561– 1626) *Novum Organum*, is characterized by a dictum of John Locke (1632–1704): “Nothing is in the understanding, which was not first in the senses.” David Hume’s (1711–1776) *A Treatise of Human Nature* (Hume, 1739) proposed what is now known as the principle of Induction: that general rules are acquired by exposure to repeated associations between their elements.

Building on the work of Ludwig Wittgenstein (1889–1951) and Bertrand Russell (1872–1970), the famous Vienna Circle, led by Rudolf Carnap (1891–1970), developed the LOGICAL POSITIVISM doctrine of logical positivism. This doctrine holds that all knowledge can be characterized by logical theories connected, ultimately, to observation sentences that correspond to sensory Observation Sentences inputs; thus, logical positivism combines rationalism and empiricism.

The confirmation theory of Carnap and Carl Hempel (1905–1997) attempted to analyze the acquisition of knowledge confirmation theory from experience.

Carnap's book *The Logical Structure of the World* (1928) defined an explicit computational procedure for extracting knowledge from elementary experiences. It was probably the first theory of mind as a computational process.

The *Novum Organum* is an update of Aristotle's *Organon*, or instrument of thought. Thus Aristotle can be seen as both an empiricist and a rationalist. In this picture, all meaningful statements can be verified or falsified either by experimentation or by analysis of the meaning of the words. Because this rules out most of metaphysics, as was the intention, logical positivism was unpopular in some circles.

The final element in the philosophical picture of the mind is the connection between knowledge and action. This question is vital to AI because intelligence requires action as well as reasoning.

Moreover, only by understanding how actions are justified can we understand how to build an agent whose actions are justifiable (or rational). Aristotle argued (in *De Motu Animalium*) that actions are justified by a logical connection between goals and knowledge of the action's outcome, But how does it happen that thinking is sometimes accompanied by action and sometimes not, sometimes by motion, and sometimes not?

Mathematics

- What are the formal rules to draw valid conclusions?
- What can be computed?
- How do we reason with uncertain information?

Philosophers staked out some of the fundamental ideas of AI, but the leap to a formal science required a level of mathematical formalization in three fundamental areas: logic, computation, and probability. The idea of formal logic can be traced back to the philosophers of ancient Greece, but its mathematical development really began with the work of George Boole (1815–1864), who worked out the details of propositional, or Boolean, logic (Boole, 1847). In 1879, Gottlob Frege (1848–1925)

extended Boole's logic to include objects and relations, creating the first order logic that is used today.

Alfred Tarski (1902–1983) introduced a theory of reference that shows how to relate the objects in a logic to objects in the real world. The next step was to determine the limits of what could be done with logic and algorithm computation. The first nontrivial algorithm is thought to be Euclid's algorithm for computing greatest common divisors.

The word algorithm (and the idea of studying them) comes from al-Khowarazmi, a Persian mathematician of the 9th century, whose writings also introduced Arabic numerals and algebra to Europe. Boole and others discussed algorithms for logical deduction, and, by the late 19th century, efforts were under way to formalize general mathematical reasoning as logical deduction. In 1930, Kurt Gödel (1906–1978) showed that there exists an effective procedure to prove any true statement in the first-order logic of Frege and Russell, but that first-order logic could not capture the principle of mathematical induction needed to characterize the natural numbers.

In 1931, Gödel showed that limits on deduction do exist. His incompleteness theorem showed that in any formal theory as strong as incompleteness theorem Peano arithmetic (the elementary theory of natural numbers), there are true statements that are undecidable in the sense that they have no proof within the theory. This fundamental result can also be interpreted as showing that some functions on the integers cannot be represented by an algorithm—that is, they cannot be computed.

This motivated Alan Turing (1912–1954) to try to characterize exactly which functions are Computable —capable of being computed. This notion is actually slightly problematic because the notion of a computation or effective procedure really cannot be given a formal definition. However, the Church–Turing thesis, which states that the Turing machine (Turing, 1936) is capable of computing any computable function, is generally accepted as providing a sufficient definition. Turing also showed that there were some functions that no Turing machine can

compute. For example, no machine can tell in general whether a given program will return an answer on a given input or run forever.

Although decidability and computability are important to an understanding of computation tractability, the notion of tractability has had an even greater impact. Roughly speaking, a problem is called intractable if the time required to solve instances of the problem grows exponentially with the size of the instances. The distinction between polynomial and exponential growth in complexity was first emphasized in the mid-1960s (Cobham, 1964; Edmonds, 1965). It is important because exponential growth means that even moderately large instances cannot be solved in any reasonable time.

Therefore, one should strive to divide the overall problem of generating intelligent behavior into tractable sub problems rather than intractable ones. NP-Completeness
How can one recognize an intractable problem? The theory of NP-completeness, pioneered by Steven Cook (1971) and Richard Karp (1972), provides a method.

Cook and Karp showed the existence of large classes of canonical combinatorial search and reasoning problems that are NP-complete. Any problem class to which the class of NP-complete problems can be reduced is likely to be intractable. (Although it has not been proved that NP-complete Frege's proposed notation for first-order logic—an arcane combination of textual and geometric features— never became popular Problems are necessarily intractable, most theoreticians believe it.) These results contrast with the optimism with which the popular press greeted the first computers—"Electronic Super-Brains" that were "Faster than Einstein!" Despite the increasing speed of computers, careful use of resources will characterize intelligent systems. Put crudely, the world is an extremely large problem instance! Work in AI has helped explain why some instances of NP-complete problems are hard, yet others are easy (Cheeseman et al., 1991).

Besides logic and computation, the third great contribution of mathematics to AI is the Probability theory of probability. The Italian Gerolamo Cardano (1501–1576) first framed the idea of probability, describing it in terms of the possible outcomes

of gambling events. In 1654, Blaise Pascal (1623–1662), in a letter to Pierre Fermat (1601–1665), showed how to predict the future of an unfinished gambling game and assign average payoffs to the gamblers. Probability quickly became an invaluable part of all the quantitative sciences, helping to deal with uncertain measurements and incomplete theories.

James Bernoulli (1654–1705), Pierre Laplace (1749–1827), and others advanced the theory and introduced new statistical methods. Thomas Bayes (1702–1761), who appears on the front cover of this book, proposed a rule for updating probabilities in the light of new evidence. Bayes’ rule underlies most modern approaches to uncertain reasoning in AI systems.

Economics

- How should we make decisions so as to maximize payoff?
- How should we do this when others may not go along?
- How should we do this when the payoff may be far in the future?

The science of economics got its start in 1776, when Scottish philosopher Adam Smith (1723–1790) published *An Inquiry into the Nature and Causes of the Wealth of Nations*. While the ancient Greeks and others had made contributions to economic thought, Smith was the first to treat it as a science, using the idea that economies can be thought of as consisting of individual agents maximizing their own economic well-being. Most people think of economics as being about money, but economists will say that they are really studying how people make choices that lead to preferred outcomes.

When McDonald’s offers a hamburger for a dollar, they are asserting that they would prefer the dollar and hoping that customers will Utility prefer the hamburger. The mathematical treatment of “preferred outcomes” or utility was first formalized by L’eon Walras (pronounced “Valrasse”) (1834-1910) and was improved by Frank Ramsey (1931) and later by John von Neumann and Oskar Morgenstern in their book *The Theory of Games and Economic Behavior* (1944).

Decision theory, which combines probability theory with utility theory, provides a formal and complete framework for decisions (economic or otherwise) made under uncertainty—that is, in cases where probabilistic descriptions appropriately capture the decision maker’s environment.

This is suitable for “large” economies where each agent need pay no attention to the actions of other agents as individuals. For “small” economies, the situation is much more like a game: the actions of one player can significantly affect the utility of another (either positively or negatively). Von Neumann and Morgenstern’s development of Game Theory included the surprising result that, for some games, Introduction a rational agent should adopt policies that are (or least appear to be) randomized.

Unlike decision theory, game theory does not offer an unambiguous prescription for selecting actions. For the most part, economists did not address the third question listed above, namely, how to make rational decisions when payoffs from actions are not immediate but instead result from several actions taken in sequence.

This topic was pursued in the field of operations research, which emerged in World War II from efforts in Britain to optimize radar installations, and later found civilian applications in complex management decisions. The work of Richard Bellman (1957) formalized a class of sequential decision problems called Markov decision processes.

Work in economics and operations research has contributed much to our notion of rational agents, yet for many years AI research developed along entirely separate paths. One reason was the apparent complexity of making rational decisions. The pioneering AI researcher Herbert Simon (1916–2001) won the Nobel Prize in economics in 1978 for his early satisficing work showing that models based on satisficing—making decisions that are “good enough,” rather than laboriously calculating an optimal decision—gave a better description of actual human behavior

(Simon, 1947). Since the 1990s, there has been a resurgence of interest in decision-theoretic techniques or agent systems (Wellman, 1995).

Neuroscience

- How do brains process information?

Neuroscience is the study of the nervous system, particularly the brain. Although the exact way in which the brain enables thought is one of the great mysteries of science, the fact that it does enable thought has been appreciated for thousands of years because of the evidence that strong blows to the head can lead to mental incapacitation. It has also long been known that human brains are somehow different; in about 335 B.C. Aristotle wrote, “Of all the animals, man has the largest brain in proportion to his size.”

Still, it was not until the middle of the 18th century that the brain was widely recognized as the seat of consciousness.

Before then, candidate locations included the heart and the spleen. Paul Broca’s (1824–1880) study of aphasia (speech deficit) in brain-damaged patients in 1861 demonstrated the existence of localized areas of the brain responsible for specific cognitive functions. In particular, he showed that speech production was localized to the portion of the left hemisphere now called Broca’s area.

By that time, it was known that the brain consisted of nerve cells, or neurons, but it was not until 1873 that Camillo Golgi (1843–1926) developed a staining technique allowing the observation of individual neurons in the brain.

This technique was used by Santiago Ramon y Cajal (1852– 1934) in his pioneering studies of the brain’s neuronal structures. Nicolas Rashevsky (1936, 1938) was the first to apply mathematical models to the study of the nervous system.

Since then, it has been discovered that the tree shrew (Scandentia) has a higher ratio of brain to body mass. Many cite Alexander Hood (1824) as a possible prior source. Golgi persisted in his belief that the brain’s functions were carried out primarily in a

continuous medium in which neurons were embedded, whereas Cajal propounded the “neuronal doctrine.” The two shared the Nobel Prize in 1906 but gave mutually antagonistic acceptance speeches.

Axon Cell body or Soma Nucleus Dendrite Synapses Axonal arborization Axon from another cell Synapse, the parts of a nerve cell or neuron. Each neuron consists of a cell body, or soma, that contains a cell nucleus. Branching out from the cell body are a number of fibers called dendrites and a single long fiber called the axon. The axon stretches out for a long distance, much longer than the scale in this diagram indicates. Typically, an axon is 1 cm long (100 times the diameter of the cell body), but can reach up to 1 meter. A neuron makes connections with 10 to 100,000 other neurons at junctions called synapses.

Signals are propagated from neuron to neuron by a complicated electrochemical reaction. The signals control brain activity in the short term and also enable long-term changes in the connectivity of neurons. These mechanisms are thought to form the basis for learning in the brain. Most information processing goes on in the cerebral cortex, the outer layer of the brain. The basic organizational unit appears to be a column of tissue about 0.5 mm in diameter, containing about 20,000 neurons and extending the full depth of the cortex about 4 mm in humans).

We now have some data on the mapping between areas of the brain and the parts of the body that they control or from which they receive sensory input. Such mappings are able to change radically over the course of a few weeks, and some animals seem to have multiple maps. Moreover, we do not fully understand how other areas can take over functions when one area is damaged. There is almost no theory on how an individual memory is stored. The measurement of intact brain activity began in 1929 with the invention by Hans Berger of the electroencephalograph (EEG).

The recent development of functional magnetic resonance imaging (fMRI) (Ogawa et al., 1990; Cabeza and Nyberg, 2001) is giving neuroscientists unprecedentedly detailed images of brain activity, enabling measurements that correspond in

interesting ways to ongoing cognitive processes. These are augmented by advances in single-cell recording of neuron activity. Individual neurons can be stimulated electrically, chemically, or even optically (Han and Boyden, 2007), allowing neuronal input– output relationships to be mapped.

Despite these advances, we are still a long way from understanding how cognitive processes actually work. The truly amazing conclusion is that a collection of simple cells can lead to thought, action, and consciousness or, in the pithy words of John Searle (1992), brains cause minds.

Psychology

- How do humans and animals think and act?

The origins of scientific psychology are usually traced to the work of the German physicist Hermann von Helmholtz (1821–1894) and his student Wilhelm Wundt (1832–1920). Helmholtz applied the scientific method to the study of human vision, and his Handbook of Physiological Optics is even now described as “the single most important treatise on the physics and physiology of human vision” (Nalwa, 1993). In 1879, Wundt opened the first laboratory of experimental psychology, at the University of Leipzig. Wundt insisted on carefully controlled experiments in which his workers would perform a perceptual or associative task while introspecting on their thought processes.

The careful controls went a long way toward making psychology a science, but the subjective nature of the data made it unlikely that an experimenter would ever disconfirm his or her own theories. Biologists studying animal behavior, on the other hand, lacked introspective data and developed an objective methodology, as described by H. S. Jennings (1906) in his influential work *Behavior of BEHAVIORISM the Lower Organisms*.

Applying this viewpoint to humans, the behaviorism movement, led by John Watson (1878–1958), rejected any theory involving mental processes on the grounds that introspection could not provide reliable evidence. Behaviorists insisted on studying

only objective measures of the percepts (or stimulus) given to an animal and its resulting actions (or response).

Behaviorism discovered a lot about rats and pigeons but had less success at understanding humans. Cognitive psychology, which views the brain as an information-processing device, cognitive psychology can be traced back at least to the works of William James (1842–1910). Helmholtz also insisted that perception involved a form of unconscious logical inference.

The cognitive viewpoint was largely eclipsed by behaviorism in the United States, but at Cambridge's Applied Psychology Unit, directed by Frederic Bartlett (1886–1969), cognitive modeling was able to flourish. The Nature of Explanation, by Bartlett's student and successor Kenneth Craik (1943), forcefully reestablished the legitimacy of such “mental” terms as beliefs and goals, arguing that they are just as scientific as, say, using pressure and temperature to talk about gases, despite their being made of molecules that have neither.

Craik specified the three key steps of a knowledge-based agent:

- (1) The stimulus must be translated into an internal representation,
- (2) The representation is manipulated by cognitive processes to derive new internal representations,
- (3) These are in turn retranslated back into action.

He clearly explained why this was a good design for an agent: If the organism carries a “small-scale model” of external reality and of its own possible actions within its head, it is able to try out various alternatives, conclude which is the best of them, react to future situations before they arise, utilize the knowledge of past events in dealing with the present and future, and in every way to react in a much fuller, safer, and more competent manner to the emergencies which face it.

(Craik, 1943) After Craik's death in a bicycle accident in 1945, his work was continued by Donald Broadbent, whose book *Perception and Communication* (1958)

was one of the first works to model psychological phenomena as information processing.

Meanwhile, in the United States, the development of computer modeling led to the creation of the field of cognitive science. The field can be said to have started at a workshop in September 1956 at MIT. (We shall see that this is just two months after the conference at which AI itself was “born.”) At the workshop, George Miller presented *The Magic Number Seven*, Noam Chomsky presented *Three Models of Language*, and Allen Newell and Herbert Simon presented *The Logic Theory Machine*.

These three influential papers showed how computer models could be used to address the psychology of memory, language, and logical thinking, respectively. It is now a common (although far from universal) view among psychologists that “a cognitive theory should be like a computer program” (Anderson, 1980); that is, it should describe a detailed information processing mechanism whereby some cognitive function might be implemented.

Computer engineering

- How can we build an efficient computer?

For artificial intelligence to succeed, we need two things: intelligence and an artifact. The computer has been the artifact of choice. The modern digital electronic computer was invented independently and almost simultaneously by scientists in three countries embattled in Introduction World War II.

The first operational computer was the electromechanical Heath Robinson, built in 1940 by Alan Turing’s team for a single purpose: deciphering German messages. In 1943, the same group developed the Colossus, a powerful general-purpose machine based on vacuum tubes.

The first operational programmable computer was the Z-3, the invention of Konrad Zuse in Germany in 1941. Zuse also invented floating-point numbers and the first

high-level programming language, Plankalkül. The first electronic computer, the ABC, was assembled by John Atanasoff and his student Clifford Berry between 1940 and 1942 at Iowa State University. Atanasoff's research received little support or recognition; it was the ENIAC, developed as part of a secret military project at the University of Pennsylvania by a team including John Mauchly and John Eckert, that proved to be the most influential forerunner of modern computers.

Since that time, each generation of computer hardware has brought an increase in speed and capacity and a decrease in price. Performance doubled every 18 months or so until around 2005, when power dissipation problems led manufacturers to start multiplying the number of CPU cores rather than the clock speed. Current expectations are that future increases in power will come from massive parallelism—a curious convergence with the properties of the brain. Of course, there were calculating devices before the electronic computer.

The earliest automated machines, dating from the 17th century. The first programmable machine was a loom, devised in 1805 by Joseph Marie Jacquard (1752–1834) that used punched cards to store instructions for the pattern to be woven. In the mid-19th century, Charles Babbage (1792–1871) designed two machines, neither of which he completed.

The Difference Engine was intended to compute mathematical tables for engineering and scientific projects. It was finally built and shown to work in 1991 at the Science Museum in London (Swade, 2000). Babbage's analytical engine was far more ambitious: it included addressable memory, stored programs, and conditional jumps and was the first artifact capable of universal computation.

Babbage's colleague Ada Lovelace, daughter of the poet Lord Byron, was perhaps the world's first programmer. (The programming language Ada is named after her.) She wrote programs for the unfinished Analytical Engine and even speculated that the machine could play chess or compose music.

AI also owes a debt to the software side of computer science, which has supplied the operating systems, programming languages, and tools needed to write modern programs (and papers about them).

But this is one area where the debt has been repaid: work in AI has pioneered many ideas that have made their way back to mainstream computer science, including time sharing, interactive interpreters, personal computers with windows and mice, rapid development environments, the linked list data type, automatic storage management, and key concepts of symbolic, functional, declarative, and object-oriented programming.

Heath Robinson was a cartoonist famous for his depictions of whimsical and absurdly complicated contraptions for everyday tasks such as buttering toast. In the postwar period, Turing wanted to use these computers for AI research—for example, one of the first chess programs (Turing et al., 1953). His efforts were blocked by the British government.

Control theory and cybernetics

- How can artifacts operate under their own control?

Ktesibios of Alexandria (c. 250 B.C.) built the first self-controlling machine: a water clock with a regulator that maintained a constant flow rate. This invention changed the definition of what an artifact could do. Previously, only living things could modify their behavior in response to changes in the environment. Other examples of self-regulating feedback control systems include the steam engine governor, created by James Watt (1736–1819), and the thermostat, invented by Cornelis Drebbel (1572–1633), who also invented the submarine.

The mathematical theory of stable feedback systems was developed in the 19th century. The central figure in the creation of what is now called control theory was Norbert Wiener (1894–1964). Wiener was a brilliant mathematician who worked with Bertrand Russell, among others, before developing an interest in biological and mechanical control systems and their connection to cognition.

Like Craik (who also used control systems as psychological models), Wiener and his colleagues Arturo Rosenblueth and Julian Bigelow challenged the behaviorist orthodoxy (Rosenblueth et al., 1943). They viewed purposive behavior as arising from a regulatory mechanism trying to minimize “error”—the difference between current state and goal state. In the late 1940s, Wiener, along with Warren McCulloch, Walter Pitts, and John von Neumann, organized a series of influential conferences that explored the new cybernetics mathematical and computational models of cognition.

Wiener’s book *Cybernetics* (1948) became a bestseller and awoke the public to the possibility of artificially intelligent machines. Meanwhile, in Britain, W. Ross Ashby (Ashby, 1940) pioneered similar ideas. Ashby, Alan Turing, Grey Walter, and others formed the Ratio Club for “those who had Wiener’s ideas before Wiener’s book appeared.” Ashby’s *Design for a Brain* (1948, 1952) elaborated on his HOMEOSTATIC idea that intelligence could be created by the use of homeostatic devices containing appropriate feedback loops to achieve stable adaptive behavior. Modern control theory, especially the branch known as stochastic optimal control, has as its goal the design of systems that maximize an objective function over time.

This roughly objective function matches our view of AI: designing systems that behave optimally. Why, then, are AI and control theory two different fields, despite the close connections among their founders? The answer lies in the close coupling between the mathematical techniques that were familiar to the participants and the corresponding sets of problems that were encompassed in each world view.

Calculus and matrix algebra, the tools of control theory, lend themselves to systems that are describable by fixed sets of continuous variables, whereas AI was founded in part as a way to escape from these perceived limitations. The tools of logical inference and computation allowed AI researchers to consider problems such as

language, vision, and planning that fell completely outside the control theorist's purview.

Linguistics

- How does language relate to thought? In 1957, B. F. Skinner published *Verbal Behavior*. This was a comprehensive, detailed account of the behaviorist approach to language learning, written by the foremost expert as an introduction to the field. But curiously, a review of the book became as well known as the book itself, and served to almost kill off interest in behaviorism.

The author of the review was the linguist Noam Chomsky, who had just published a book on his own theory, *Syntactic Structures*. Chomsky pointed out that the behaviorist theory did not address the notion of creativity in language—it did not explain how a child could understand and make up sentences that he or she had never heard before. Chomsky's theory—based on syntactic models going back to the Indian linguist Panini (c. 350 B.C.)—could explain this, and unlike previous theories, it was formal enough that it could in principle be programmed.

Modern linguistics and AI, then, were “born” at about the same time, and grew up together, intersecting in a hybrid field called computational linguistics or natural language computational linguistics processing.

The problem of understanding language soon turned out to be considerably more complex than it seemed in 1957. Understanding language requires an understanding of the subject matter and context, not just an understanding of the structure of sentences.

This might seem obvious, but it was not widely appreciated until the 1960s. Much of the early work in knowledge representation (the study of how to put knowledge into a form that a computer can reason with) was tied to language and informed by research in linguistics, which was connected in turn to decades of work on the philosophical analysis of language.

As a conclusion, we could summarize the disciplines that contributed ideas, viewpoints, and techniques to AI as follows:

Field	Description
Philosophy	Logic, methods of reasoning, mind as physical system foundations of learning, language, rationality
Mathematics	Formal representation and proof algorithms, computation, (un)decidability, (in)tractability, probability
Economics	utility, decision theory
Neuroscience	physical substrate for mental activity
Psychology	phenomena of perception and motor control, experimental techniques
Computer engineering	building fast computers
Control Theory	design systems that maximize an objective function over time
Linguistics	knowledge representation, grammar

Abridged history of AI

- 1943 McCulloch & Pitts: Boolean circuit model of brain
- 1950 Turing's "Computing Machinery and Intelligence"
- 1956 Dartmouth meeting: "Artificial Intelligence" adopted
- 1960s Early AI programs, including Samuel's checkers program,
- 1965 Robinson's complete algorithm for logical reasoning
- 1966-73 AI discovers computational complexity Neural network research
- 1969-79 Early development of knowledge-based systems

- 1980 AI becomes an industry
- 1986 Neural networks return to popularity
- 1987 AI becomes a science
- 1995 The emergence of intelligent agents

1.4 AI state of the art

Here are some examples of the AI state of the art deployment:

1. In 1997, Man vs. machine: fight of the 20th century: supercomputers Deep Blue took on world chess champion Garry Kasparov. The IBM-built machine was, on paper, far superior to Kasparov - capable of evaluating up to 200 million positions a second. But could it think strategically? The answer was a resounding yes. The supercomputer won the contest, dubbed 'the brain's last stand', with such flair that Kasparov believed a human being had to be behind the controls.
2. AI Proved a mathematical conjecture (Guising) which was unsolved for decades.
3. Also by AI usage, Researchers proved Autonomous Driving Technologies during a 3,000 Mile, Hands-off-the-Wheel Trip (driving autonomously 98%) from Pittsburgh to San Diego under the project “No-hands across America”.
4. During the 1991 Gulf War, US forces deployed an AI logistics planning and scheduling program that involved up to 50,000 vehicles, cargo, and people.
5. NASA's on-board autonomous planning program controlled the scheduling of operations for a spacecraft. Some programs solve crossword puzzles better than most humans.
6. A team of leading roboticists have created a real bionic man - complete with artificial organs, synthetic blood and robot limbs. The astonishing creation incorporates some of the latest advances in prosthetic technology, as well as an artificial pancreas, kidney, spleen and trachea, and a functional blood circulatory system. The robot has about 60 to 70 percent of the function of a human.

7. Known as Rex – short for robotic exoskeleton – his hi-tech frame is made up of an array of artificial limbs and organs from around the world. It stands six-and-a-half feet tall and can step, sit and stand with the help of a Rex walking machine that's used by people who've lost the ability to walk due to a spinal injury. It also has a functioning heart that, using an electronic pump, beats and circulates artificial blood, which carries oxygen just like human blood. This robot shares quite a bit in common with Steve Austin, the original 'bionic man' from the cult 1970s TV series the Six Million Dollar Man.

1.5 The Future of AI

Driven by three big trends, we're currently in the middle of a powerful new wave of AI. Computer power is growing, algorithms and AI models are becoming more sophisticated, and, perhaps most important of all, the world is generating unimaginable volumes of data. Data, measured in billions of gigabytes every day, is collected by networked devices in virtually every industry. As a result, AI is being tightly interwoven into almost every aspect of our lives, from our cars and medical devices to robots and entertainment. It's here to stay

Businesses are eagerly embracing robotics coupled with AI, recognizing the potential to lower costs, increase productivity and safety, make new product offerings possible, and provide actionable business insights. Interestingly, only a few business leaders are motivated by potential labour force reductions or risky "moonshots." Rather, 78% of leaders surveyed hope to use AI technology to transform existing operations. Robots that collaborate with people in the workplace are sometimes known as 'cobots'. Their duty is to operate alongside humans, working together to complete tasks.

Some people are concerned that if research in AI is successful, that computers will become too powerful. There are many complicated ethical issues in AI. For example, if a doctor asks an AI program for help with a diagnosis, and the machine gives a wrong answer, who is responsible? For most equipment a doctor uses, the

doctor herself is obliged to use it correctly. But for an AI system claiming to have the knowledge of other experts at work, could those other experts, or the programmers, be held responsible? In the future AI will also need to involve fields such as social policy and ethics.

So far though, AI products have been welcomed by our society. Current research should soon bring us programs to help us use the power of computers more naturally. You may be able to communicate with machines by talking, or they may even guess what you are trying to do by watching you and offer advice and help. Hopefully, AI will one day help us solve problems we cannot currently solve at all using traditional approaches to science and engineering.

People with no idea about AI
saying it will take over the world:

My Neural Network:



Revision Sheet # 1

Assignment #1

Prepare a survey on **reasoning** including:

definitions ,theories, applications ,problems and research areas and inter-relations with the AI Main Topics (reasoning, knowledge, planning , learning, communication, perception , ability to move and manipulate object).

Assignment #2

Examine whether the following tasks can currently be solved by computers:

- Playing a decent game of table tennis (Ping-Pong).
- Buying a week's worth of groceries at the market.
- Buying a week's worth of groceries on the Web.
- Playing a decent game of bridge at a competitive level.
- Discovering and proving new mathematical theorems.
- Giving competent legal advice in a specialized area of law.
- Translating spoken English into spoken Swedish in real time.
- Performing a complex surgical operation.

Assignment #3

Various subfields of AI have held competitions by defining a standard task and inviting researchers to do their best. Examples include:

- The DARPA Grand Challenge for robotic cars.
- The International Planning Competition.
- The Robocup robotic soccer league.
- The TREC information retrieval event.
- Contests in machine translation, speech recognition.

Investigate these competitions, and describe the progress made over the years.

To what degree have the competitions advanced to the state of the art in AI?

Do what degree do they hurt the field by drawing energy away from new ideas?

CHAPTER 2

Robotics Basic Concepts

2.1 Robotics Scope

'Robot' was first applied as a term for artificial automata in the 1920 play R.U.R. by the Czech writer, Karel Čapek. The word 'robot' itself was not new, having been in the Slavic language as robota (forced laborer), a term which classified those peasants obligated to compulsory service. Čapek's fictional story postulated the technological creation of artificial human bodies without souls.

The Robot Institute of America defines a robot as a programmable, multifunction manipulator designed to move material, parts, tools, or specific devices through variable programmed motions for the performance of a variety of tasks. This definition is not very demanding; a conveyor belt with a two-speed switch would arguably satisfy it. We will define robot simply as an active, artificial agent whose environment is the physical world.

The active part rules out rocks, the artificial part rules out animals, and the physical part rules out pure software agents or softbots, whose environment consists of computer file systems, databases and networks. We will be concerned primarily with autonomous robots, those that make decisions on their own, guided by the feedback they get from their physical sensors. Most of the design of an autonomous robot is the same as the design of any autonomous agent. To some extent, we could take a generic planning agent or decision-making agent, equip it with wheels, grippers, and a camera, point it out the door, and wish it good luck. Unfortunately, unless we pointed it at an exceptionally benign environment, it would not fare very well. The real world, in general, is very demanding.

We can see this by considering the five properties of environments:

- The real world is inaccessible. Sensors are imperfect, and in any case can only perceive stimuli that are near the agent.
- The real world is nondeterministic, at least from the robot's point of view. Wheels slip, batteries run down, and parts break, so you never know if an action is going to work. That means a robot needs to deal with uncertainty.
- The real world is non episodic—the effects of an action change over time. So a robot has to handle sequential decision problems and learning.
- The real world is dynamic. Therefore, a robot has to know when it is worth deliberating and when it is better to act immediately.
- The real world is continuous, in that states and actions are drawn from a continuum of physical configurations and motions. Because this makes it impossible to enumerate the set of possible actions, many of the search and planning algorithms described in earlier chapters will need to be modified.

2.1.1 AI as ‘Engineering and Science’

Artificial Intelligence (AI) combines science and engineering in order to build machines capable of intelligent behavior. It brings together work from the fields of philosophy, psychology, and computer.

- **AI as Engineering**

AI systems are often thought of as science fiction, but in fact are all around us. AI techniques are used, for example, by credit card companies to notice whether your card has been stolen, by looking for changes in your spending patterns. AI is also used in chess computers; in intelligent agents that search for information on the internet; for scheduling the observations of the Hubble space telescope; and for helping doctors diagnose illnesses. Robots were used to help clean up after the Chernobyl nuclear power disaster. Even your pocket calculator could be said to be AI - 100 years ago, no one would have expected a machine to know how to do maths!

- **AI as Science**

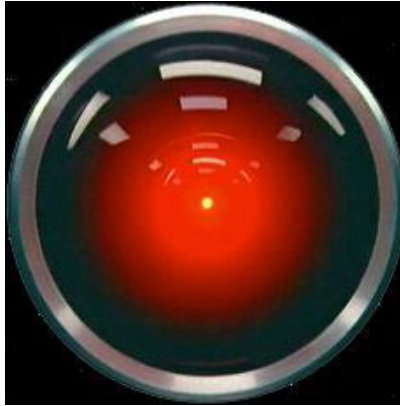
Although we can write chess playing programs that compete for the world championship it is difficult to build a robot that can walk or catch a ball. Computers can do calculus, but they can't learn to talk as well as a two-year-old child. Trying to understand why this might be is the scientific side of AI - it is the attempt to understand our own intelligence.

We can use intelligent machines to help us answer these questions about what intelligence is, and how it works. When we think we understand how a mental process works in a human or an animal, we can model it on a computer and see if our theory is correct. Sometimes it turns out to be simpler to test our theories of intelligence by building intelligent machines rather than by analyzing people and animals. In this way AI cooperates with other sciences like psychology, and philosophy of mind, which tries to understand what it means to be human, and how we think.

2.1.2 Robots ruling the World

When discussing whether robots shall take over the world, it is important to address the following inquiries:

- Which decisions can the machine make without human supervision?
- May machine-intelligent systems make mistakes (at the same level as humans)?
- May intelligent systems gamble when uncertain (as humans do)?
- Can (or should) intelligent systems exhibit personality?
- Can (or should) intelligent systems express emotion?
- How much information should the machine display to the human operator?



HAL - 2001 Space Odyssey series is a science fiction series of four novels and two films (1968 to 1997)

2.1.3 Robots Definition

According to The Robot Institute of America (1979): "A reprogrammable, multifunctional manipulator designed to move materials, parts, tools, or specialized devices through various programmed motions for the performance of a variety of tasks." According to the Webster dictionary: "An automatic device that performs functions normally qualified to humans or a machine in the form of a human (Webster, 1993)."

Alternative terms for Robotics

Robotics could have different name based on its operation environment, UAV: unmanned aerial vehicle (in air), UGV: unmanned ground vehicle (on ground) and UUV: unmanned undersea vehicle under sea).

Robot Components

The basic components of the robot are power supply, movable physical structure, sensor, effectors/actuator, motion system, on-board computer system and controllers for all of the above (smart methods everywhere).

2.1.4 Robots Benefits

Surprisingly, tasks which we find hard, computers find easy and vice versa. Examples are Actions involving Repetitive tasks, Continuous operations, complicated calculations and any actions referring to huge databases.

So we can currently say that what is easy for a human is hard for robots. Examples are any actions involving: Reasoning, Adapting to new situations, Flexible to changing requirements, integrating multiple sensors, Resolving conflicting data, Synthesizing unrelated information, and Creativity.

2.1.5 Robots Tasks

Robots perform jobs that humans find dull, dangerous or difficult. Either labor intensive or mundane, robots can be adapted to a variety of tasks. Industry, farming, medicine, the military, and space have all seen the expansion of robotics in their practice. The home has also become a hotbed for technology, as many appliances have now become "smart" devices over the years. The impact on humans is varied, but the utilization of robots will continue throughout this century and beyond.

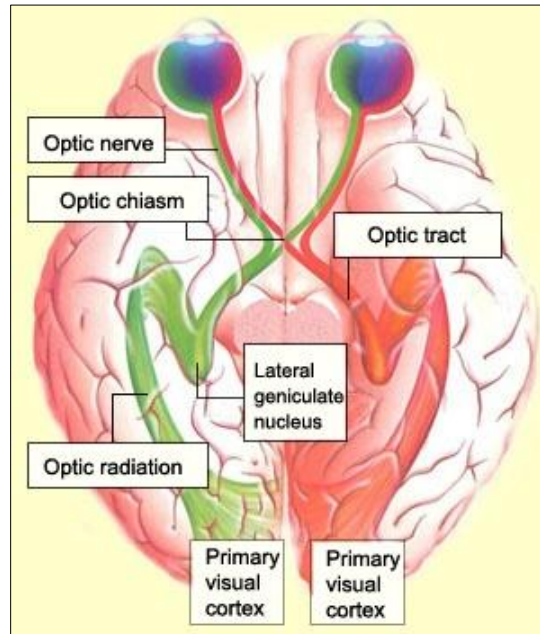
- Dangerous Tasks including space exploration, chemical spill cleanup, disarming bombs and disaster cleanup
- Boring and/or repetitive including welding car frames, part pick and place and manufacturing parts.
- High precision or high speed including electronics testing, surgery and precision machining.



2.1.6 What building Robots teaches us about Humans

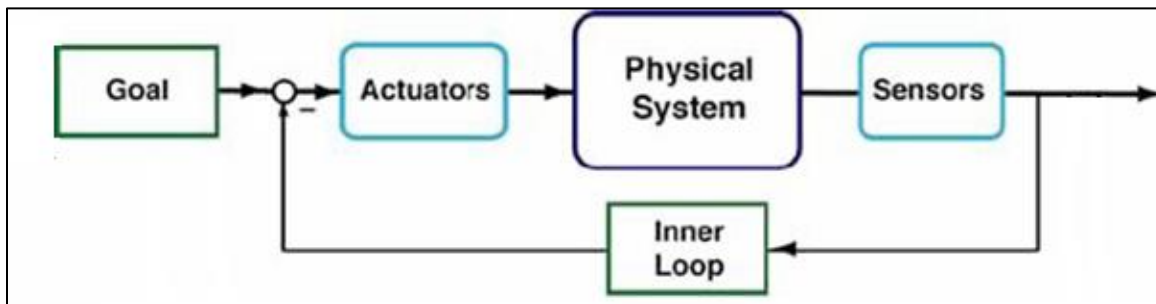
In order to build these intelligent machines AI uses many techniques. Expert Systems use the specialist knowledge that people like doctors and lawyers have in order to assist other people by giving advice on those subjects. Machine Learning is the study of how computers and robots can learn from their experience. Neural Networks are computers that work a little bit like brains (see BRAIN, MENTAL PROCESSES). Natural Language studies human languages, like English or Japanese, by trying to teach computers to understand them. The process of building robots makes us wonder about the following questions:

- How do our sensors work? (eyes and brain)
- How do we integrate sensors?
- How does our muscular-skeletal system work?
- How do we grab and hold an object?
- How does our brain process information?
- What is nature of intelligence?
- How do we make decisions?



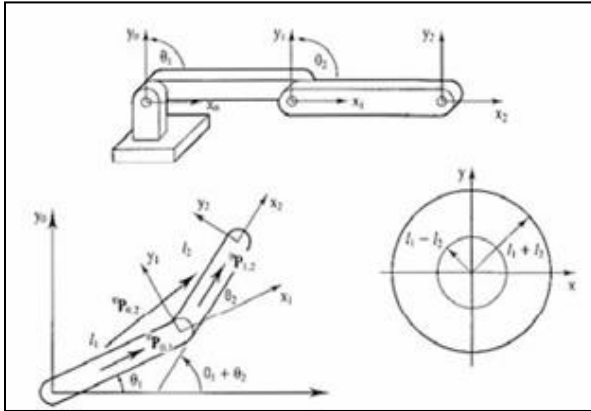
2.1.7 Robot Subsystems

A robot shall consist of the following subsystems: Action, Stationary base, Mobile, Sensors, Control and Power supply.



The actions of robots can be categorized as following:

- Actuators (Pneumatic (working by Air), Hydraulic (use of pressurized liquids) and electric solenoid).
- Motor (Analog (continuous) and Stepping (discrete increments)).
- Gears, belts, screws, levers.
- Manipulations.

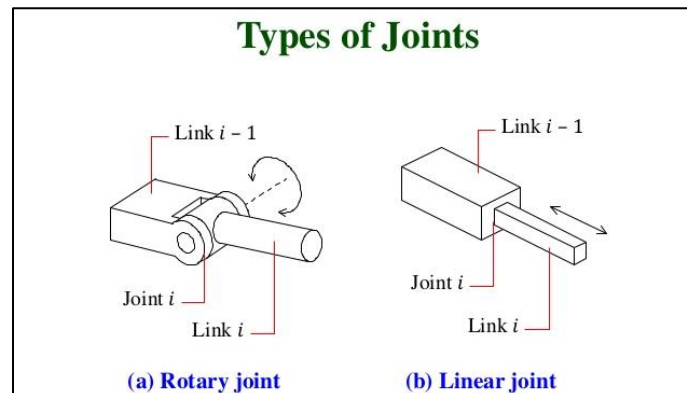


The actions of robots can be also be classified according to the movement as follows:

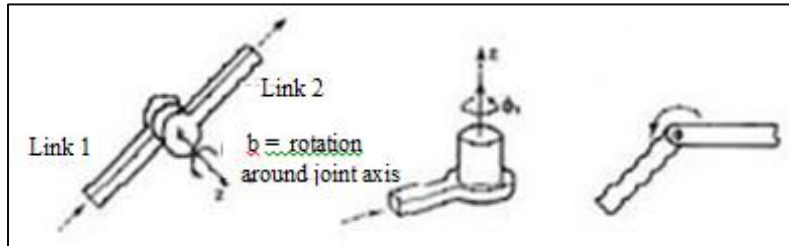
- Pick and place: Moves items between points.
- Continuous path control: Moves along a programmable path.
- Sensory: Employs sensors for feedback.

2.1.8 Robot Movement Process

- Simple joints (2D) containing Prismatic (sliding along one axis) and Revolute (rotating about one axis).



- Compound joints (3D) containing ball and socket = 3 revolute joints and round cylinder in tube = 1 prismatic, 1 revolute.

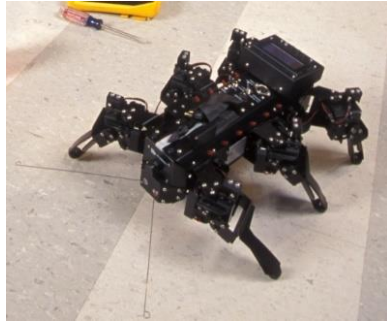


- Degrees of freedom = Number of independent motions containing 3 degrees of freedom: 2 translation, 1 rotation and 6 degrees of freedom: 3 translation, 3 rotation.



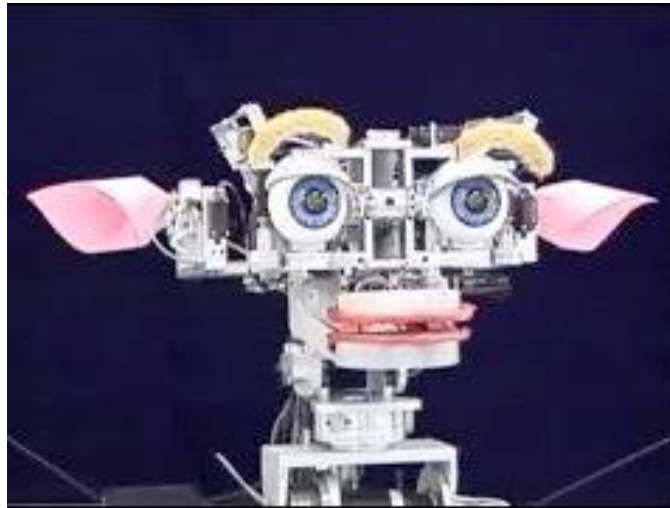
A Robot will use (Legs, Wheels, Tracks, Crawls (edges), or Rolls) for mobility.





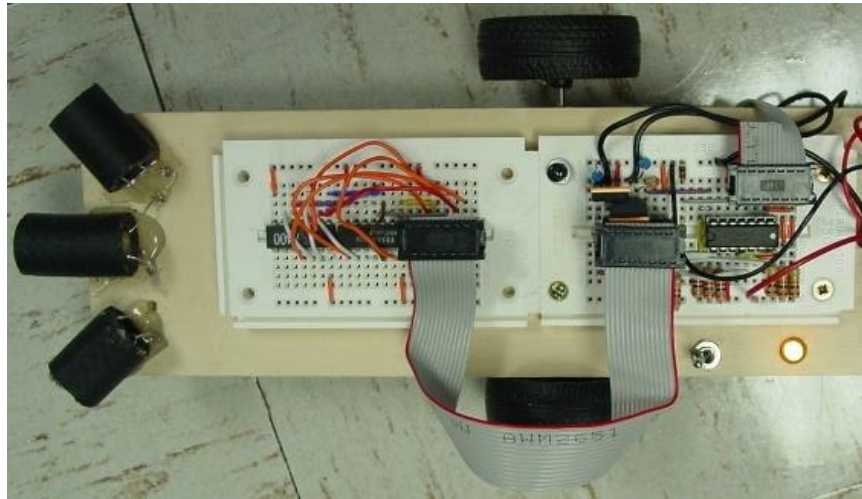
2.1.9 Robot Sensors Types

Optical (Laser / radar, 3D and Color spectrum), Pressure, Temperature, Chemical, Motion & Accelerometer and Acoustic (Ultrasonic).



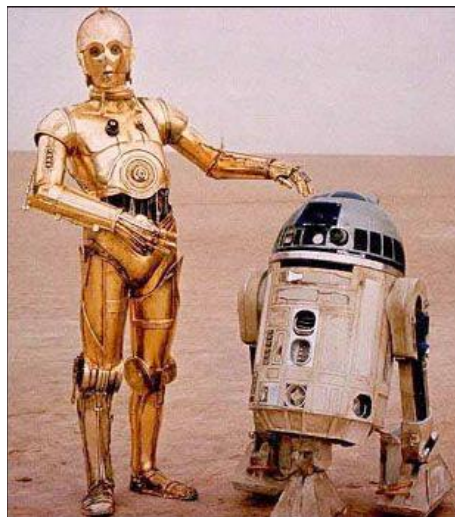
2.1.10 what use are sensors

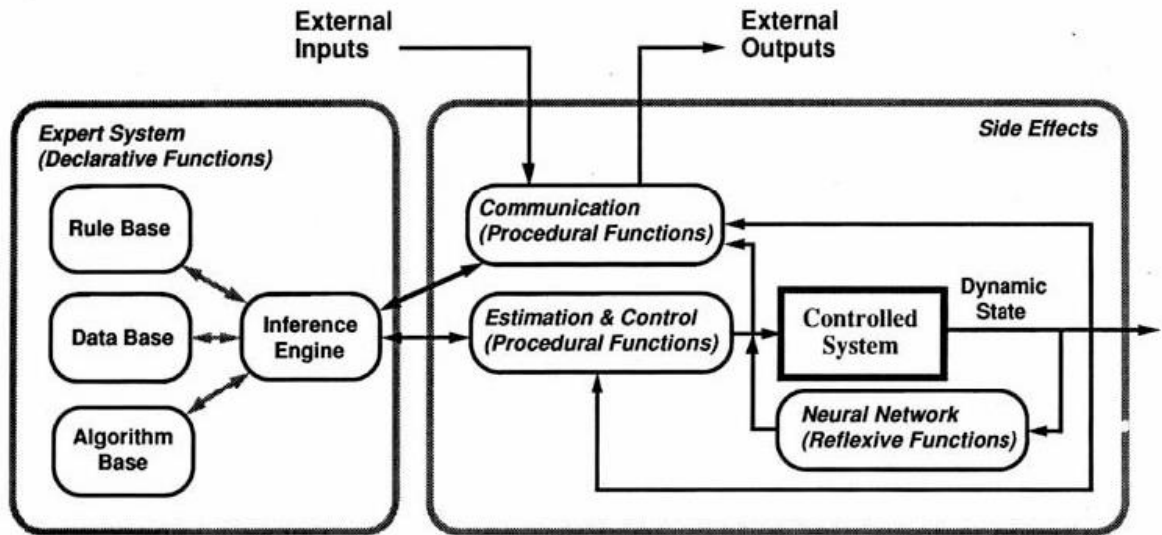
Robots use sensors for feedback. Closed-loop robots use sensors in conjunction with actuators to gain higher accuracy – servo motors. Uses include mobile robotics, telepresence, search and rescue, pick and place with machine vision.



Uses sensors for Control - the Brain

- Open loop, i.e., no feedback, deterministic(Instructions and Rules).
- Closed loop, i.e., feedback (Learn and Adapt).





2.1.11 Problems in Controlling Robot Actions

- Joint play compounded through N joints.
- Accelerating masses produce vibration, elastic deformations in links.
- Torques, stresses transmitted depending on end actuator loads.
- Feedback loop creates instabilities (Delay between sensing and reaction).
- Firmware and software problems (Especially with more intelligent approaches).

2.1.12 Measures of performance of robot

1. Speed and acceleration:

- Faster speed often reduces resolution or increases cost
- Varies depending on position, load.
- Speed can be limited by the task the robot performs (welding, cutting)

2. Resolution (measure of movement):

- Often a speed tradeoff.
- The smallest movement the robot can make.

3. Working volume (measure of theatre of operation):

- The space within which the robot operates.
- Larger volume costs more but can increase the capabilities of a robot

4. Accuracy:

- The difference between the actual position of the robot and the programmed position.
- Repeatability
 - Will the robot always return to the same point under the same control conditions?
 - Varies depending on position, load?

5. Cost: typically a tradeoff – improved performance on other metrics vs lower cost

6. Others: all the kinds of evaluation functions we have talked about for any AI system.

2.1.13 Robots vs. Automated Machinery

Robots are programmable machines which are able to carry out a series of actions autonomously, or semi-autonomously. They interact with the physical world via sensors and actuators. Because they are reprogrammable, they are more flexible than single-function machines. Machinery is designed to carry out a specific task (bottling machine, dishwasher and paint sprayer). But, robots are designed to carry out a variety of tasks (pick and place arms, mobile robots and numerical control machines).

2.2 Robotics Design Issues

Robot must be matched properly by capabilities to task requirements.

- Criteria For Robot Selection:
 - Technical issues :(type: nonservo, servo, servo-controlled, work envelope: rectangular, cylindrical, spherical, jointed arm, scara, payload, cycle time, repeatability, drive: electric, pneumatic, hydraulic, any combination and unique capabilities).
- Non-technical Issues:

- (Cost and benefit consideration, commonality of equipment, training and maintenance requirements, reliability, service, “systems” help and safety).

Robot Technology Limitations

- Sensors are limited and simple.
- Manipulators (Effectors) are limited and simple.
- State (internal and external, but mostly external) is partially-observable.
- Environment is dynamic (changing over time).
- Environment is full of potentially useful information.

Robotics Degree of Complexity

- Grounding in reality: not just planning in an abstract world.
- Situatedness (biological dynamics) : tight connection with the environment
- Embodiment: having a body.
- Emergent behavior: interaction with the environment.
- Scalability: increasing task and environment complexity.

2.2.1 Robot Technology

Robot technology is an applied science that is referred to as a combination of machine tools and computer applications. it includes such diverse fields as (machine design, control theory, ,microelectronics, ,computer programming, ,artificial intelligence, ,human factors, and production theory).

Robot Technology Terminologies

- Work envelope: the set of points representing the maximum extent or reach of the robot hand or working tool in all directions.
- Payload: The ability to carry, continuously and satisfactorily, a given maximum weight at a given speed.

- **Velocity:** The maximum speed at which the tip of a robot is capable of moving at full extension, expressed in inches or millimeters per second.
- **Cycle:** Time it takes for the robot to complete one cycle of picking up a given object at a given height, moving it to a given distance, lowering it, releasing it, and returning to the starting point.
- **Accuracy:** A robot's ability to position the end effector at a specified point in space upon receiving a control command without previously having attained that position.
- **Repeatability:** The ability of a robot to return consistently to a previously defined and achieved location.
- **Resolution:** The smallest incremental change in position that it makes, or its control system can measure.
- **Size:** The physical size of a robot, which influences its capacity and its capabilities.

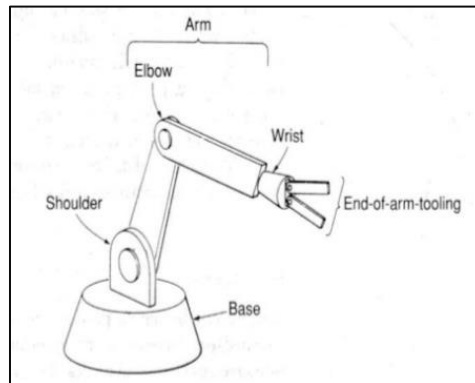
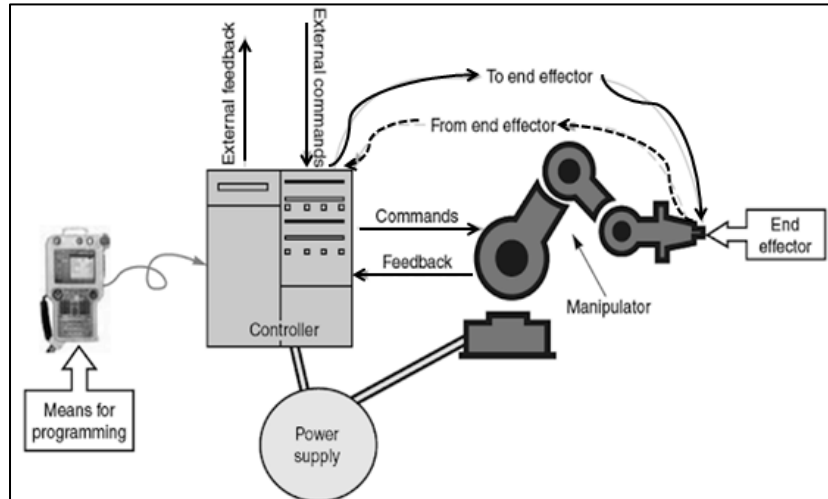
Basic Components of an Industrial Robot

The basic components of an industrial robot are the manipulator, the end effector (which is the end part of the manipulator), the power supply and the controller.

The manipulator, which is the robot's arm, consists of segments jointed together with axes capable of motion in various directions allowing the robot to perform work. The end effector which is a gripper tool, a special device, or fixture attached to the robot's arm, performs the work.

Power supply provides and regulates the energy that is converted to motion by the robot actuator, and it may be either electric, pneumatic, or hydraulic.

The controller initiates, terminates, and coordinates the motion of sequences of a robot. Also, it accepts the necessary inputs to the robot and provides the outputs to interface with the outside world.



Parts of a manipulator: the industrial robot manipulator has a body, arm and wrist (names match those of the corresponding human parts)

Robot Anatomy

Robot anatomy is concerned with the physical construction and characteristics of the body, arm, and wrist, which are components of robot manipulator. Movements between the various components of the body, arm, and wrist are provided by a series of joints. Attached to the robot wrist is the end effector (or end-of-arm tooling) that performs the work. The end effector is not considered a part of the robot's anatomy.

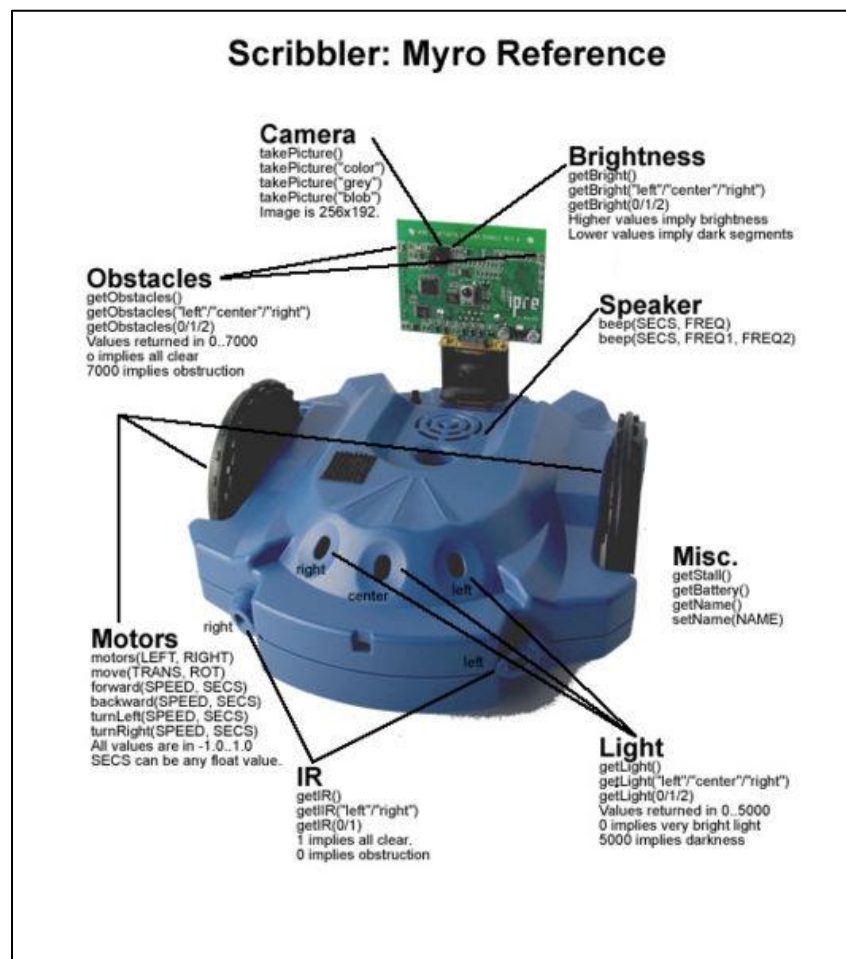
Robot Configurations

Industrial robots are available in a wide range of shapes, sizes, speeds, load capacities, and other capabilities. The vast majority of today's commercially available robots possess five distinct design configurations such as Rectangular or

Cartesian, cylindrical (or post-type), spherical (or polar), jointed arm (or articulated), scara (selective compliance assembly robot arm).

2.2.2 Sensors and Actuators

A famous example of is the Scribbler. It is a fully programmable, intelligent robot with multiple sensor systems that let it interact with people and objects. It navigates on its own as it explores its surroundings, and then reports back about what it senses using light and sound.



The Scribbler Robot comes fully assembled with these systems:

1. A built-in BASIC Stamp® 2 microcontroller brain (inside)
2. three photo resistor light sensors to detect bright light, or its absence
3. A pair of infrared emitter “headlights” and an infrared detector to see objects

4. A pair of infrared line sensors to follow lines or read bar codes
5. Two independent DC motors to drive the wheels
6. A built-in stall sensor to protect the motors (inside)
7. Three indicator LED lights that can be separately program-controlled
8. A speaker to play a wide range of sounds and musical notes
9. A Pen Port to insert a marker so the Scribbler can draw as it drives
10. A Programming Port that accepts a serial cable for a computer interface
11. Battery Cover – requires 6 standard, alkaline, or NiMH batteries



Scribbler Top & Bottom View

Fluke Built-in Board

The Fluke add-on board has its own IR obstacle sensors and a camera.



Robot Actuators

An actuator is an electromechanical device which converts energy into mechanical work. It is a component of a machine that is responsible for moving and controlling a mechanism or system, for example by opening a valve. An actuator requires a control signal and a source of energy. There are several types of actuators used in robots.

2.2.3 Speaker command: Beep

- beep() - issues a random beep
- beep (1) – random beep for 1 second
- beep (1,800) – Beep at 800Hz for 1 second
- beep (1,440,880) – two tone beep at 440 & 880Hz for 1 second.

2.2.4 Motors

- Two motors (left, right) that can be set to a power level between -1 and 1(-1 is full reverse, 0 is stopped and 1 is full speed forward).
- Controlled directly with the motors (left, right) function.
- These functions make it easier to use the motors to do simple things:
 - Forward (speed, seconds) – Moves the robot forward at the specified speed for the specified seconds, and then stops.
 - turnLeft (speed, seconds) – Turns the robot left at the specified speed and for the specified seconds, then stops.
 - turnRight () and backward () are similar.
- The previous functions would all start the robot, go for a certain amount of time, and then stop the robot automatically.
- You can also start the robot moving without specifying how long it should do so:
 - Forward (speed), turnLeft (speed), backward (speed), turnRight (speed), stop ().
 - Very important to call the stop () function when you want the robot to stop!

- The wait(seconds) function can be used to specify how long the robot should travel:(forward(0.85),wait(1.5) and stop()).
- Motor commands – Translate & Rotate
 - Other functions exist: translate (speed) and rotate(speed).
 - These functions allow you to add together different types of motion, allowing the robot to move forward – translate (1) while also turning—rotate (0.5).
 - Stop () is equivalent to translate (0); rotate (0).

2.2.5 LED Lights commands – setLED()

- The Scribbler robot has 3 green LED lights.
- You can control these three lights with settle D (position, value).
- Position is a number: 0, 1, or 2 or a string “left”, “center”, or “right”.
- Value is either 0 (“off”) or 1 (“on”).
- The center light blinks continuously, so unless you keep setting it's value, it will just start blinking again.

LED commands – setLEDFront() and setLEDBack()

- The Fluke board has 2 user controlled red LED lights.
- One on the front (small, near the camera) and one on the back (large, near the white connector).
- You can control these three lights with setLEDFront (value) and setLEDBack (value) functions.
- Value is either 0 (“off”) or 1 (“on”).
- The back LED is VERY bright! And you can set it to half power with a 0.5 value, or quarter power with 0.25.

2.2.6 Mobile Robot locomotion (Moving Robot Style)

- Classification of wheels (fixed wheel, centered orientable wheel, off-centered orientable wheel and Swedish wheel).

- Mobile Robot Locomotion (Differential Drive, Tricycle, Synchronous Drive, Omni-directional And Ackerman Steering (as in Car Drive))

Kinematics Models of Wheeled Mobile Robots (WMR)



It is a Combination of various physical (hardware) and computational (software) components. A collection of subsystems:

- **Locomotion:** how the robot moves through its environment.
- **Sensing:** how the robot measures properties of itself and its environment.
- **Control:** how the robot generates physical actions.
- **Reasoning:** how the robot maps measurements into actions.
- **Communication:** how the robots communicate with each other or with an outside operator.

Locomotion

It is the process of causing a robot to move. In order to produce motion, forces must be applied to the robot. Motor output, payload must be sufficient to do so.

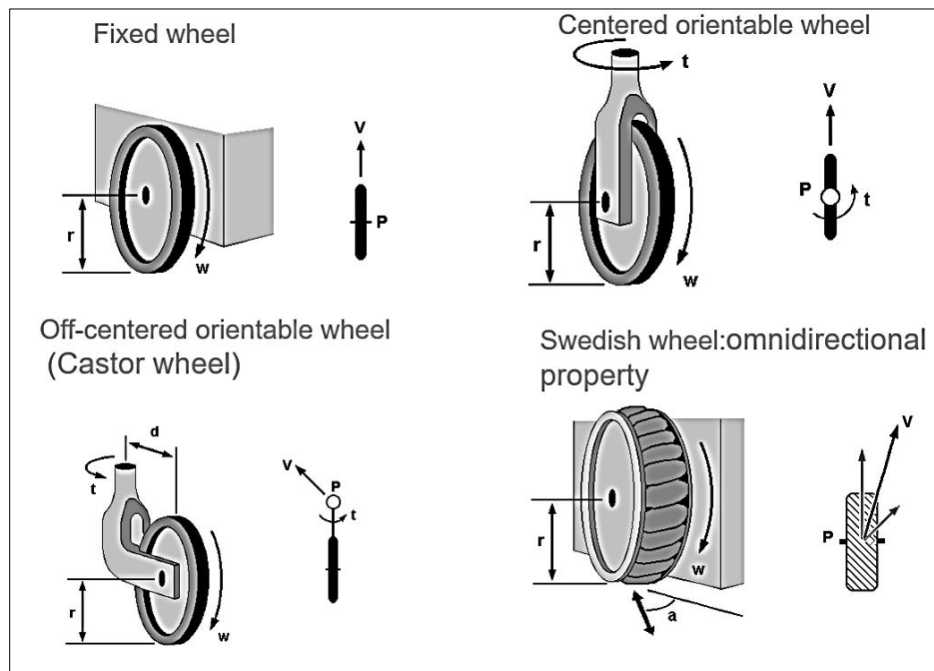
Dynamics

Study of motion in which these forces are modeled. Deals with the relationship between force and motions.

Kinematics

Study of the mathematics of motion without considering the forces that affect the motion. It deals with the geometric relationships that govern the system. It deals with the relationship between control parameters and the behavior of a system.

Wheel Types



Mobile Robot Locomotion Moving Styles

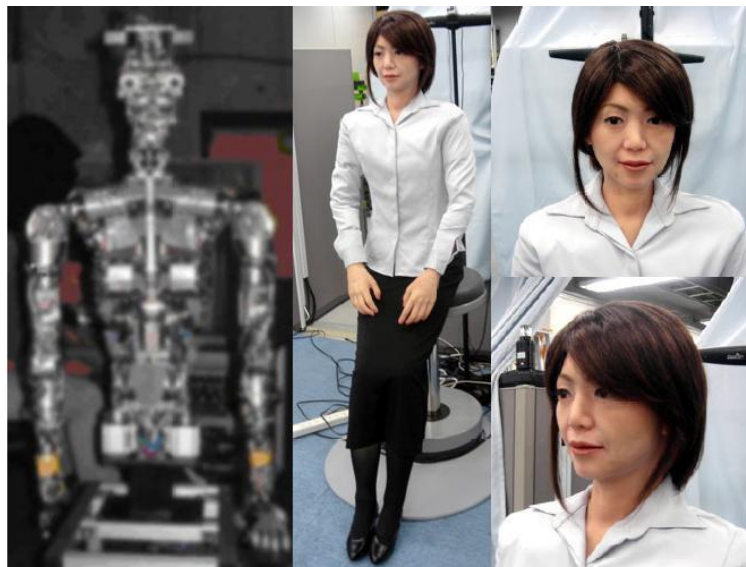
- Differential Drive (two driving wheels (plus roller-ball for balance), simplest drive mechanism and sensitive to the relative velocity of the two wheels (small error result in different trajectories, not just speed)).
- Tricycle (Steering wheel with two rear wheels, cannot turn $\pm 90^\circ$ and limited radius of curvature, Synchronous Drive, Omni-directional. and Car Drive (Ackerman Steering)).

2.2.7 Robot Control

Repliee Q2: is a type of android (humanoid robot) with strong visual human-likeness developed by Osaka University and manufactured by Kokoro Company Ltd. It was first unveiled at the 2003 International Robot Exhibition in Tokyo.

The researchers transfer human motion measured by a motion capture system to the android by coping changes in the positions of body surfaces because the android's appearance demands movements that look human but its kinematics is sufficiently different that copying joint-angle information would not yield good results. By comparing the similarity of the android's visible movement enables us to develop more natural movements for the android.

Repliee Q2 is modeled after a Japanese woman. The standing height is about 160 cm. The skin is composed of a kind of silicone that has a humanlike feel and neutral temperature. The silicone skin covers the upper torso, neck, head and forearms with clothing covering other body parts. The soft skin gives the android a human look and enable natural interaction. It has forty-two highly sensitive tactile sensors composed of peizo diaphragms are mounted under the android's skin and clothes throughout the body except for the shins, calves and feet.



The Developed Android Repliee Q2



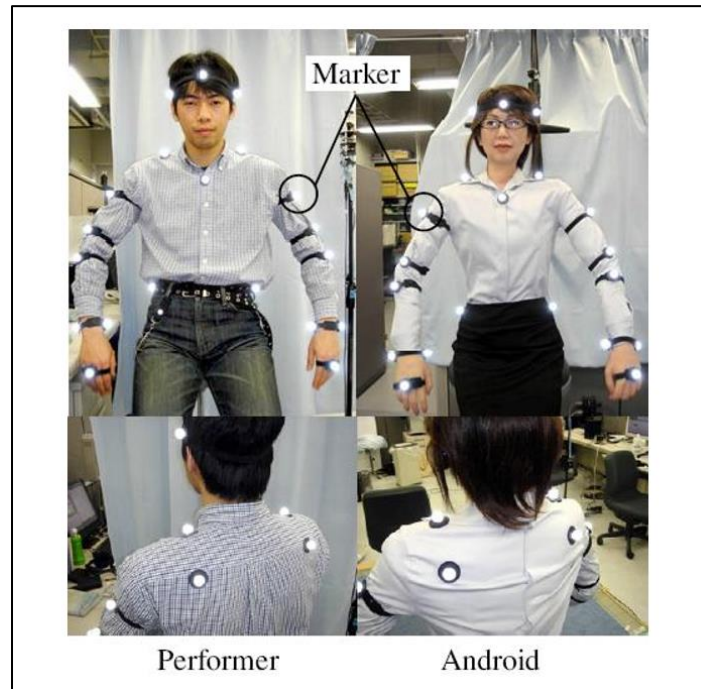
Examples of Motion and Facial Expressions

The android is driven by air actuators that give it 42 degrees of freedom (DoFs) from the waist up which can generate a wide range of motions and gestures as well as various kinds of micro-motions such as the shoulder movements typically caused by human breathing.

THE DOF CONFIGURATION OF REPLIEE Q2	
	Degree of freedom
Eyes	pan $\times$ 2 + tilt $\times$ 1
Face	eyebrows $\times$ 1 + eyelids $\times$ 1 + cheeks $\times$ 1
Mouth	7 (including the upper and lower lips)
Neck	3
Shoulder	5 $\times$ 2
Elbow	2 $\times$ 2
Wrist	2 $\times$ 2
Fingers	2 $\times$ 2
Torso	4

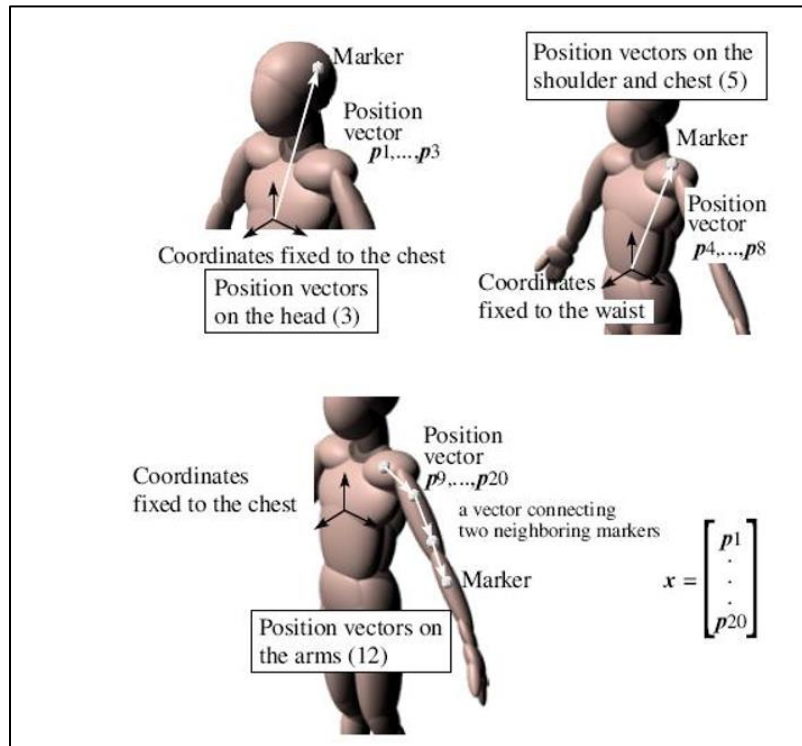
This below picture is an experiment to transfer human motion to the android Repliee Q2. They used 21 of the android's 42 DoFs by excluding the 13 DoFs of the face, the 4 of the wrists, and the 4 of the fingers ($n = 21$). They used a Hawk Digital System, which can track more than 50 markers in real-time. The system is highly accurate with a measurement error of less than 1 mm. Twenty markers were attached

to the performer and another 20 to the android as shown in Fig. 6 ($m = 20$). Because the android's waist is fixed, the markers on the waist set the frame of reference for an android-centered coordinate space.



To facilitate learning, they introduce a representation of the marker position as shown in Fig.7. The effect of waist motions is removed with respect to the markers on the head. To avoid accumulating the position errors at the end of the arms, vectors connecting neighboring pairs of markers represent the positions of the markers on the arms. They used arc tangents for the transformation T , in which the joint angle is an angle between two neighboring links where a link consists of a straight line between two markers.

There are 60 nodes in the input layer ($20 \text{ markers} \times x, y, z$), 300 in the hidden layer, and 21 in the output layer (for the 21 DoFs). Using 300 units in the hidden layer provided a good balance between computational efficiency and accuracy. Using significantly fewer units resulted in too much error, while using significantly more units provided only marginally higher accuracy but at the cost of slower convergence.



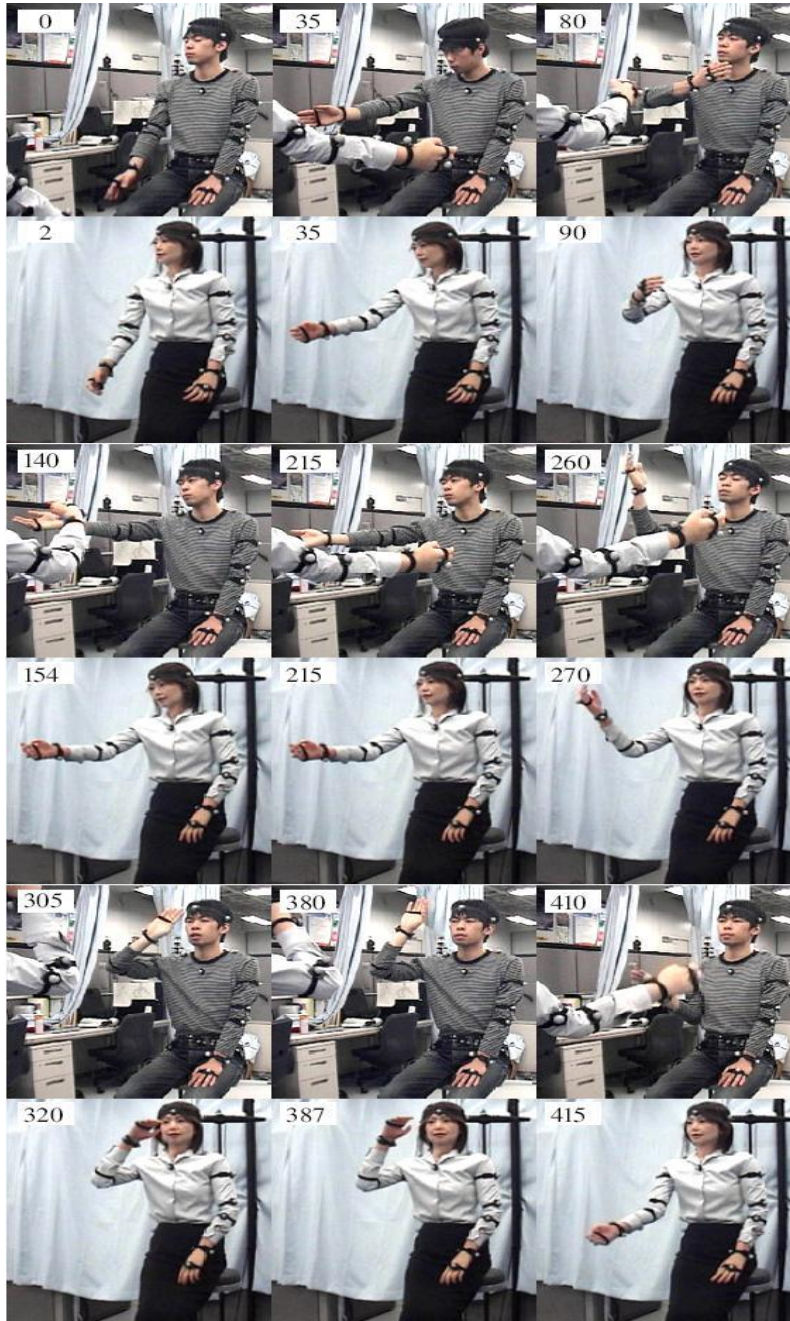
The representation of the Marker positions.

A marker's diameter is about 18 mm.

The Figure below shows the performer's postures during a movement and the corresponding postures of the android. The android followed the performer's movement with some delay. The trajectories of the positions of the android's markers are considered to be similar to those of the performer, but errors still remain, and they cannot be ignored. Researchers recognized that the android is making the same gesture as the performer, the quality of the movement is not the same. There are few reasons that causing this problem:

The kinematics of the android is too complicated to represent with an ordinary neural network. The method deals with a motion as a sequence of postures; it does not precisely reproduce higher order properties of motion such as velocity and acceleration because varying delays can occur between the performer's movement and the android's imitation of it. The proposed method is limited by the speed of

motion. The android has absolute physical limitations such as a fixed compliance and a maximum speed that is less than that of a typical human being.

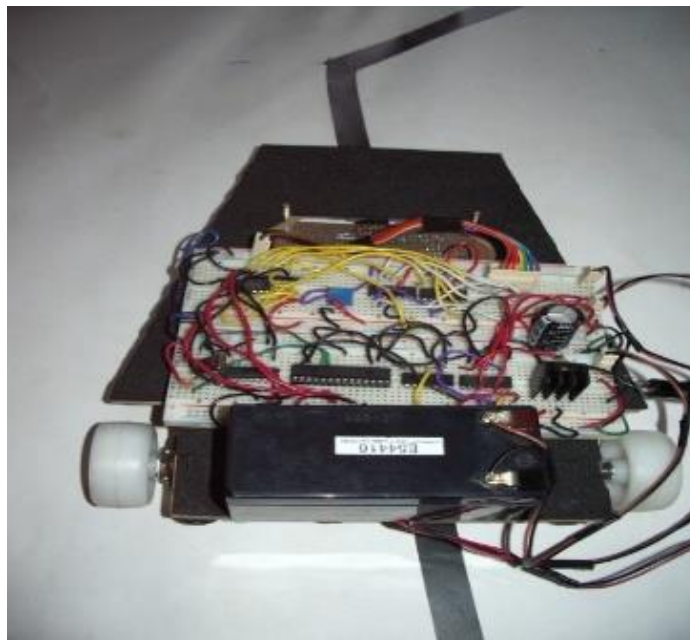


The Generated android's motion compared to the motion. The number represents the step. Different Design implies different behavior (needed no of steps to perform an action).

2.2.8 Case Study: Line Following Robot

The LFR has the following requirements:

1. The robot must be capable of following a line.
2. It should be capable of taking various degrees of turns
3. It must be prepared of a situation that it runs into a territory which has no line to follow.
4. The robot must also be capable of following a line even if it has breaks.
5. The robot must be insensitive to environmental factors such as lighting and noise.
6. It must allow calibration of the line's darkness threshold.
7. Scalability must be a primary concern in the design.
8. The color of the line must not be a factor as long as it is darker than the surroundings.

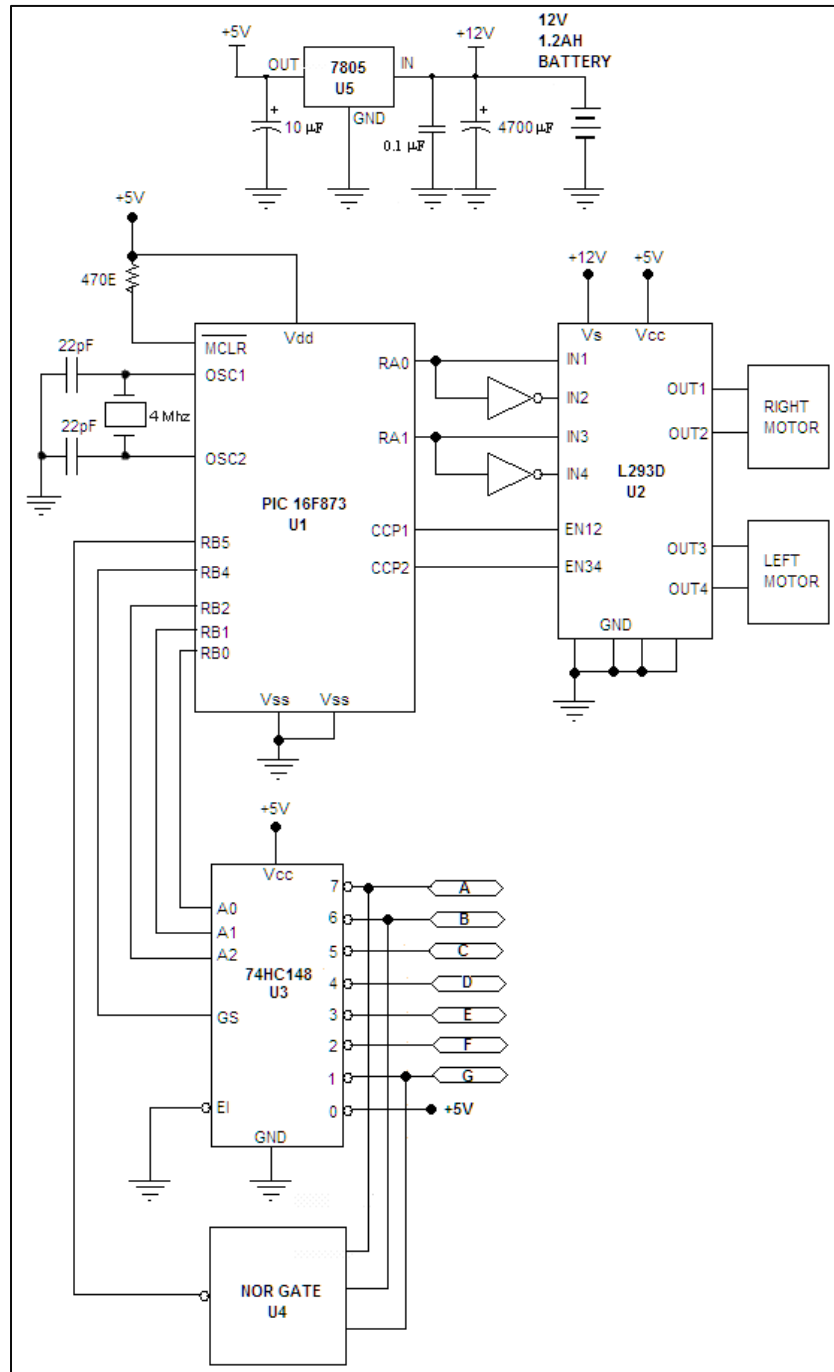


The Pic Microcontroller (PIC 16F873)

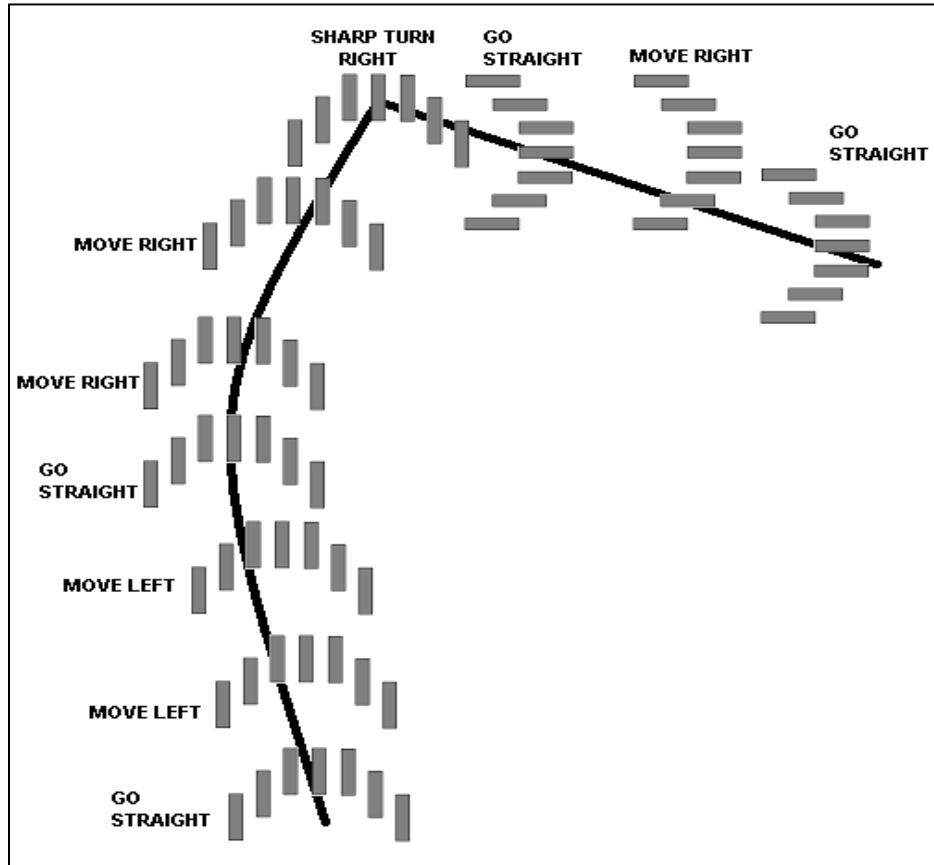
(Reduced Instruction Set Computer (RISC) Architecture, 1 Word Instruction Length, fixed Instruction Execution Time, 3 Ports (A, B & C), 2 Ccp (Capture, Compare))

Modules, 192 Bytes Data Ram & 128 Bytes of EPROM Data Memory, Upto 13 Interrupt Sources, 3 Timers, power Saving SLEEP Mode and A/D Converter)..

The Schematic Diagram



LFR Processes Involved



LFR Applications

(Industrial automated equipment carriers, automated cars, tour guides in museums and other similar applications and second wave robotic reconnaissance. operations).

2.3 Robotics Applications

2.3.1 Early Robotics

- Early Robots: SHAKEY
- Early Robots: HILARE

(LAAS in Toulouse, France (late 1970s) ,Video, ultrasound, laser range-finder ,Still in use! And Multi-level spatial representations).

2.3.2 Robot Generations

The five generations of robot controllers after the high-tech inception in 1960 are as follows:

- First generation: repeating robots. These were generally ‘pick and place’ robots, with mechanical sequences defining stop points.
- Second generation: hardwired controllers provided the first programmable units.
- Third generation: programmable logic controllers (plc), introduced in the industry over thirteen years ago, provided a microprocessor-based robotic controller that is easy to program.
- Fourth generation: when control beyond the plc is required, a microcomputer may control the entire system, including other programmable machinery in a robot work cell.
- Fifth generation: robot controller will involve complete artificial intelligence (AI), miniature sensors, and decision-making capabilities.

An artificial biological robot might provide the force for sixth and higher generation robots.

2.3.3 Robotics Today

- Assembly and manufacturing (most numbers of robots, least autonomous)
- Materials handling
- Gophers (hospitals, security guards)
- Hazardous environments (Chernobyl)
- Remote environments (Pathfinder)
- Surgery (brain, hips)
- Tele-presence and virtual reality
- Entertainment

2.3.4 Robotics Road Map

Building a robot

- [How to Make a Drone / UAV – Lesson 1: Terminology](#) (435)
- [How to Make a Robot – Lesson 1: Getting Started](#) (411)
- [How to Build a Simple Robot – Beetle Robot](#) (391)
- [How To Build A Simple Robot](#) (369)
- [How Do I Interpret DC Motor Specifications?](#) (309)[how to build a robot.docx](#)
- [Litter-Robot II Basic Troubleshooting Guide](#) (265)
- [How to Make a Robot – Lesson 10: Programming Your Robot](#) (258)
- [Arduino 5 Minute Tutorials: Lesson 7 – Accelerometers, Gyros, IMUs](#) (233)

Revision Sheet # 2

Assignment # 4

Answer the following (MCQs) focuses on “Robotics”.

1. What is the name for information sent from robot sensors to robot controllers?
 - a) Temperature
 - b) pressure
 - c) feedback
 - d) signal
2. Which of the following terms refers to the rotational motion of a robot arm?
 - a) Swivel
 - b) axle
 - c) retrograde
 - d) roll
3. What is the name for space inside which a robot unit operates?
 - a) Environment
 - b) spatial base
 - c) work envelope
 - d) exclusion zone
4. Which of the following terms IS NOT one of the five basic parts of a robot?
 - a) Peripheral tools
 - b) end effectors
 - c) controller
 - d) drive
5. Decision support programs are designed to help managers make _____.
 - a) budget projections
 - b) visual presentations
 - c) business decisions
 - d) vacation schedules
6. PROLOG is an AI programming language which solves problems with a form of symbolic logic known as predicate calculus. It was developed in 1972 at the University of Marseilles by a team of specialists. Can you name the person who headed this team?
 - a) Alain Colmerauer
 - b) Niklaus Wirth
 - c) Seymour Papert
 - d) John McCarthy

7. The number of moveable joints in the base, the arm, and the end effectors of the robot determines_____

- a) degrees of freedom
- b) payload capacity
- c) operational limits
- d) flexibility

8. Which of the following places would be LEAST likely to include operational robots?

- a) Warehouse
- b) factory
- c) hospitals
- d) private homes

9. For a robot unit to be considered a functional industrial robot, typically, how many degrees of freedom would the robot have?

- a) Three
- b) four
- c) six
- d) eight

10. Which of the basic parts of a robot unit would include the computer circuitry that could be programmed to determine what the robot would do?

- a) Sensor
- b) controller
- c) arm
- d) end effector

Assignment # 5

Prepare a survey on **knowledge** including:

definitions ,theories, applications ,problems and research areas and inter-relations with the AI Main Topics (reasoning, knowledge, planning , learning, communication, perception , ability to move and manipulate object)

CHAPTER 3

Intelligent Agents

3.1 Intelligent Agent Definition

In artificial intelligence, an intelligent agent (IA) refers to an autonomous entity which acts, directing its activity towards achieving goals (i.e. it is an agent), upon an environment using observation through sensors and consequent actuators (i.e. it is intelligent). In other words, an agent is anything that can be viewed as perceiving its environment through sensors_and acting upon that environment through actuators.

Human agent has different sensor (eyes, ears, tongue, skin and other organs) and (hands, legs, mouth, and other body parts) for actuators. That is why the robotic agent has cameras, infrared range finders for sensors and various motors for actuator.

History of Agent

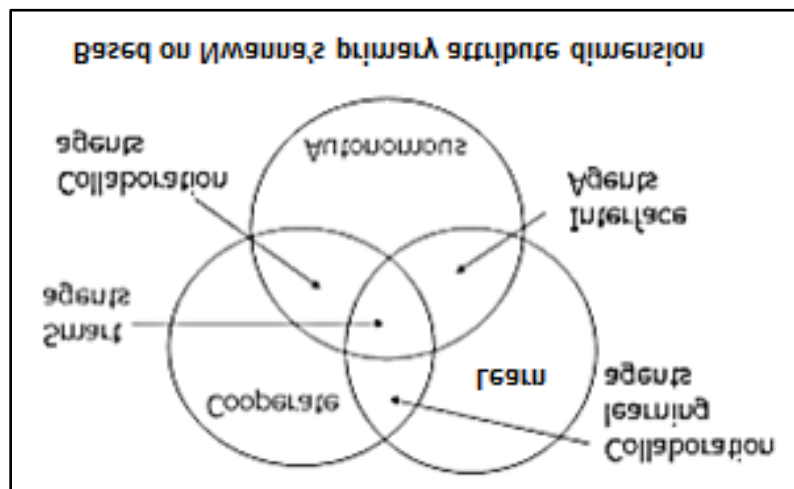
The concept of an agent can be traced back to Hewitt's Actor Model (Hewitt, 1977) - "A self-contained, interactive and concurrently-executing object, possessing internal state and communication capability."

Software agent systems are a direct evolution from Multi-Agent Systems (MAS). MAS evolved from Distributed Artificial Intelligence (DAI), Distributed Problem Solving (DPS) and Parallel AI (PAI), thus inheriting all characteristics (good and bad) from DAI and AI.

John Sculley's 1987 "Knowledge Navigator" video portrayed an image of a relationship between end-users and agents. Being an ideal first, this field experienced a series of unsuccessful top down implementations, instead of a piece-by-piece, bottom-up approach.

Capabilities of Intelligent Agents

- **Ability to adapt:** Adaptation implies sensing the environment and reconfiguring in response. This can be achieved through the choice of alternative problem-solving-rules or algorithms v the discovery of problem-solving strategies, other aspects of an agent's internal construction, such as and recruiting processor or storage resources.
- **Ability to learn:** Learning may proceed through trial-and-error, then it implies a capability of introspection and analysis of behavior and success. Alternatively, learning may proceed by example and generalization, then it implies a capacity to abstract and generalize.



Nwana's Category of Software Agent

Agents and environments

So far, we have talked about agents by describing their behavior—the action that is performed after any given sequence of percepts. Now, we will have to bite the bullet and talk about how agent program the insides work. The job of AI is to design the agent program: a function that implements the agent mapping from percepts to actions. We assume this program will run on some sort of architecture computing device, which we will call the architecture.

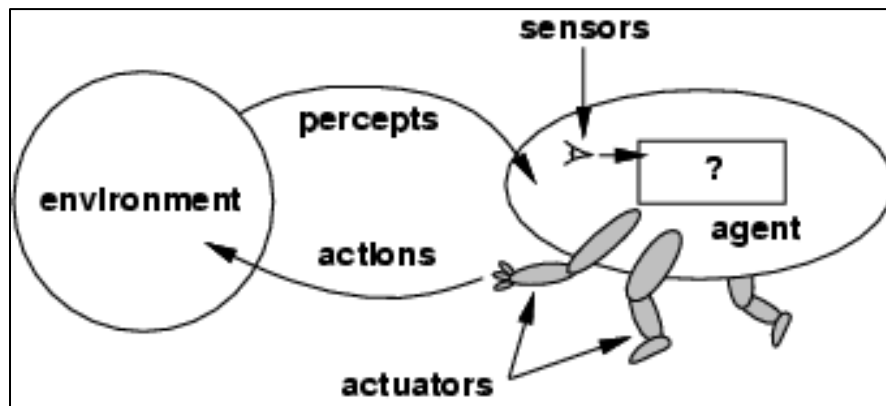
Obviously, the program we choose has to be one that the architecture will accept and run. The architecture might be a plain computer, or it might include special-purpose hardware for certain tasks, such as processing camera images or filtering audio input. It might also include software that provides a degree of insulation between the raw computer and the agent program, so that we can program at a higher level. In general, the architecture makes the percepts from the sensors available to the program, runs the program, and feeds the program's action choices to the effectors as they are generated. The relationship among agents, architectures, and programs can be summed up as follows:

The agent function maps from Percept histories to Actions:

$$[f: P^* \rightarrow A]$$

The agent program runs on the physical architecture to produce f

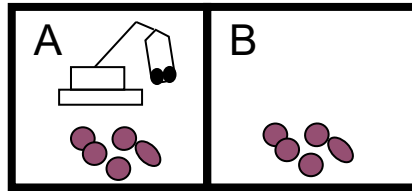
$$\text{Agent} = \text{architecture} + \text{program}$$



A Vacuum-cleaner agent

Percepts: location and contents, e.g., (A, dirty) (Idealization: locations are discrete)

Actions: move, clean, do nothing: LEFT, RIGHT, SUCK, and NOP.



Vacuum-cleaner world: agent function

Percept sequence	Action
$[A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Dirty}]$	<i>Suck</i>
$[B, \textit{Clean}]$	<i>Left</i>
$[B, \textit{Dirty}]$	<i>Suck</i>
$[A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$

function REFLEX-VACUUM-AGENT($[location, status]$) **returns** an action

if $status = \textit{Dirty}$ then **return** *Suck*
 else if $location = A$ then **return** *Right*
 else if $location = B$ then **return** *Left*



Rational Agents

An agent should strive to "do the right thing", based on what it can perceive and the actions it can perform. The right action is the one that will cause the agent to be most successful.

Performance measure: An objective criterion for success of an agent's behavior. e.g., performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed and amount of noise generated.

Intelligent agents are often described schematically as an abstract functional system, similar to a computer program. For this reason, intelligent agents are sometimes called abstract intelligent agents to distinguish them from their real-world implementations as computer systems, biological systems, or organizations. Some definitions of intelligent agents emphasize their autonomy, and so prefer the term autonomous intelligent agents. Still others consider goal-directed behavior as the essence of rationality and so prefer the term rational agent.

Rational Agent mechanism

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has. Rationality is distinct from knowing every knowledge (it is not a matter of memory capacity). Agents can perform actions in order to modify future percepts so as to obtain useful information (information gathering, exploration). An agent is autonomous if its behavior is determined by its own experience (with ability to learn and adapt).

In summary, what is rational at any given time depends on four things:

- The performance measure that defines degree of success.
- Everything that the agent has perceived so far. We will call this complete perceptual history the percept sequence.
- What the agent knows about the environment.

- The actions that the agent can perform.

This leads to a definition of an ideal rational agent: For each possible percept sequence, an ideal rational agent should do whatever action is expected to maximize its performance measure, on the basis of the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

We need to look carefully at this definition. At first glance, it might appear to allow an agent to indulge in some decidedly under intelligent activities.

For example, if an agent does not look both ways before crossing a busy road, then its percept sequence will not tell it that there is a large truck approaching at high speed. The definition seems to say that it would be OK for it to cross the road.

In fact, this interpretation is wrong on two counts. First, it would not be rational to cross the road: the risk of crossing without looking is too great. Second, an ideal rational agent would have chosen the "looking" action before stepping into the street, because looking helps maximize the expected performance. Doing actions in order to obtain useful information is an important part of rationality.

PEAS (Task Environment Description)

PEAS is an acronym for (Performance measure, Environment, Actuators, Sensors). It must first specify the setting for intelligent agent design.

PEAS EXAMPLE 1: Specifying an automated taxi driver

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

The following table illustrates examples of agent types and their PEAS descriptions.

Agent	Performance measure	Environment	Actuators	Sensors
Medical Diagnose	- Healthy patient - Minimized cost	- Patient - Hospital - Staff	- Tests - Treatments	Keyboard Entry of symptoms, findings, patient's answers
Vacuum Cleaner	- Cleanness - Efficiency - Battery life - Security	- Room - Table - Wood floor - Carpet - Various obstacles	- Wheels - Brushes - Vacuum Extractor	- Camera - Dirt detection sensor - Cliff sensor - Bump Sensor - Infrared Wall Sensor
Part - picking Robot	- Percentage of parts in correct bins.	- Conveyor belt with parts, - Bins	- Jointed Arms - Hand	- Camera - Joint angle sensors.
Refinery Controller	- Purity - Yield - Safety	- Refinery - Operators	- Valves, Pumps - Heaters - Displays	- Temperature - Pressure - Chemical Sensors
Interactive English Tutor	- Student's score on test	- Set of Students - Testing Agency	- Display of exercises, - suggestions - corrections	- Keyboard Entry

3.2 Environment Types

Fully observable (vs. partially observable): An agent's sensors give it access to the complete state of the environment at each point in time.

Deterministic (vs. stochastic): The next state of the environment is completely determined by the current state and the action executed by the agent. (If the environment is deterministic except for the actions of other agents, then the environment is strategic).

Periodic (vs. sequential): The agent's experience is divided into loosely connected modules (each period consists of the agent perceiving and then performing a single action)

Static (vs. dynamic): The environment is unchanged while an agent is studying its decision about action. (The environment is semi-dynamic if the environment itself does not change with the passage of time, but the agent's performance score does).

Discrete (vs. continuous): A limited number of distinct, clearly defined percepts and actions.

Single agent (vs. multivalent): An agent operating by itself in an environment.

The following table illustrate practical examples environments types.

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

The question that arises is that: What is the structure of the agent?

The answer is clarified in the following figure:

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

3.3 Types of Intelligent Agent

Russell & Norving (2003) describe multiple types of agents and sub-agents

- **As a first vision**
 - **Physical Agents** - A physical agent is an entity which percepts through sensors and acts through actuators (as a traditional agent).
 - **Temporal Agents** - A temporal agent may use time based stored information to offer instructions or data acts to a computer program or human being and takes program inputs percepts to adjust its next behaviors.
 - **Believable Agents** - An agent exhibiting a personality via the use of an artificial character for the interaction to practically express believes (believes against characters).

- **As a Second vision**

Also, some references classify agents into five classes based on their degree of perceived intelligence and capability to be: simple reflex agents, model-based reflex agents, goal-based agents, utility-based agents, and learning agents.

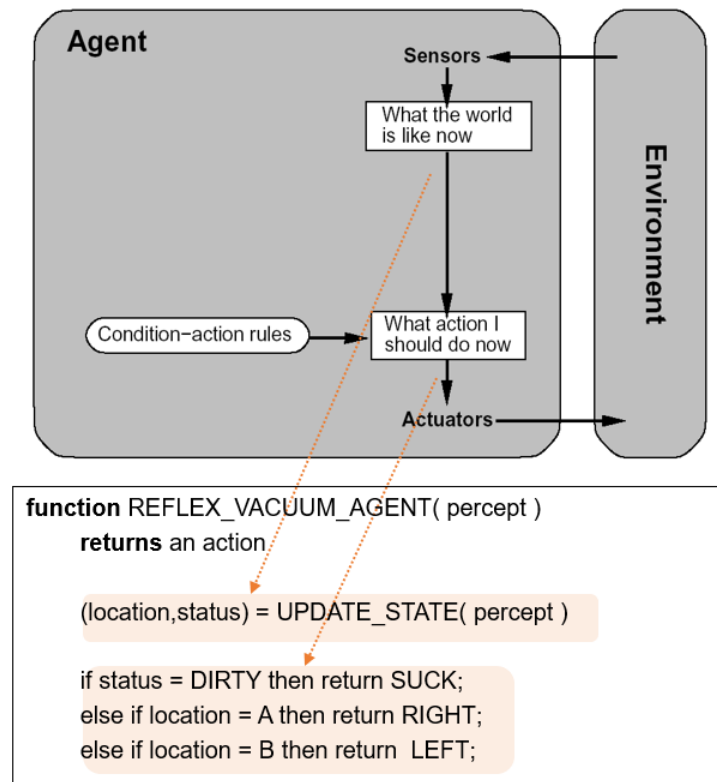
3.3.1 Simple reflex agents

Acts only on the basis of the current percept. The agent function is based on the condition-action rule:

if condition then action rule

This agent function only succeeds when the environment is fully observable. Some reflex agents can also contain information on their current state which allows them to disregard conditions whose actuators are already triggered (self-filtering).

The following figure simplify the structure of the Simple reflex agents:



Implementation and properties

- function ReflexAgentWithState (percept) returns action
 - static: state, description of current world state; rules, set of condition-action rules
 - state ← Update-State (state, percept)
 - rule ← Rule-Match (state, rules)
 - action ← Rule-Action {rule}

- return action

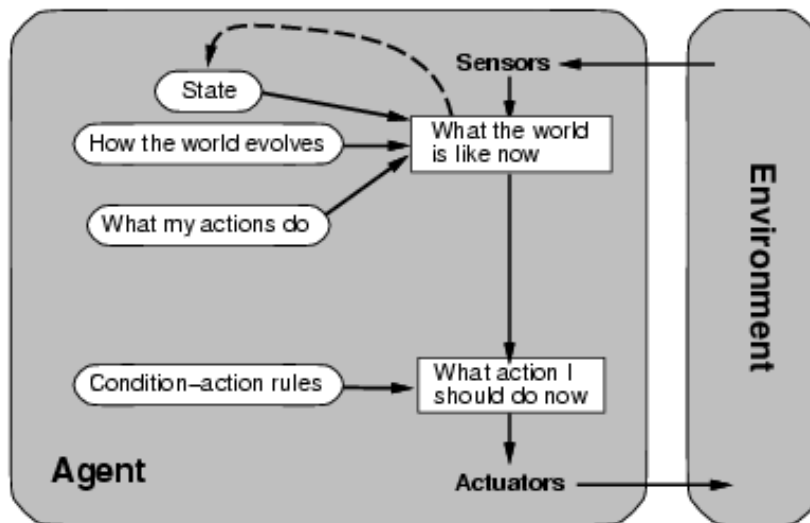
Advantages

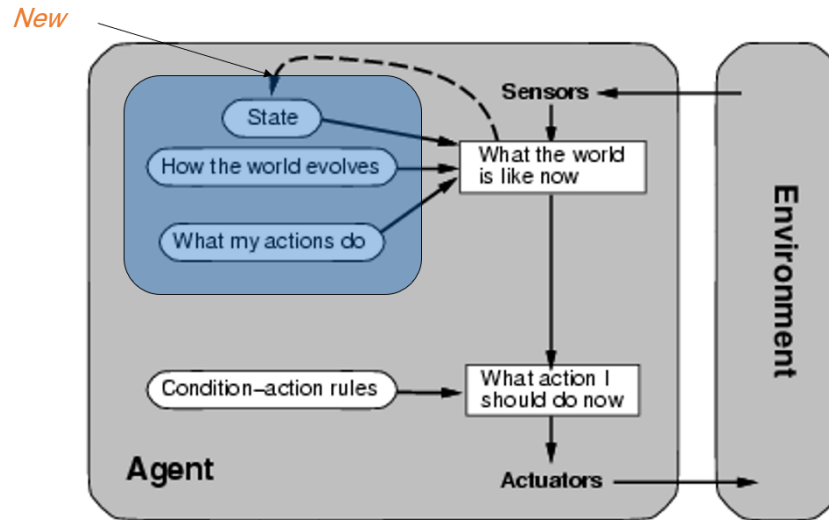
- Selection of best action based only on current state of world and rules
- Able to reason over past states of world
- Still efficient, somewhat more robust

Limitations and Disadvantages

- No way to express goals and preferences relative to goals
- Still limited range of applicability

3.3.2 Model-based reflex agents

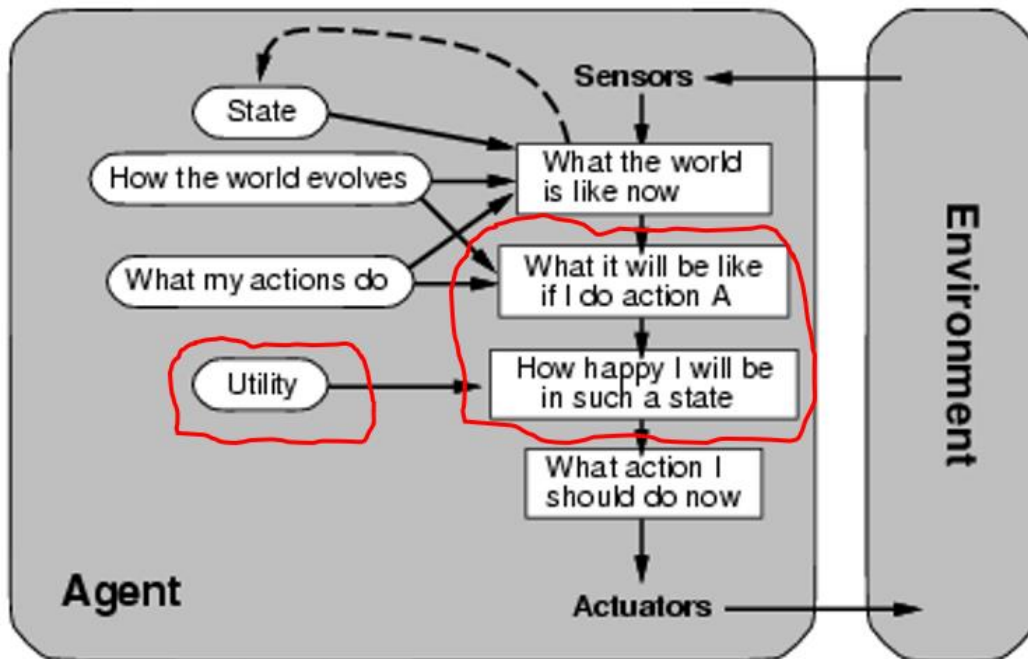




Model-based agents can handle partially observable environments. Its current state is stored inside the agent maintaining some kind of structure which describes the part of the world which cannot be seen. This behavior requires information on how the world behaves and works. This additional information completes the “World View” model.

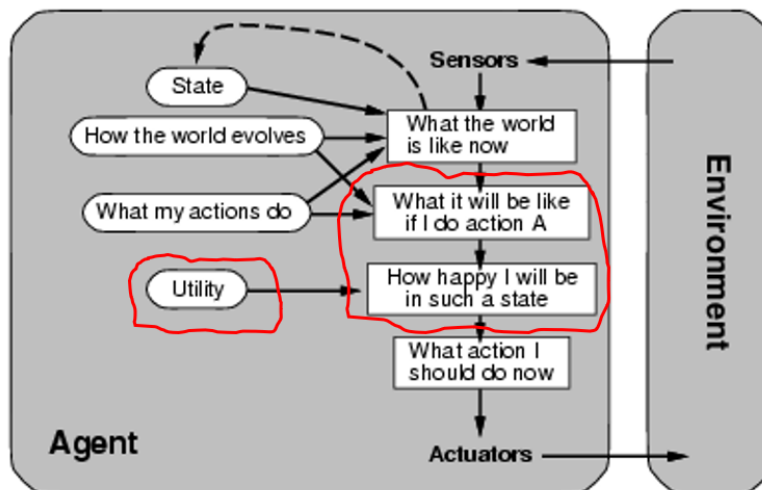
3.3.3 Utility-based agents

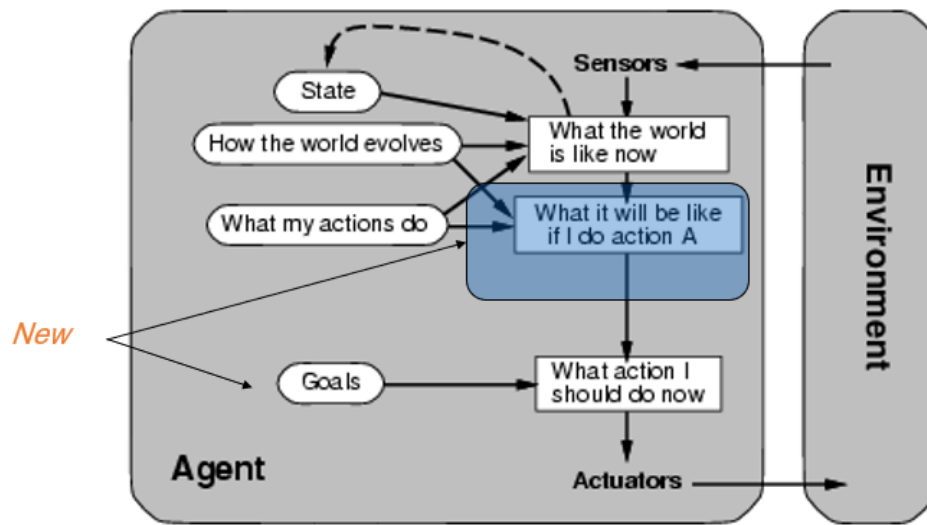
Goal-based agents only distinguish between goal states and non-goal states. It is possible to define a measure of how desirable a particular state is. This measure can be obtained through the use of a utility function which maps a state to a measure of the utility of the state.



3.3.4 Goal-based agents

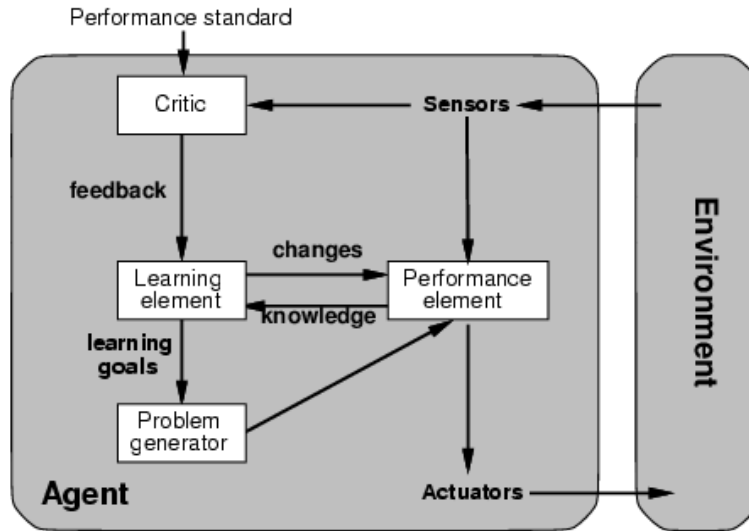
Goal-based agents are model-based agents which store information regarding situations that are desirable. This allows the agent a way to choose among multiple possibilities, selecting the one which reaches a goal state.





- Implementation and Properties
 - Functional description
 - Complete description using classical planning
 - Requires more formal specification
- Advantages
 - Able to reason over goal, intermediate, and initial states
 - Basis: automated reasoning
 - One implementation: theorem proving (first-order logic)
 - Powerful representation language and inference mechanism
- Limitations and Disadvantages
 - Efficiency limitations: can't feasible solve many general problems
 - No way to express preferences

3.3.5 Learning agents



Learning Agents Components

- New components:
 - Learning element – Makes improvements to performance element
 - Performance element – Selects actions – Previously this was the whole agent
 - Critic – Gives performance feedback to learning element – Needed because precepts don't capture performance
 - Problem generator – Suggests innovative actions

Intelligent software agents types (Third Vision)

There are only four essential types of intelligent software agents:

- Buyer agents or shopping bots
- User or personal agents
- Monitoring-and-surveillance agents
- Data Mining agents

Buyer agents (shopping bots)

Buyer agents travel around the internet retrieving information about goods and services. These agents, also known as 'shopping bots', work very efficiently for commodity products such as CDs, books, electronic components. Amazon.com is a good example of a shopping bot. The website will offer you a list of books that you

might like to buy on the basis of what you're buying now and what you have bought in the past.

Another example is used on eBay. At the bottom of its page there is a list of similar products that other customers who did the same search looked at. This is because it is assumed the user tastes are relatively similar and they will be interested in the same products. This technology is known as collaborative filtering.

User agents (personal agents)

These are intelligent agents that act on your behalf. In this category belong those intelligent agents that already perform, or will shortly perform, the following tasks:

- Check your e-mail, sort it according to the user's order of preference, and alert you when important emails arrive.
- Play computer games as your opponent or patrol game areas for you.
- Assemble customized news reports for you. There are several versions of these, including news hub and CNN.
- Find information for you on the subject of your choice.
- Fill out forms on the Web automatically for you, storing your information for future reference (e.g. news hub).
- Scan Web pages looking for and highlighting text that constitutes the "important" part of the information there.
- "Discuss" topics with you ranging from your deepest fears to sports.

Monitoring-and-surveillance (predictive) agents

Used to observe and report on equipment, usually computer systems (some sort of activity in DSS applications). The agents may keep track of company inventory levels, observe competitors' prices and relay them back to the company, watch stock manipulation by insider trading and rumors.

For example, NASA's Jet Propulsion Laboratory has an agent that monitors inventory, planning, and scheduling equipment ordering to keep costs down, as well as food storage facilities. These agents usually monitor complex computer networks that can keep track of the configuration of each computer connected to the network.

Data mining agents

This agent uses information technology to find trends and patterns in plenty of information from many different sources. The user can sort through this information in order to find whatever information they are seeking. A data mining agent operates in a data warehouse discovering information. A 'data warehouse' brings together information from lots of different sources.

"Data mining" is the process of looking through the data warehouse to find information that you can use to take action, such as ways to increase sales or keep customers who are considering defecting.

'Classification' is one of the most common types of data mining, which finds patterns in information and categorizes them into different classes. Data mining agents can also detect major shifts in trends or a key indicator and can detect the presence of new information and alert you to it.

For example, the agent may detect a degradation in the construction industry for an economy; based on this relayed information construction companies will be able to make intelligent decisions regarding the hiring/firing of employees or the purchase/lease of equipment in order to best suit their firm (USA, 2009 disaster).

3.4 Applications of Intelligent Agents

Agent software is used in a variety of contexts:

- system administration,
- information searching on the Internet,
- current awareness services,

- online shopping:
 - enrich the user's net browsing by suggesting additional hyperlinks, based on previous user behavior;
 - give added value to search engine searches;
 - to run searches on multiple databases, filter and assemble the results in an intelligent way;
 - to find the best price for a given product from online markets;
 - to engage in transactions and negotiations on behalf of the user;
 - to monitor databases on the WWW and provide a regular updating service;
 - to monitor Web site changes;
 - to compile a daily personalized newspaper;
 - to visit sites nominated by the user and download either the entire site or selected pages for offline viewing;
 - to discover other people with common interests;
 - to check and prioritize email, make decisions based on content and act on them.

3.5 More Examples on software Agents

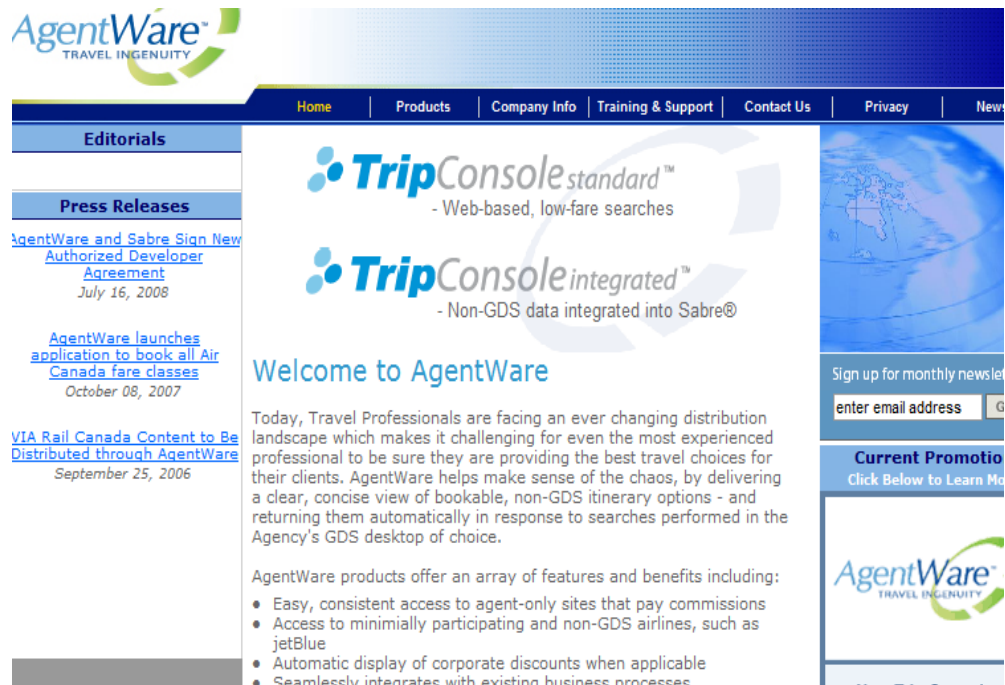
- **BotSpot** (<http://www.botspot.com/>)

It gives many examples of bots and agents.



- **Autonomy Agentware** (<http://www.agentware.com/index.html>)

Autonomy has developed a suite of products using agent technology to generate personalized, organized content from large bodies of unstructured online information. The Agentware software focuses on concepts occurring in text taking into account their frequency and context and adapts over time to improve on its recognition of the same concepts. The technology is used in products for new media publishing and knowledge management such as Content Server, Daily Briefing, Knowledge Server, Knowledge Update, and Portal-in-a-Box.

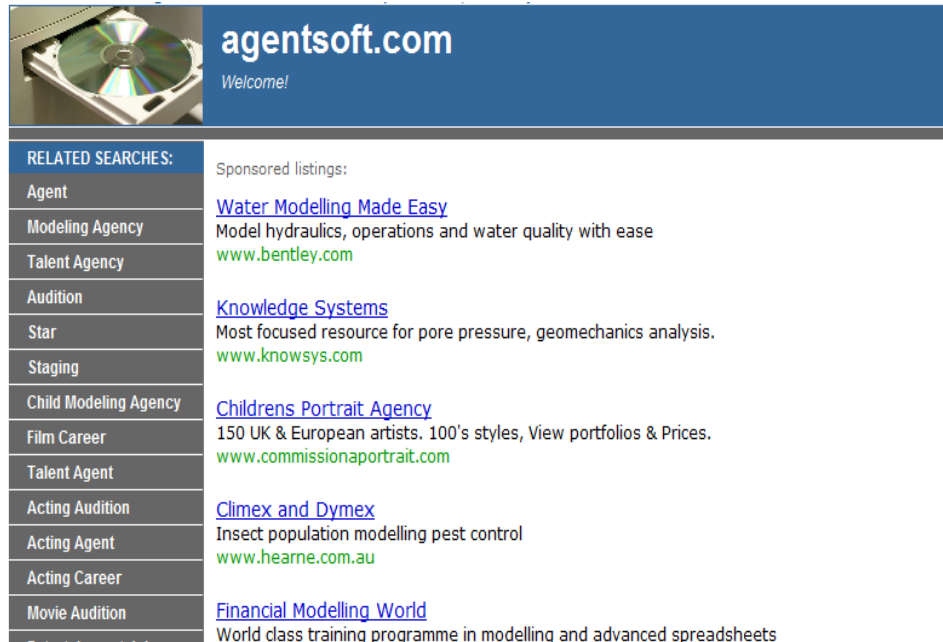


- **LiveAgent(AgentSoft)** (<http://www.agentsoft.com/liveagen.htm>)

A utility for creating personalized Java agents. The user goes through a regular Web browsing session. LiveAgent learns from this walk-through session and generates an agent that is capable of doing that activity by itself whenever the user wants.

It provides personalized entry to its information service. On registering, the user is asked to fill in a form and thereafter at login, gets a personalized version of Excite's information service. The personal profile is progressively added to by NewsTracker

which learns more about the user's favorite topics as they respond to and rate the articles it returns.



- **CareerSite** (<http://www.careersite.com/>)

It is an example of the practical application of agent technology. It uses confidential candidate profiles and concept-based searches to link employers with job seekers.



- In common with a number of other portal Live <http://live.excite.com/>
- Another example of agent technology on the Excite site is the Product Finder (<http://www.jango.com/xsh/index.dcg?>) employing Jango agent software which was designed specifically for shopping on the Net.



- eWatch at <http://www.ewatch.com/>, is an example of an agent-based alerting service available through subscription.

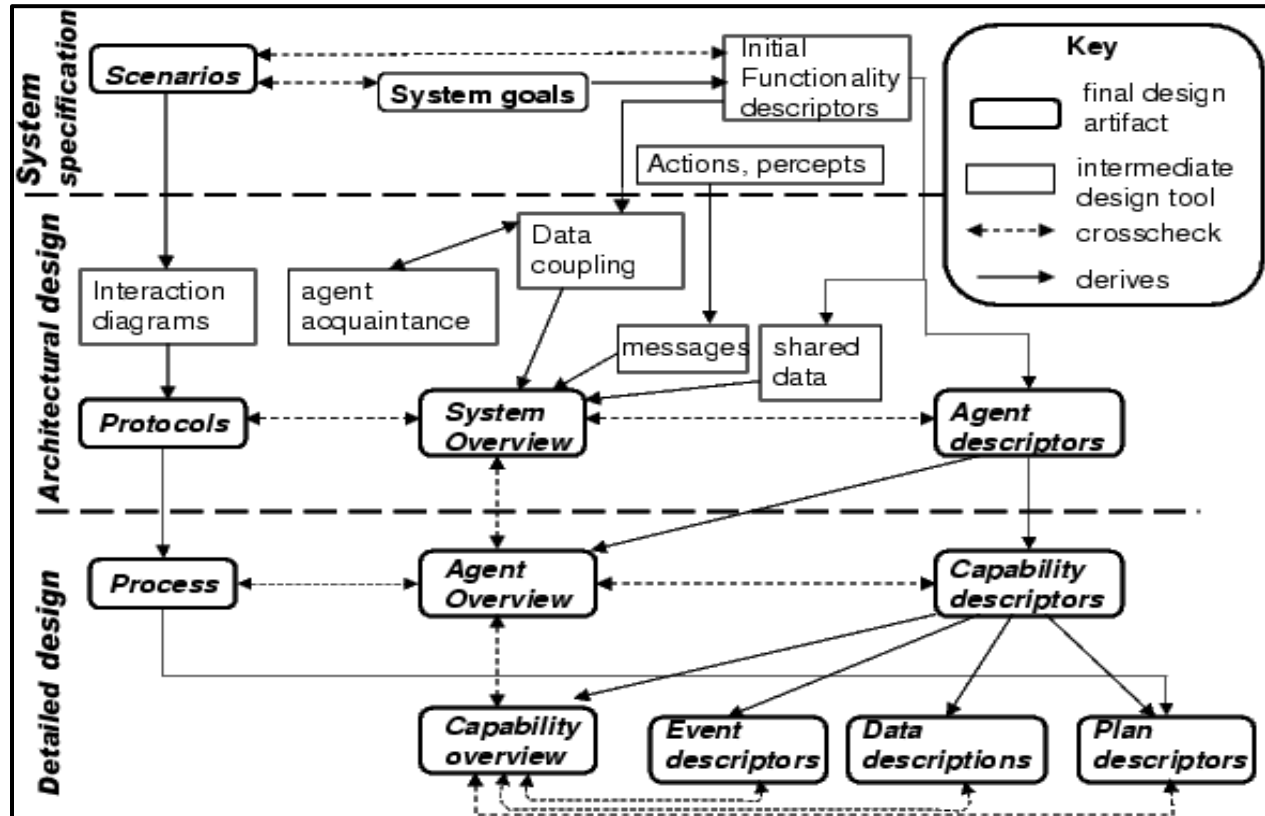


It monitors mailing lists, News groups, forums, and Web sites for information on insider trading, stock manipulation, rumors and anti-corporate activism.

- WavePhone Newscast <http://www.newscast.com/> delivers the eWatch information in real-time using push technology.



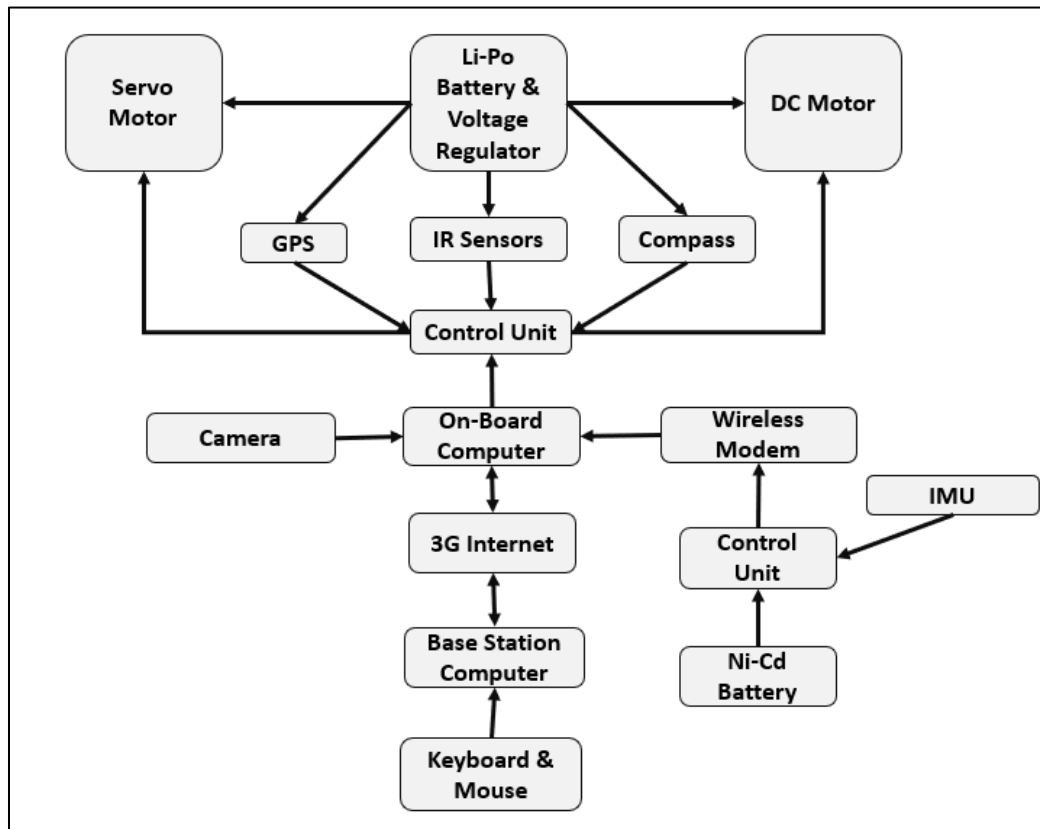
3.6 Intelligent Agent Design



Intelligent Agent Design issues

- How tasks are scheduled and how synchronization of tasks is achieved?
- How tasks are prioritized by agents?
- How agents can collaborate, or recruit resources?
- How agents can be re-instantiated in different environments, and how their internal state can be stored?
- How the environment will be probed and how a change of environment leads to behavioral changes of the agents?
- How messaging and communication can be achieved?
- What hierarchies of agents are useful (e.g. task execution agents, scheduling agents, resource providers ...)?

General Block Diagram of the Unmanned Ground Vehicle



Base Station: It's a computer system located at a remote place away from the UGV which controls it using keyboard, mouse for mode control and movement and live video feedback for monitoring the environment.

Keyboard and mouse: They are used to handle the motion of the UGV and the movement of the turret for wide angle vision.

3G Internet: Communication medium for system to system interaction so as to control the UGV wirelessly.

On-board system: A computer system placed on the UGV itself which receives the commands and delivers it to the control unit.

Camera: An image acquiring device which provides the video required for the UGV vision.

Control Unit: It's the Arduino microcontroller which receives signals from the user and other sensors and performs tasks such as turret movement and UGV movement.

GPS Unit: A navigation system used in the autonomous mode for obtaining location co-ordinates.

Compass: To acquire the direction to which the UGV is facing

IR Sensors: Infrared sensors used in the obstacle avoidance mechanism incorporated into the autonomous mode.

Servo Motor: They are used to control the direction turn of the UGV and the 2-axis movement of the turret.

DC Motor: these are used mainly for the UGV movement.

Li-PO Battery and voltage regulator: the power source supplying the entire UGV with voltage regulation to provide optimum power ratings.

Wireless modem: Zigbee to provide wireless data transfer for the ArmCon mode (ZigBee is a wireless technology designed to connect simple high-tech devices for useful purposes.)

IMU: An inertial measurement unit which tracks the orientation of the hand used for hand Gesture control (ArmCon mode).

Ni-Cd battery: Used for powering up the Control Unit, Zigbee and the IMU.



RoboCar autonomous Vehicle

A British-made driverless sports car has set the record for the world's fastest autonomous vehicle, published in the 2020 book of Guinness World Records. Robocar sets world record for fastest driverless vehicle with top speed of 175 mph. The car completed two runs at Elvington Airfield in York, which were averaged to produce an official top speed of 282.42kph (175.49mph). There was no official benchmark to beat, but Guinness World Records set 257kph (160mph) as the unofficial speed that Robocar needed to surpass.

For More Knowledge

automated Taxi Driver\Automated Taxi Driver Case study_1.docx

automated Taxi Driver\UGV case study_2.pdf

automated Taxi Driver\Thesis Unmanned Ground Vehicle UG thesis.pdf

automated Taxi Driver\Paper UGV design and implementation prototype.pdf

Revision Sheet # 3

Assignment # 6

Prepare a survey on **planning** including:

definitions ,theories, applications ,problems and research areas and inter-relations with the AI Main Topics (reasoning, knowledge, planning , learning, communication, perception , ability to move and manipulate object).

Assignment # 7

Consider the vacuum-world problem:

- a. Which of the algorithms defined in this chapter would be appropriate for this problem? Should the algorithm use tree search or graph search?
- b. Apply your chosen algorithm to compute an optimal sequence of actions for a 3×3 world whose initial state has dirt in the three top squares and the agent in the center.
- c. Construct a search agent for the vacuum world, and evaluate its performance in a set of 3×3 worlds with probability 0.2 of dirt in each square. Include the search cost as well as path cost in the performance measure, using a reasonable exchange rate.
- d. Compare your best search agent with a simple randomized reflex agent that sucks if there is dirt and otherwise moves randomly.
- e. Consider what would happen if the world were enlarged to $n \times n$.

CHAPTER 4

Searching

4.1 Problem Solving Agents

In this chapter, we describe one kind of goal-based agent called a problem-solving agent. Problem-solving agents decide what to do by finding sequences of actions that lead to desirable states. We discuss informally how the agent can formulate an appropriate view of the problem it faces. The problem type that results from the formulation process will depend on the knowledge available to the agent: principally, whether it knows the current state and the outcomes of actions.

We then define more precisely the elements that constitute a "problem" and its "solution," and give several examples to illustrate these definitions. Given precise definitions of problems, it is relatively straightforward to construct a search process for finding solutions. We cover six different search strategies and show how they can be applied to a variety of problems. Also we will cover search strategies that make use of more information about the problem to improve the efficiency of the search process.

The pseudo code for this approach is illustrated in the following algorithm:

```

function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  static: seq, an action sequence, initially empty
           state, some description of the current world state
           goal, a goal, initially null
           problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then do
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
  action ← FIRST(seq)
  seq ← REST(seq)
  return action

```

Example: Egypt

- On holiday in Egypt; currently in Cairo.
- Flight leaves tomorrow from Cairo.
- Formulate goal: be in Alex.
- Formulate problem: (states: various cities and actions: drive between cities).
- Find solution: Sequence of cities, e.g., Banha, Tanta, Kafr Alziat, Damanhur and Alex.

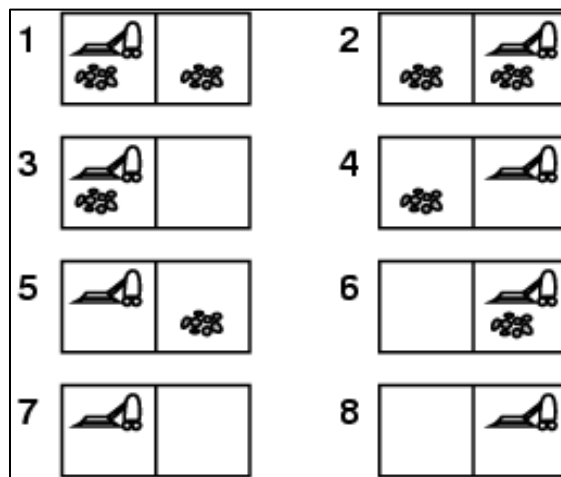


4.2 Search Problem Types

- **Deterministic, fully observable → single-state problem**
 - Agent knows exactly which state it will be in; solution is a sequence.
- **Deterministic, non-observable → sensor less problem (conformant problem)**
 - Agent may have no idea where it is; solution is a sequence.
- **Nondeterministic and/or partially observable → event problem**
 - percepts provide new information about current state
 - often interleave search, execution
- **Unknown state space → exploration problem**

Example: vacuum world

Let the world contain just two locations. Each location may or may not contain dirt, and the agent may be in one location or the other. There are 8 possible world states, as shown in the next figure. The agent has three possible actions in this version of the vacuum world: Left, Right, and Suck. Assume, for the moment, that sucking is 100% effective. The goal is to clean up all the dirt. That is, the goal is equivalent to the state set $\{7, 8\}$.



First, suppose that the agent's sensors give it enough information to tell exactly which state it is in (i.e., the world is accessible); and suppose that it knows exactly what each of its actions does. Then it can calculate exactly which state it will be in after any sequence of actions. For example, if its initial state is 5, then it can calculate that the action sequence [Right, Suck] will get to a goal state. This is the simplest case, which we call a single-state problem.

Second, suppose that the agent knows all the effects of its actions, but has limited access to the world state. For example, in the extreme case, it may have no sensors at all. In that case, it knows only that its initial state is one of the set $\{1, 2, 3, 4, 5, 6, 7, \text{ and } 8\}$. One might suppose that the agent's predicament is hopeless, but in fact it can do quite well. Because it knows what its actions do, it can, for example, calculate that the action Right will cause it to be in one of the states $\{2, 4, 6, 8\}$.

In fact, the agent can discover that the action sequence [Right, Suck, Left, Suck] is guaranteed to reach a goal state no matter what the start state. To summarize: when the world is not fully accessible, the agent must reason about sets of states that it might get to, rather than single states. We call this a multiple-state problem. Although it might seem different, the case of ignorance about the effects of actions can be treated similarly. Suppose, for example, that the environment appears to be nondeterministic in that it obeys Murphy's Law: the so-called Suck action sometimes deposits dirt on the carpet but only if there is no dirt there already? For example, if the agent knows it is in state 4, then it knows that if it sucks, it will reach one of the states {2, 4}. For any known initial state, however, there is an action sequence that is guaranteed to reach a goal state. Sometimes ignorance prevents the agent from finding a guaranteed solution sequence. Suppose, for example, that the agent is in the Murphy's Law world, and that it has a position sensor and a local dirt sensor, but no sensor capable of detecting dirt in other squares. Suppose further that the sensors tell it that it is in one of the states {1, 3}. The agent might formulate the action sequence [Suck, Right, and Suck].

Sucking would change the state to one of {5, 7}, and moving right would then change the state to one of {6, 8}. If it is in fact state 6, then the action sequence will succeed, but if it is state 8, the plan will fail. If the agent had chosen the simpler action sequence [Suck], it would also succeed some of the time, but not always.

It turns out there is no fixed action sequence that guarantees a solution to this problem. Obviously, the agent does have a way to solve the problem starting from one of {1, 3}: first suck, then move right, then suck only if there is dirt there. Thus, solving this problem requires sensing during the execution phase.

Notice that the agent must now calculate a whole tree of actions, rather than a single action sequence. In general, each branch of the tree deals with a possible contingency that might arise. For this reason, we call this a contingency problem. Many problems in the real, physical world are contingency problems, because exact prediction is

impossible. For this reason, many people keep their eyes open while walking around or driving. Single-state and multiple-state problems can be handled by similar search techniques.

Contingency problems, on the other hand, require more complex algorithms, which we cover in next chapters. They also lend themselves to a somewhat different agent design, in which the agent can act before it has found a guaranteed plan. This is useful because rather than considering in advance every possible contingency that might arise during execution, it is often better to actually start executing and see which contingencies do arise. The agent can then continue to solve the problem given the additional information. This type of interleaving of search and execution is also covered in the following chapters.

- **Single-state (Deterministic, fully observable),**
 - start in #5.
 - Solution: *[Right, Suck]*
- **Sensor less (Non-observable),**
 - start in {1,2,3,4,5,6,7,8} e.g., Right goes to {2,4,6,8}
 - Solution: *[Right,Suck,Left,Suck]*
 - Event
 - Nondeterministic: Suck may dirty a clean carpet
 - Partially observable: location, dirt at current location.
 - Percept: [L, Clean], i.e., start in #5 or #7
 - Solution: *[Right, if dirt then Suck]*

4.3 Search problem definition

A problem is defined by four items:

1. initial state e.g., "at Cairo"
2. actions or successor function $S(x)$ = set of action–state pairs
 - e.g., $S(\text{Cairo}) = \{ \langle \text{Cairo} \rightarrow \text{Banha}, \text{Banha} \rangle, \dots \}$

3. goal test, can be

- explicit, e.g., $x = \text{"at Alex"}$
- implicit, e.g., $\text{Checkmate}(x)$

4. path cost (additive)

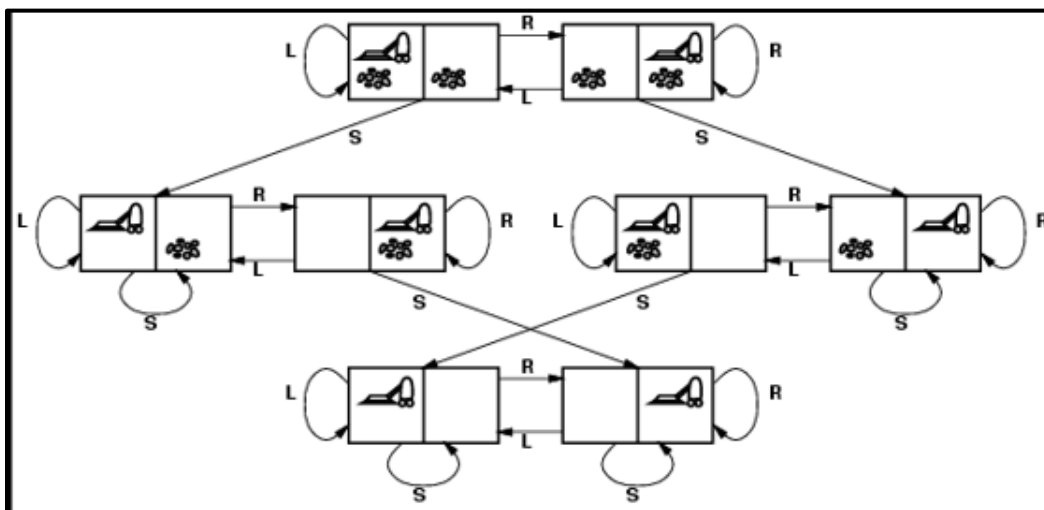
- e.g., sum of distances, number of actions executed.
- $c(x,a,y)$ is the step cost, assumed to be ≥ 0

A solution is a sequence of actions leading from the initial state to a goal state.

Selecting a state space

- Real world is absolutely complex $\rightarrow$ state space must be abstracted for problem solving.
- (Abstract) state = set of real states
- (Abstract) action = complex combination of real actions e.g., "Cairo $\rightarrow$ Banha " represents a complex set of possible routes, detours, rest stops, etc.
- For guaranteed realizability, any real state "in Cairo" must get to some real state "in Banha"
- (Abstract) solution = set of real paths that are solutions in the real world
- Each abstract action should be "easier" than the original problem.

Vacuum world state space graph

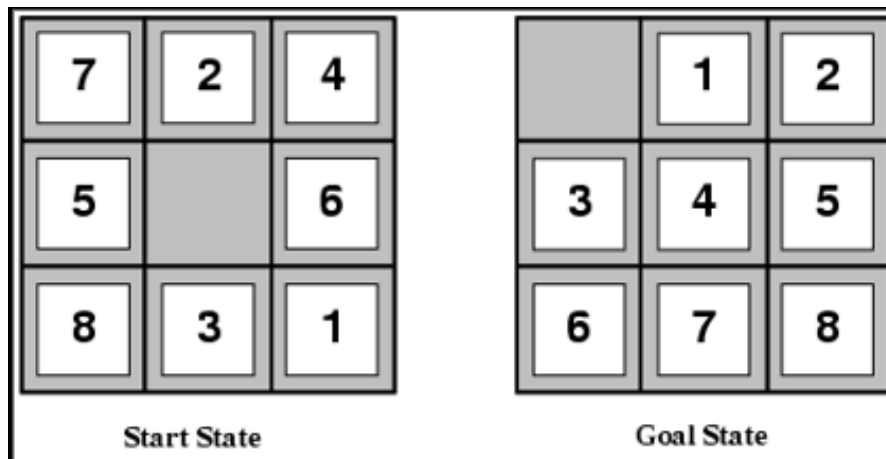


- states: integer dirt and robot location
- actions: *Left, Right, Suck*
- goal test: no dirt at all locations
- path cost: 1 per action

4.4 Search Problem Examples

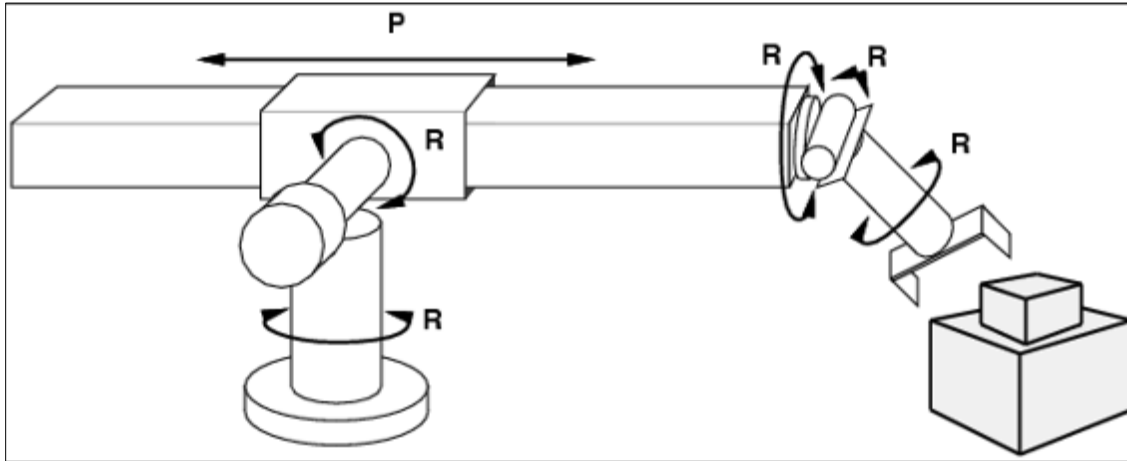
Example 1: The 8-puzzle

- states: locations of tiles
- actions: move blank left, right, up, down
- goal test: goal state (given): blank at left corner
- path cost: 1 per move



Example 2: robotic assembly

- states: real-valued coordinates of robot joint angles parts of the object to be assembled
- actions: continuous motions of robot joints
- goal test: complete assembly
- path cost: time to execute



4.5 Search Strategies

A search strategy is defined by picking the order of node expansion. Strategies are evaluated along the following dimensions:

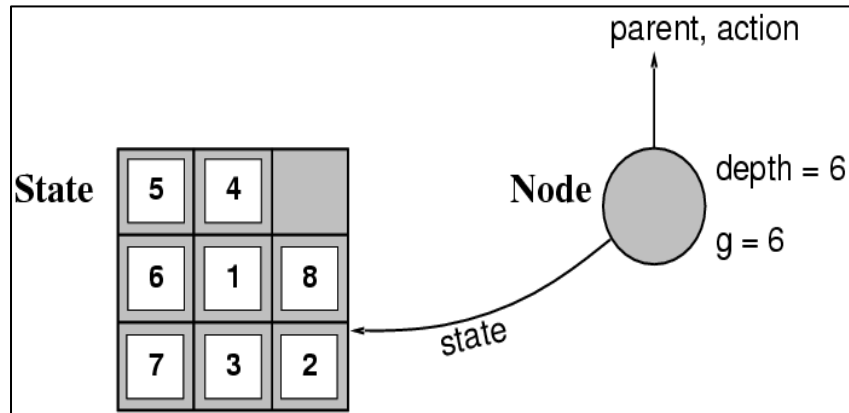
- **Completeness:** does it always find a solution if one exists?
- **time complexity:** number of nodes generated
- **space complexity:** maximum number of nodes in memory
- **Optimality:** does it always find a least-cost solution?

Time and space complexity (for comparison purpose) are measured in terms of:

- $b \rightarrow$ maximum branching factor of the search tree
- $d \rightarrow$ depth of the least-cost solution
- $m \rightarrow$ maximum depth of the state space

Implementation (states vs. nodes)

A state is a (representation of) a physical configuration. A node is a data structure constituting part of a search tree includes different fields as: state, parent node, action, path cost $g(x)$, depth



The **Expand** function creates new nodes, filling in the various fields and using the **SuccessorFn** of the problem to create the corresponding states.

4.6 Types of uninformed search techniques

Uninformed search strategies use only the information available in the problem definition:

- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening search

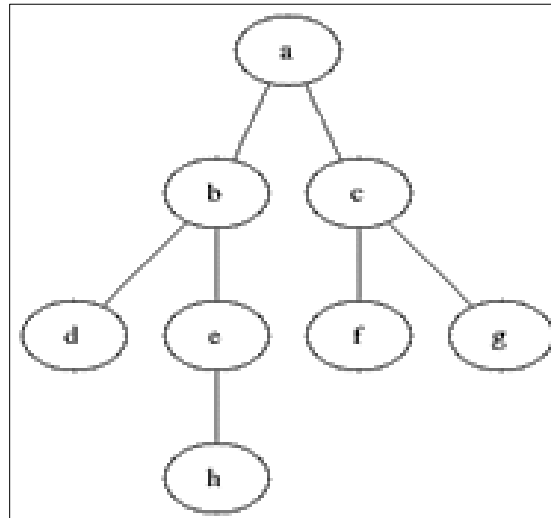
4.6.1 Breadth-first search

It is an uninformed search method that aims to expand and examine all nodes of a graph or combinations of sequence by systematically searching through every solution. It exhaustively searches the entire graph or sequence without considering the goal until it finds it. It does not use a heuristic (math function). All child nodes obtained by expanding a node are added to a FIFO queue.

In typical implementations, nodes that have not yet been examined for their neighbors are placed in some container (such as a queue or linked list) called "open" and then once examined are placed in the container "closed".

Breadth-first search Algorithm

1. Put the root node on the queue.
2. Pull a node from the beginning of the queue and examine it.
3. If the searched element is found in this node, quit the search and return a result.
4. Otherwise push all the (so-far-unexamined) successors (the direct child nodes) of this node into the end of the queue, if direct child nodes) of this node into the end of the queue, if there are any.
5. If the queue is empty, every node on the graph has been examined -- quit the search and return "not found".
6. Repeat from Step 2.



Applications of BFS

Breadth-first search can be used to solve many problems in graph theory, for example:

- Finding all connected components in a graph.
- Finding all nodes within one connected component
- Copying Collection, Cheney's algorithm
- Finding the shortest path between two nodes u and v (in an unweighted graph)
- Finding the shortest path between two nodes u and v (in a weighted graph)

Space Complexity

Since all of the nodes of a level must be saved until their child nodes in the next level have been generated, the space complexity is proportional to the number of nodes at the deepest level. Given a branching factor b and graph depth d the asymptotic space complexity is the number of nodes at the deepest level, $O(b^d)$.

When the number of vertices and edges in the graph are known ahead of time, the space complexity can also be expressed as $O(|E| + |V|)$ where $|E|$ is the cardinality of the set of edges (the number of edges), and $|V|$ is the cardinality of the set of vertices.

Time Complexity

Since in the worst case breadth-first search has to consider all paths to all possible nodes, the time complexity of breadth-first search is $b + b^2 + b^3 + \dots + b^d$ which asymptotically approaches $O(b^d)$.

The time complexity can also be expressed as $O(|E| + |V|)$ since every vertex and every edge will be explored in the worst case.

Completeness

Breadth-first search is complete. This means that if there is a solution, breadth-first search will find it regardless of the kind of graph. However, if the graph is infinite and there is no solution breadth-first search will diverge.

Optimality

For unit-step cost, breadth-first search is optimal. In general, breadth-first search is not optimal since it always returns the result with the fewest edges between the start node and the goal node:

- If the graph is a weighted graph, and therefore has costs associated with each step, a goal next to the start does not have to be the cheapest goal available. This problem is solved by improving breadth-first search to uniform-cost search which considers the path costs.

- Nevertheless, if the graph is not weighted, and therefore all step costs are equal, breadth-first search will find the nearest and the best solution.

4.6.2 Uniform-cost search

In computer science, uniform-cost search (UCS) is a tree search algorithm used for traversing or searching a weighted tree, tree structure, or graph. Intuitively, the search begins at the root node. The search continues by visiting the next node which has the least total cost from the root. Nodes are visited in this manner until a goal state is reached.

Typically, the search algorithm involves expanding nodes by adding all unexpanded neighboring nodes that are connected by directed paths to a priority queue. The uniform-cost search is complete and optimal if the cost of each step exceeds some positive bound ϵ .

Dijkstra's algorithm, can be regarded as a variant of uniform-cost search, where there is no goal state and processing continues until all nodes have been removed from the priority queue. As in Dijkstra's algorithm, UCS guarantees that (if all edge weights are non-negative) the shortest path to a particular node has been found once the node is extracted from the priority queue.

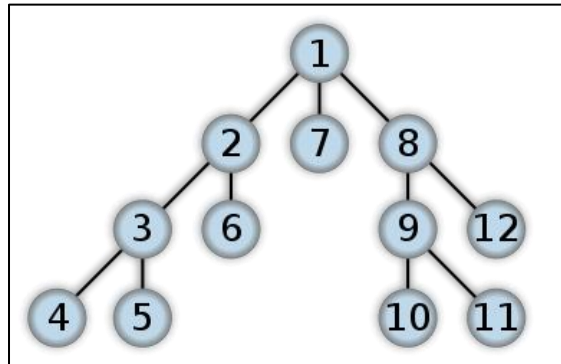
Breadth-first search (BFS) is a special case of uniform-cost search (UCS) when all edge costs are positive and identical:

- Where BFS first visits the node with the shortest path length (number of nodes) from the root node,
- UCS first visits the node with the shortest path costs (sum of edge weights) from the root node.

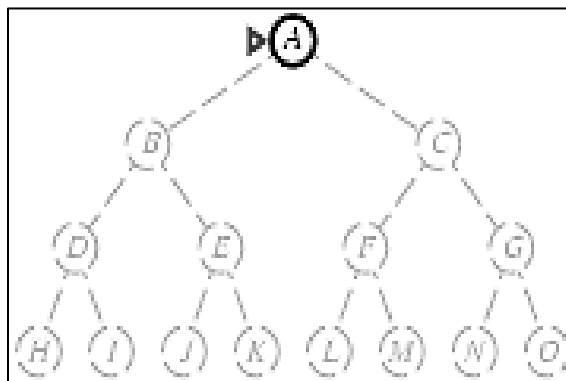
4.6.3 Depth-first search

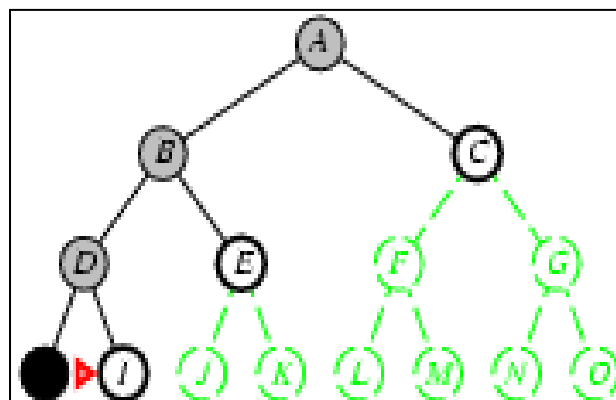
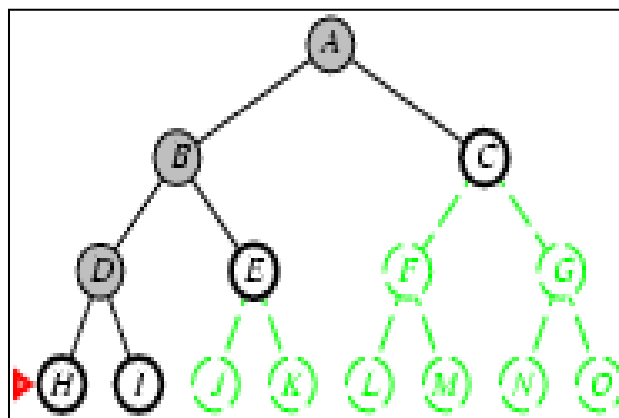
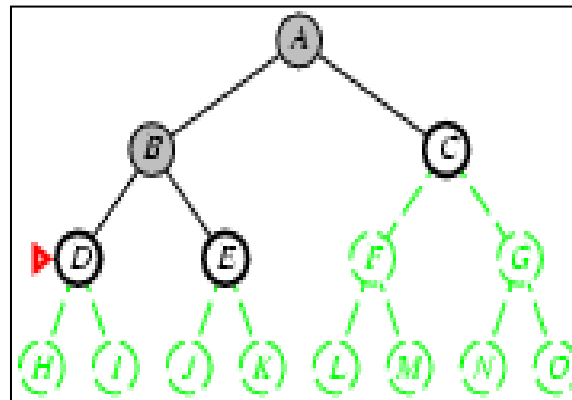
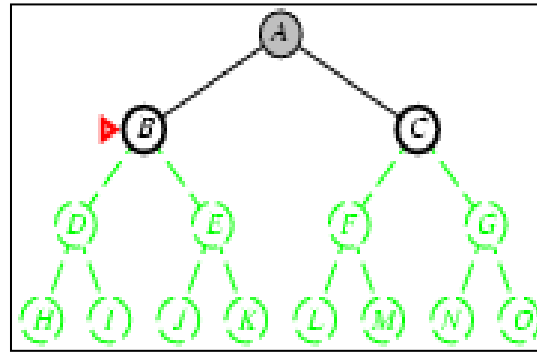
Depth-first search (DFS) is an algorithm for traversing or searching a tree, tree structure, or graph. One starts at the root (selecting some node as the root in the graph case) and explores as far as possible along each branch before backtracking.

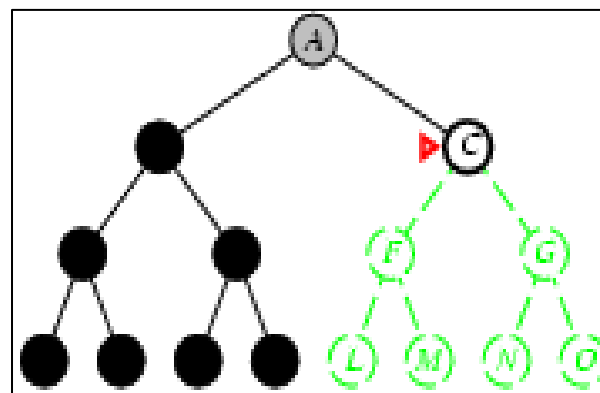
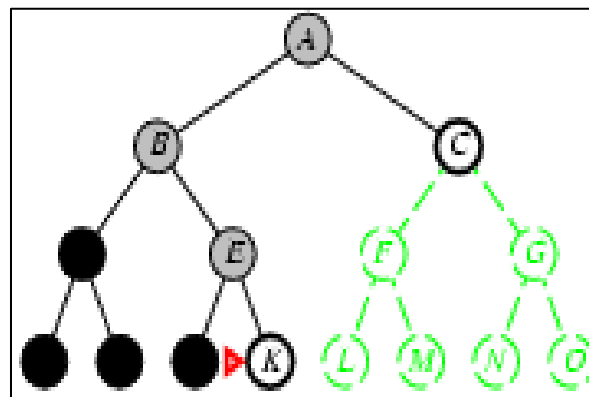
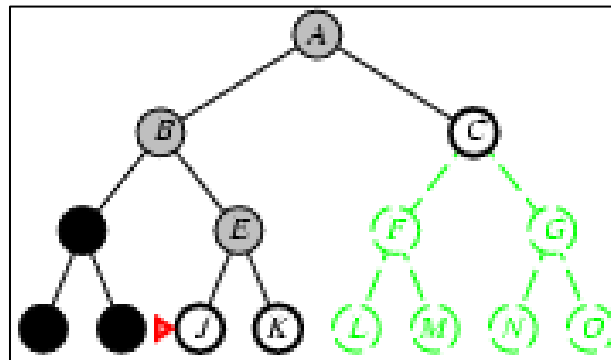
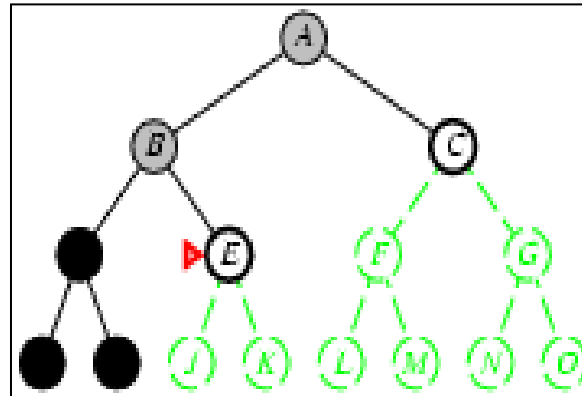
Formally, DFS is an uninformed search that progresses by expanding the first child node of the search tree that appears and thus going deeper and deeper until a goal node is found, or until it hits a node that has no children. Then the search backtracks, returning to the most recent node it hadn't finished exploring. In a non-recursive implementation, all freshly expanded nodes are added to a LIFO stack for exploration.

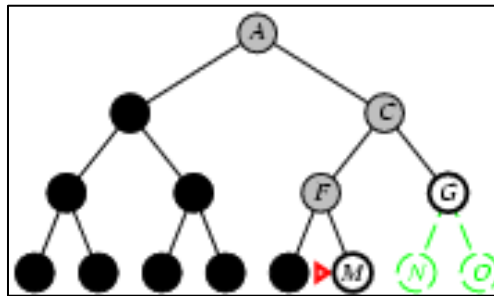
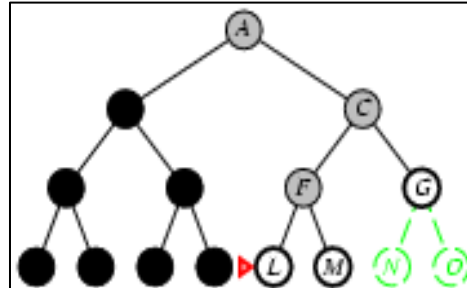
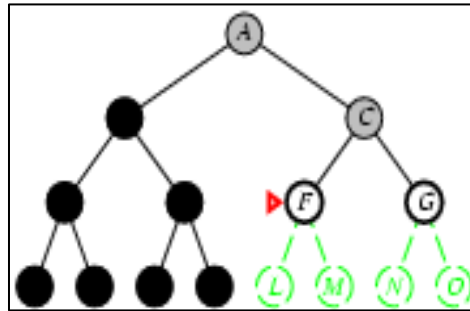


- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front









Summarized Properties of DFS

- Complete? No: fails in infinite-depth spaces, spaces with loops
Modify to avoid repeated states along path → complete in finite spaces
- Time? $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space? $O(bm)$, i.e., linear space!
- Optimal? No

Depth-first search Applications

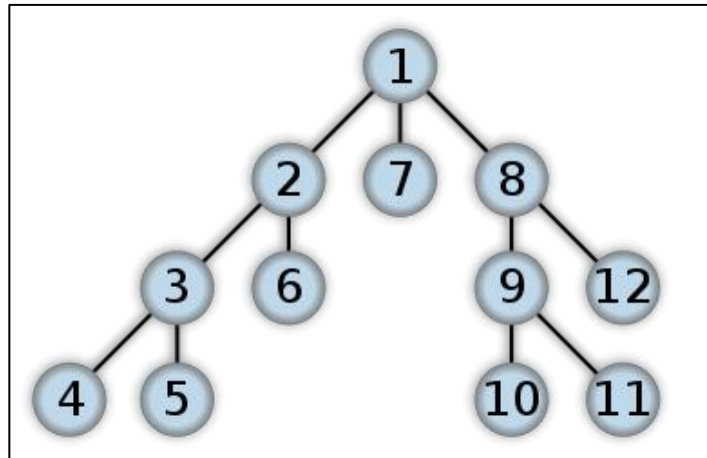
Some algorithms where DFS is used:

- Finding connected components.
- Topological sorting.

- Finding strongly connected components.
- Solving puzzles with only one solution, such as mazes.

Exercise

- In the figure below, implement by any high level language; a code for the depth first search algorithm.
- Describe how to look up for the node # 9
- Repeat step (a) for the breadth first search algorithm
- Repeat step (b) with breadth first search
- Compare time complexity and space complexity in (b) and (d).



4.6.4 Depth-limited search

It is an uninformed search. It works exactly like depth-first search but avoids its drawbacks regarding completeness by imposing a maximum limit on the depth of the search. The advantage is that even if the search could still expand a vertex beyond that depth, it will not do so and thereby it will not follow infinitely deep paths or get stuck in cycles. Depth-limited search will find a solution if it is within the depth limit, which guarantees at least completeness on all graphs.

Depth-limited search Algorithm

Depth-first search with depth limit l , i.e., nodes at depth l have no successors.

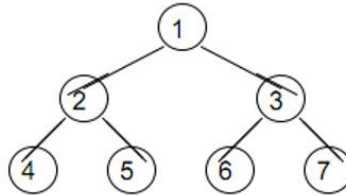
Recursive implementation:

```

function DEPTH-LIMITED-SEARCH( problem, limit) returns soln/fail/cutoff
    RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns soln/fail/cutoff
    cutoff-occurred?  $\leftarrow$  false
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
    else if DEPTH[node] = limit then return cutoff
    else for each successor in EXPAND(node, problem) do
        result  $\leftarrow$  RECURSIVE-DLS(successor, problem, limit)
        if result = cutoff then cutoff-occurred?  $\leftarrow$  true
        else if result  $\neq$  failure then return result
    if cutoff-occurred? then return cutoff else return failure
  
```

DLS: Expand Order



Evolution of the queue (**bold**=expanded and then added):

(*<depth>*, *<node>*); **Depth limit = 3**

1. [(d1, 1)] : initial state
2. **[(d2,2)][(d2,3)]** : pop 1 and push 2 and 3
3. **[(d3,4)][(d3,5)]** [(d2, 3)] : pop 2 and push 4 and 5
4. [(d3, 5)] [(d2, 3)] : pop 4, cannot expand it further
5. [(d2, 3)] : pop 5, cannot expand it further
6. **[(d3,6)][(d3,7)]** : pop 3, and push 6, 7

1. Determine the vertex where the search should start and assign the maximum search depth
2. Check if the current vertex is the goal state
 - If not: Do nothing ,If yes: return
3. Check if the current vertex is within the maximum search depth:
 - If not: Do nothing ,If yes:
 - i. Expand the vertex and save all of its successors in a stack

- ii. Call DLS recursively for all vertices of the stack and go back to Step 2

Depth-limited search properties

- **Space complexity:** Since depth-limited search internally uses depth-first search, the space complexity is equivalent to that of normal depth-first search $O(bm)$.
- **Time complexity:** Since depth-limited search internally uses depth-first-search, the time complexity is equivalent to that of normal depth-first search, and is: $O(|V| + |E|)$ where $|V|$ stands for the number of vertices and $|E|$ for the number of edges in the explored graph.
- **Completeness:** Even though depth-limited search cannot follow infinitely long paths, nor can it get stuck in cycles, in general the algorithm is not complete since it does not find any solution that lies beyond the given search depth. But if you choose the maximum search depth to be greater than the depth of a solution the algorithm becomes complete.
- **Optimality:** Depth-limited search is not optimal. It still has the problem of depth-first search that it first explores one path to its end, thereby possibly finding a solution that is more expensive than some solution in another path.

4.6.5 Iterative deepening search (DLS by increasing limit)

Iterative deepening depth-first search (IDDFS) is a state space search strategy in which a depth-limited search is run repeatedly, increasing the depth limit with each iteration until it reaches d , the depth of the shallowest goal state. On each iteration, IDDFS visits the nodes in the search tree in the same order as depth-first search, but the cumulative order in which nodes are first visited.

Iterative deepening search properties

Space complexity: is $O(bd)$, where b is the branching factor and d is the depth. Since iterative deepening visits states multiple times, it may seem wasteful, but it turns out to be not so costly, since in a tree most of the nodes are in the bottom level, so it does not matter much if the upper levels are visited multiple times.

Time complexity of IDDFS in well-balanced trees works out to be the same as Depth-first search: $O(b^d)$. In an iterative deepening search, the nodes on the bottom level are expanded once, those on the next to bottom level are expanded twice, and so on, up to the root of the search tree, which is expanded: $d + 1$ times. So, the total number of expansions in an iterative deepening search is:

$$(d + 1)1 + (d)b + (d - 1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + b^d$$

For $b = 10$ and $d = 5$ the number is:

$$6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$$

Iterative deepening search Vs. depth-limited search

Number of nodes generated in a depth-limited search to depth d with branching factor b :

$$N_{DLS} = b^0 + b^1 + b^2 + \dots + b^{d-2} + b^{d-1} + b^d$$

Number of nodes generated in an iterative deepening search to depth d with branching factor b :

$$N_{IDS} = (d+1)b^0 + d b^1 + (d-1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + 1b^d$$

Example: $b = 10$, $d = 5$,

$$N_{DLS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$$

$$N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$$

$$\text{Overhead} = (123,456 - 111,111)/111,111 = 11\%$$

Altogether, an iterative deepening search from depth 1 to depth d expands only about 11% more nodes than a single breadth-first or depth-limited search to depth d , when $b = 10$. The higher the branching factor, the lower the overhead of repeatedly expanded states, but even when the branching factor is 2, iterative deepening search only takes about twice as long as a complete breadth-first search.

This means that the time complexity of iterative deepening is still $O(b^d)$, and the space complexity is $O(bd)$. In general, iterative deepening is the preferred search method when there is a large search space and the depth of the solution is not known.

Iterative deepening search algorithm

```

function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or fail-
ure
  inputs: problem, a problem
  for depth ← 0 to ∞ do
    result ← DEPTH-LIMITED-SEARCH(problem, depth)
    if result ≠ cutoff then return result
  
```

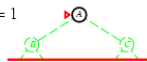
Iterative deepening search $l = 0$

Limit = 0



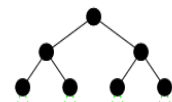
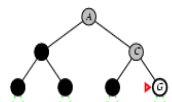
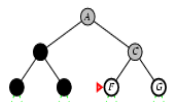
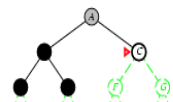
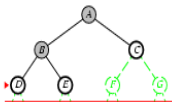
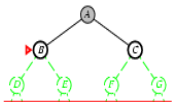
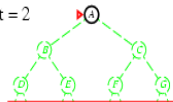
Iterative deepening search $l = 1$

Limit = 1

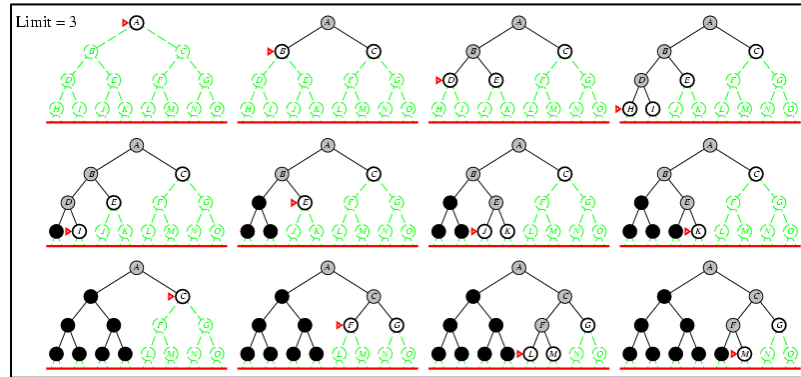


Iterative deepening search $l = 2$

Limit = 2



Iterative deepening search $l = 3$



Properties of iterative deepening search

- Complete? Yes
- Time? $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step cost = 1

4.7 Types of Informed Search Techniques

4.7.1 Best-first search

It is a search algorithm which explores a graph by expanding the most promising node chosen according to some rule.

Judea Pearl described best-first search, as the simplest informed search algorithm; which estimate the promise of node n by a "heuristic evaluation function $f(n)$ which, in general, may depend on:

- the description of n ,
- the description of the goal,
- the information gathered by the search up to that point
- Any extra knowledge about the problem domain."

Other authors have used "best-first search" to refer specifically to a search with a heuristic that attempts to predict how close the end of a path is to a solution, so that paths which are judged to be closer to a solution are extended first. This

specific type of search is called greedy best-first search. Efficient selection of the current best candidate for extension is typically implemented using a priority queue as a data structure.

Examples of best-first search algorithms include the A* search algorithm, and in turn, Dijkstra's algorithm (which can be considered a specialization of A*). Best-first algorithms are often used for path finding in combinatorial search.

Best-first search algorithm

1. Start with OPEN containing just the initial state
2. Until a goal state is found or there is no node left on OPEN do:
 - a. Pick the best node on OPEN II.
 - b. Generate its successors III.
 - c. For each successor do:
 - i. If it has not been generated before (new node), evaluate it; add it to OPEN and record its parent
 - ii. Otherwise (old node); change the parent if this new path is better than previous one.

Best-First Properties

- **Not complete**, may follow infinite path if heuristic rates each state on such a path as the best option. Most reasonable heuristics will not cause this problem however.
- Worst case **time complexity** is still $O(bm)$ where m is the maximum depth, b is the branching factor
- Since must maintain a queue of all unexpanded states, **space complexity** is also $O(bm)$.

However, a good heuristic will avoid this worst-case behavior for most problems.

Example: Romania with step costs in [km]

Imagine our agent in the city of Arad, Romania, toward the end of a touring holiday. The agent has a ticket to fly out of Bucharest the following day. The ticket is nonrefundable, the agent's visa is about to expire, and after tomorrow, there are no seats available for six weeks. Now the agent's performance measure contains many other factors besides the cost of the ticket and the Undesirability of being arrested and deported. For example, it wants to improve its suntan, improve its Romanian and take in the sights, and so on.

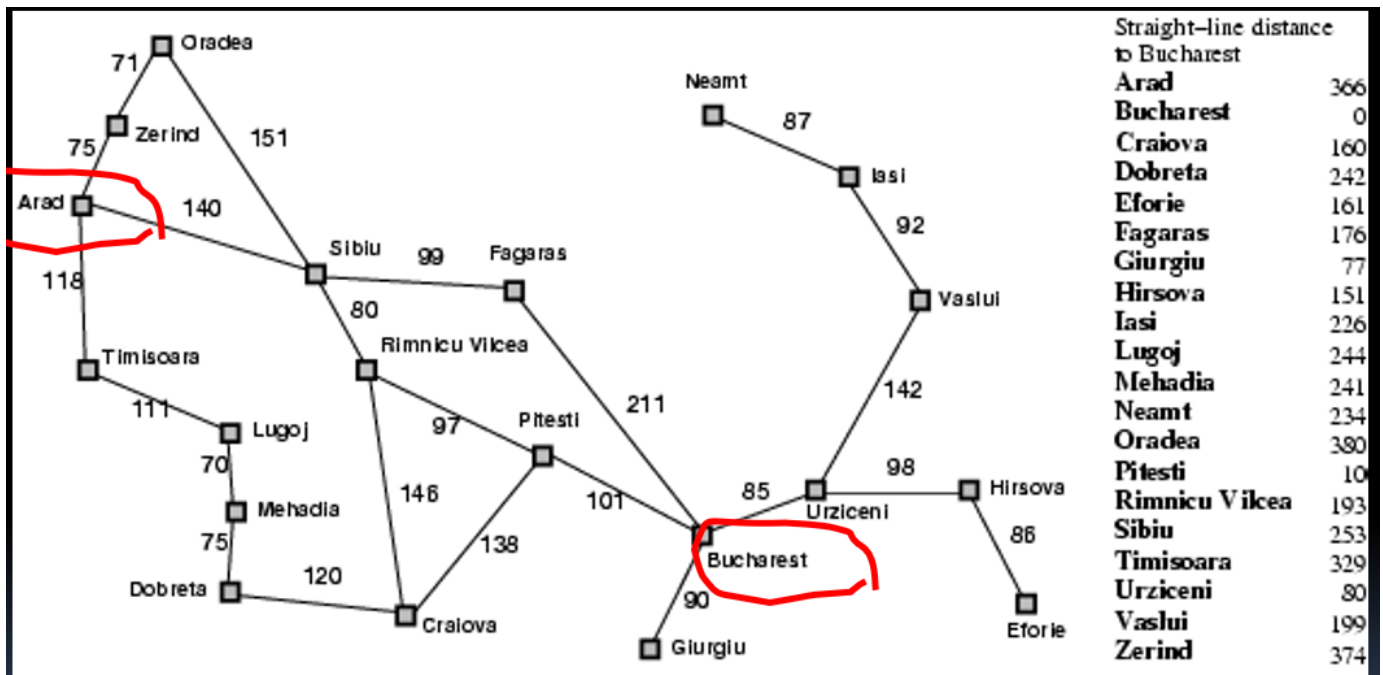
All these factors might suggest any of a vast array of possible actions. Given the seriousness of the situation, however, it should adopt the goal of driving to Bucharest. Actions that result in a failure to reach Bucharest on time can be rejected without further consideration. Goals such as this help organize behavior by limiting the objectives that the agent is trying to achieve.

Goal formulation, based on the current situation, is the first step in problem solving. As well as formulating a goal, the agent may wish to decide on some other factors that affect the desirability of different ways of achieving the goal.

Our agent has now adopted the goal of driving to Bucharest, and is considering which town to drive to from Arad. There are three roads out of Arad, one toward Sibiu, one to Timisoara, and one to Zerind.

None of these achieves the goal, so unless the agent is very familiar with the geography of Romania, it will not know which road to follow.² In other words, the agent will not know which of its possible actions is best, because it does not know enough about the state that results from taking each action. If the agent has no

additional knowledge, then it is stuck. The best it can do is choose one of the actions at random.



Disadvantage: Does not find shortest path to goal (through all Rimnicu) since it is only focused on the cost remaining rather than the total cost.

Greedy algorithm Concept

As an example, the greedy algorithm determines the minimum number of US coins to give while making change (e.g. 1.75 US dollar). These are the steps; a human would take; to emulate a greedy algorithm. The coin of the highest value less than the remaining change owed, is the local optimum [7 quarters equivalent to 1.75 change owed, is the local optimum [7 quarters equivalent to 1.75 US dollars rather than 175 cent], 7 pieces compared to 175 pieces.

Note that in general the change-making problem requires dynamic programming to find an optimal solution.

4.7.2 Greedy best-first search

The Evaluation function $f(n) = h(n)$ (heuristic) = estimate of cost from n to goal.

e.g., $h_{SLD}(n)$ = Straight-line Distance in km from n to Bucharest

Greedy best-first search expands the node that appears to be closest to goal.

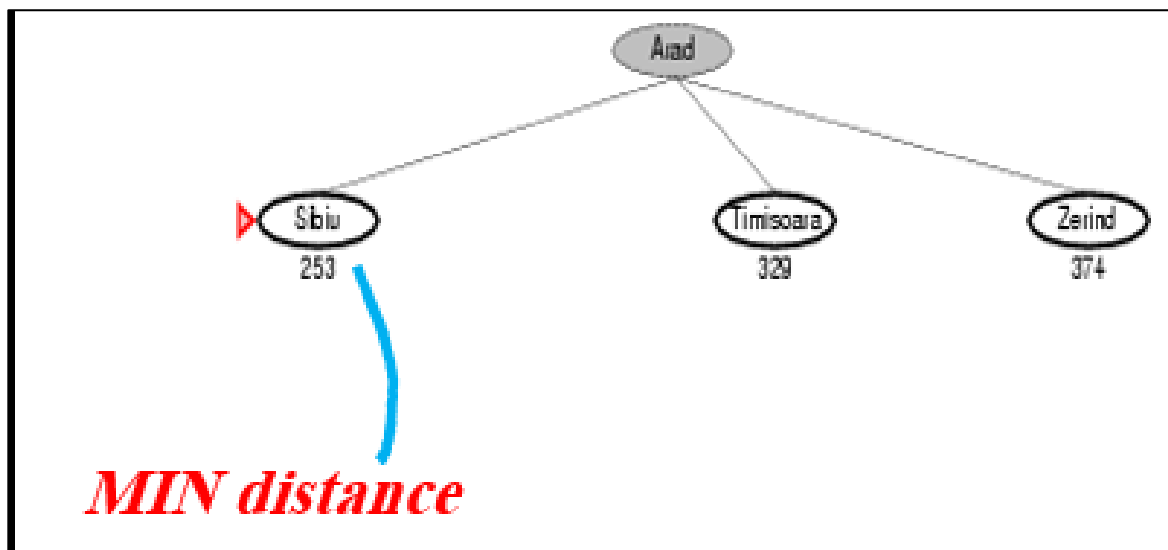
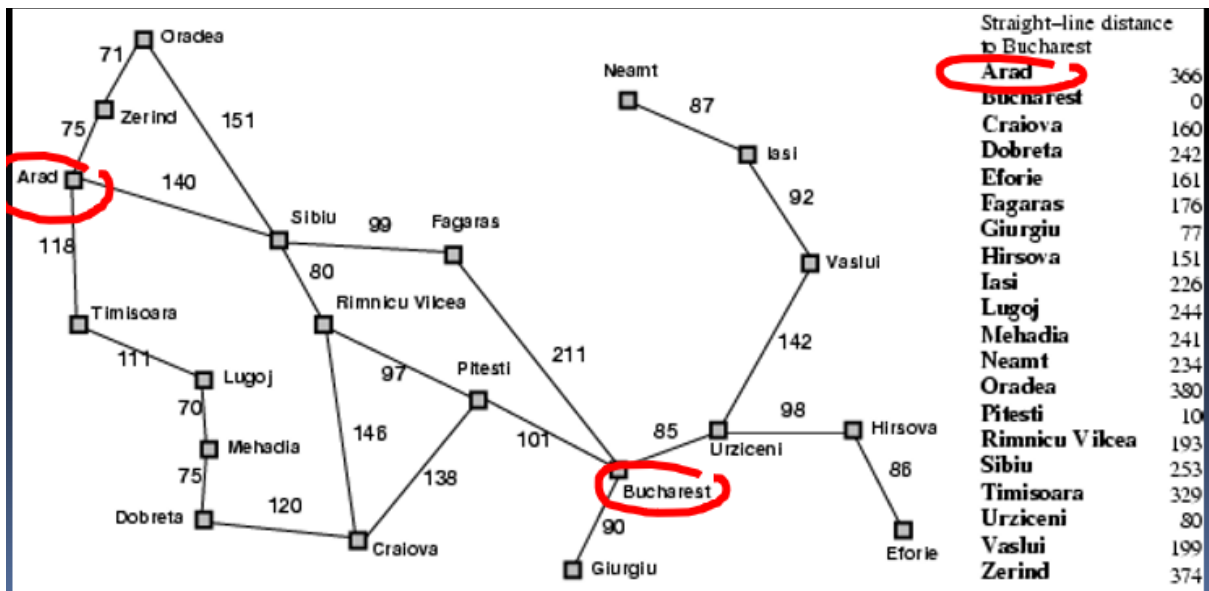
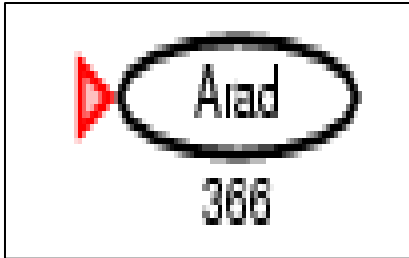
Pseudo-code for greedy best-first search (GBFS)

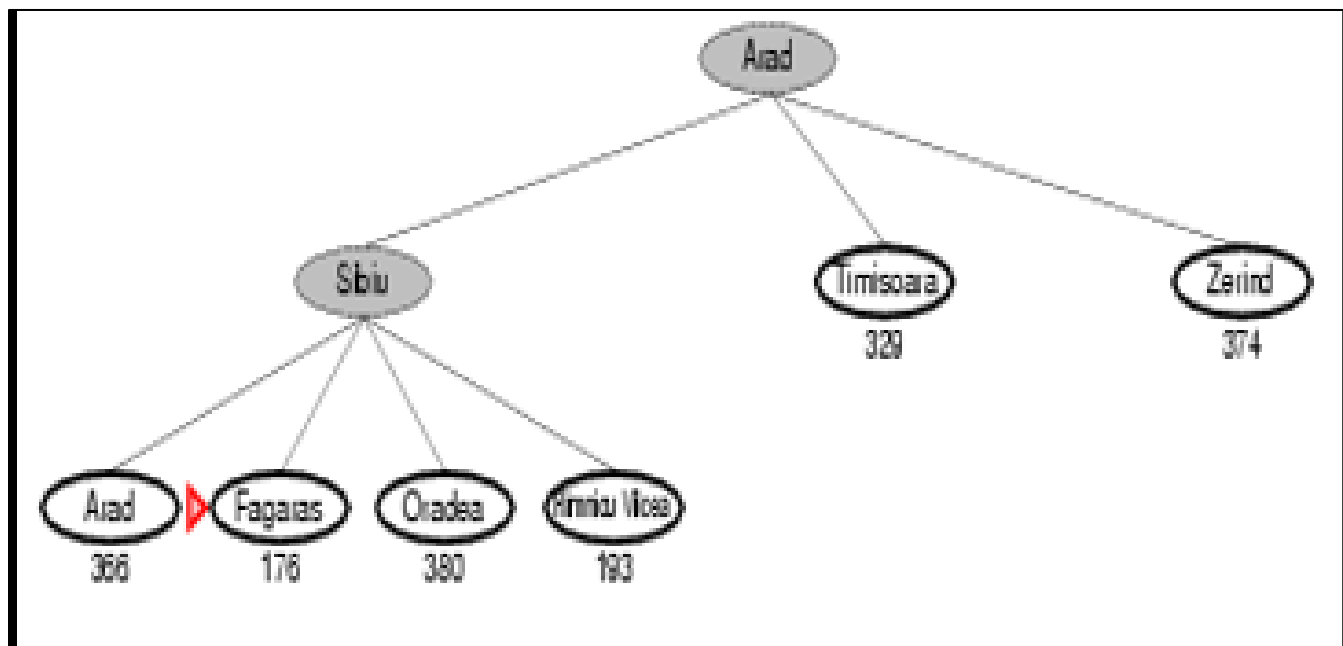
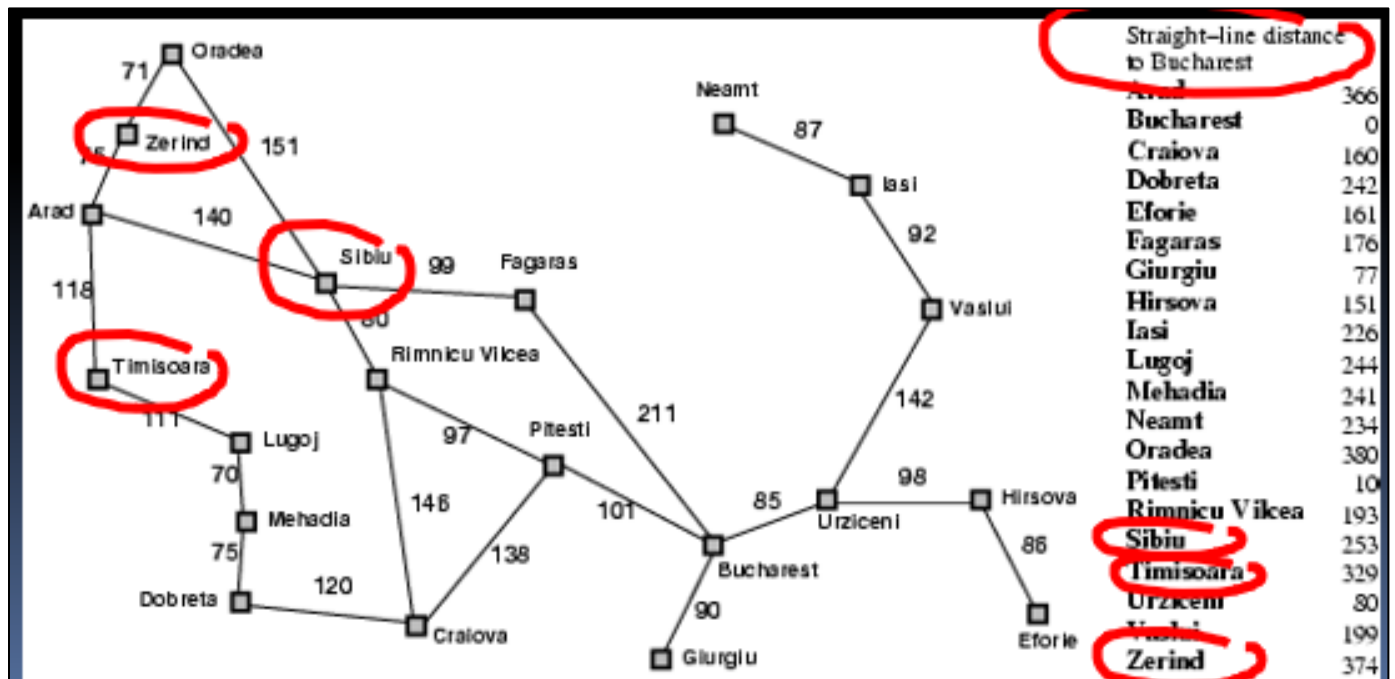
```

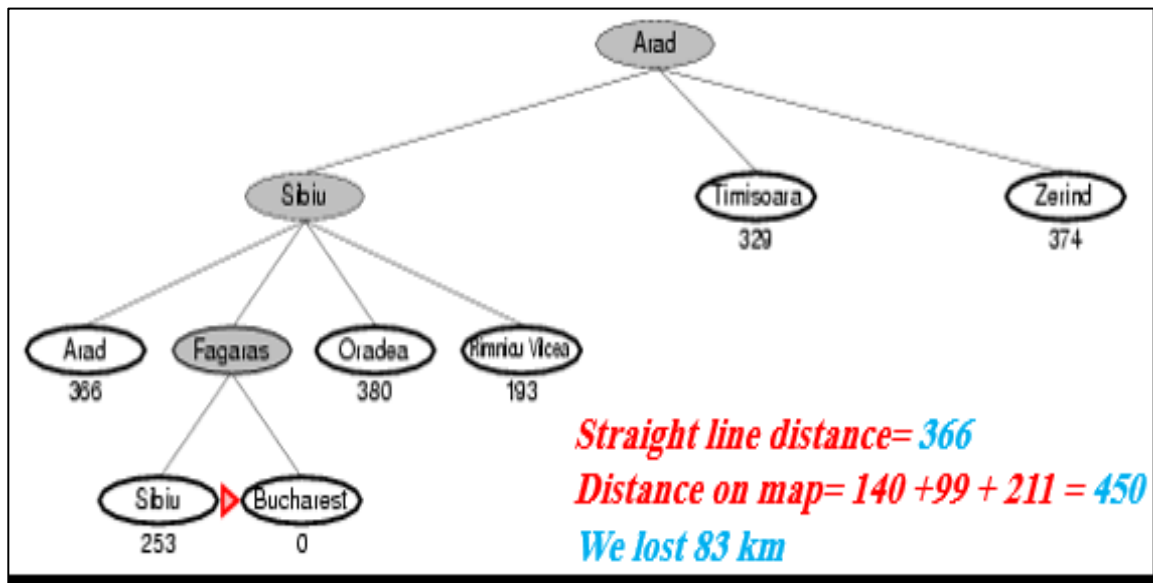
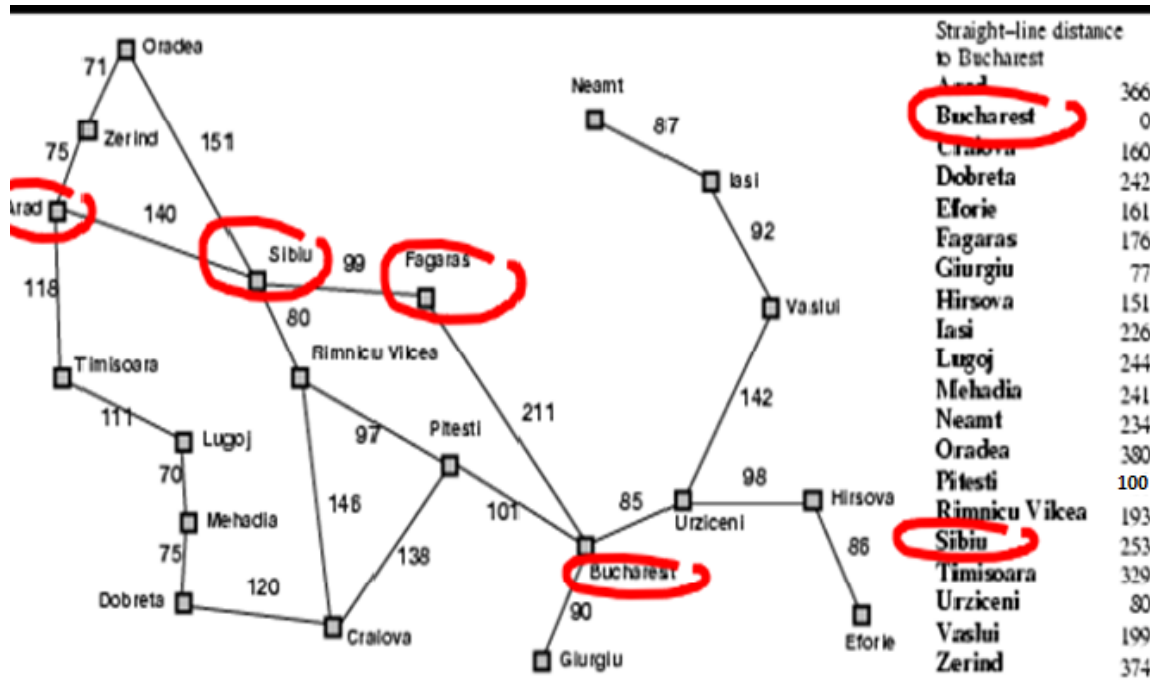
1: Procedure: GreedyBFS
2: insert (state=initial_state, h=initial_heuristic, counter=0) into search_queue;
3:
4: while search_queue not empty do
5:   current_queue_entry = pop item from front of search_queue;
6:   current_state = state from current_queue_entry;
7:   current_heuristic = heuristic from current_queue_entry;
8:   starting_counter = counter from current_queue_entry;
9:   applicable_actions = array of actions applicable in current_state;
10:
11:   for all index  $?i$  in applicable_actions  $\geq$  starting_counter do
12:     current_action = applicable_actions[ $?i$ ];
13:     successor_state = current_state.apply(current_action);
14:
15:     if successor_state is goal then
16:       return plan and exit;
17:     end if
18:     successor_heuristic = heuristic value of successor_state;
19:
20:     if successor_heuristic < current_heuristic then
21:       insert (current_state, current_heuristic,  $?i + 1$ ) at front of search_queue;
22:       insert (successor_state, successor_heuristic, 0) at front of search_queue;
23:       break for;
24:
25:     else
26:       insert (successor_state, successor_heuristic, 0) into search_queue;
27:     end if
28:   end for
29: end while
30: exit - no plan found;

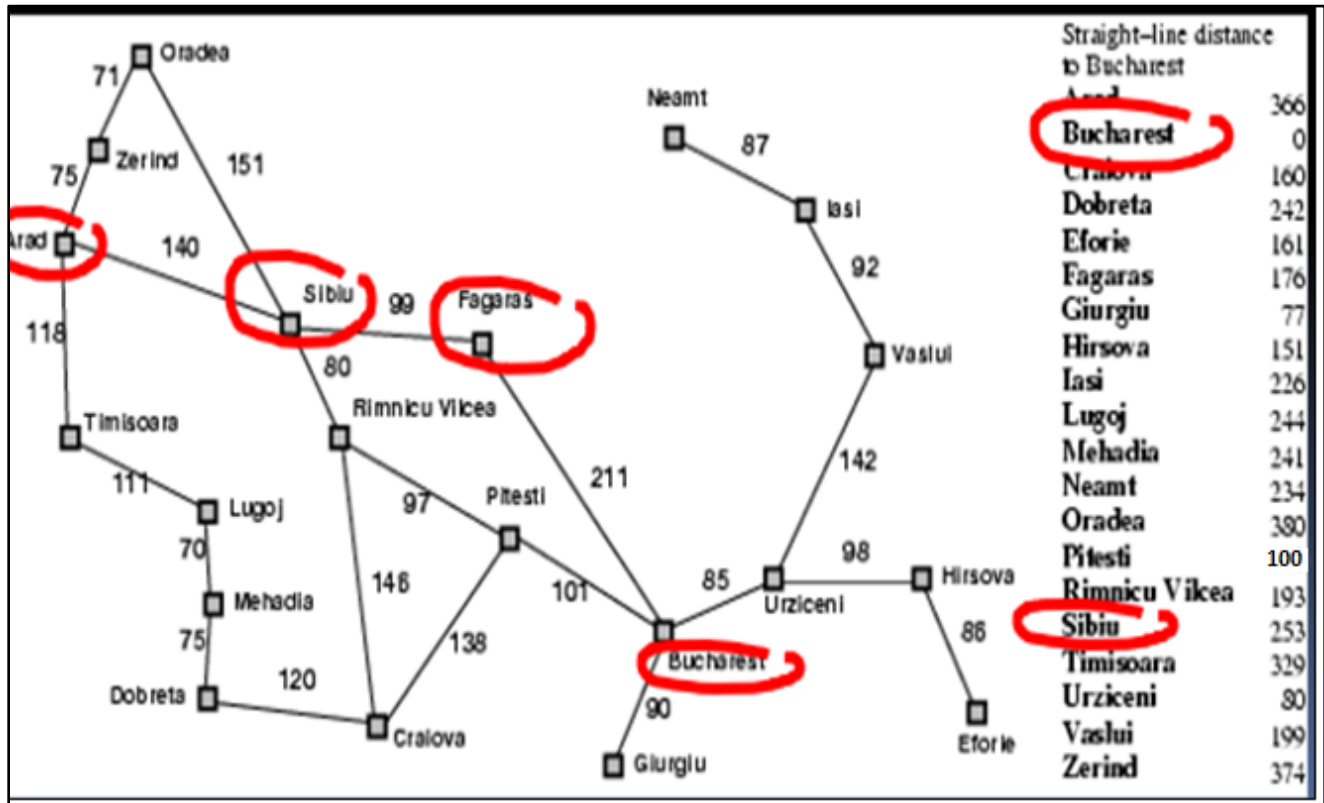
```

Greedy best-first search example









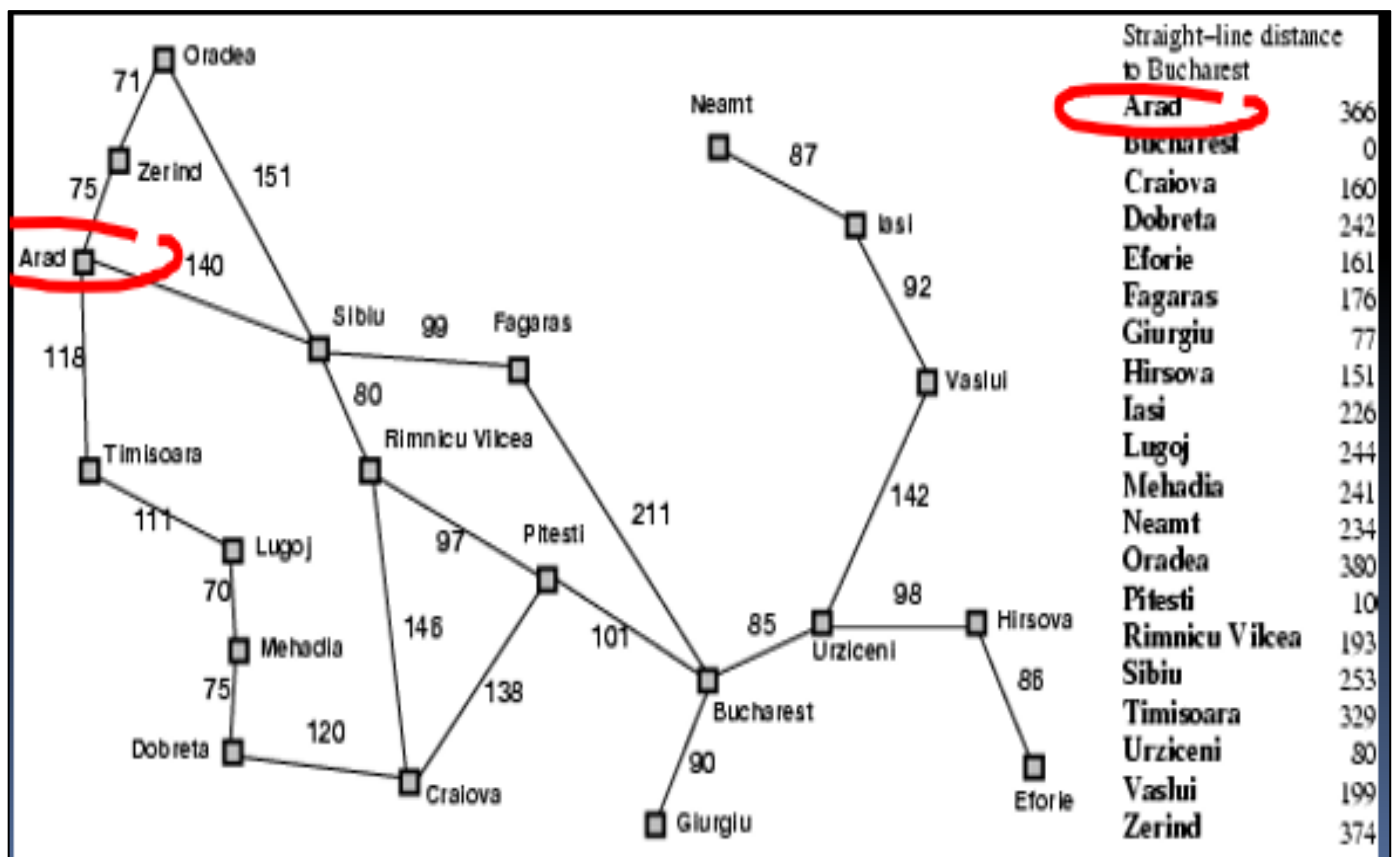
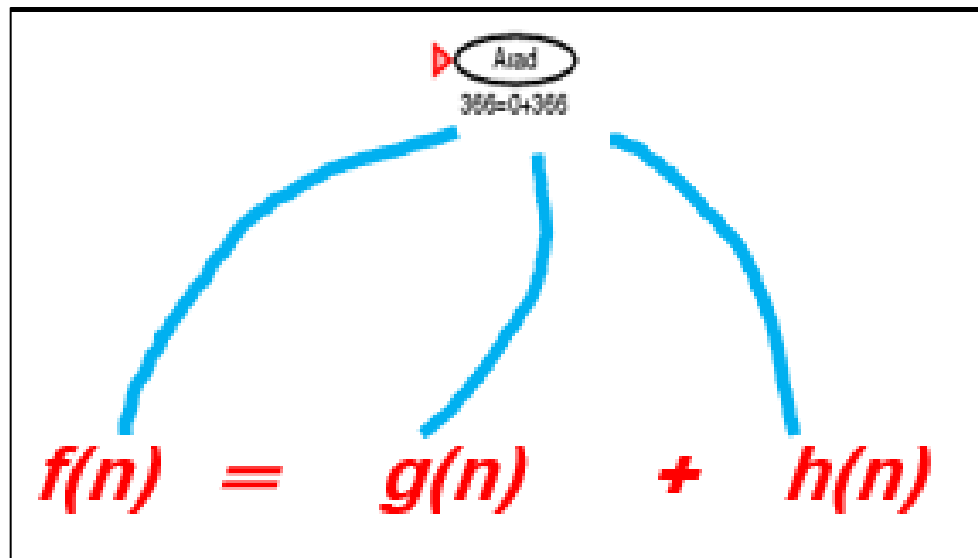
Properties of Greedy best-first search

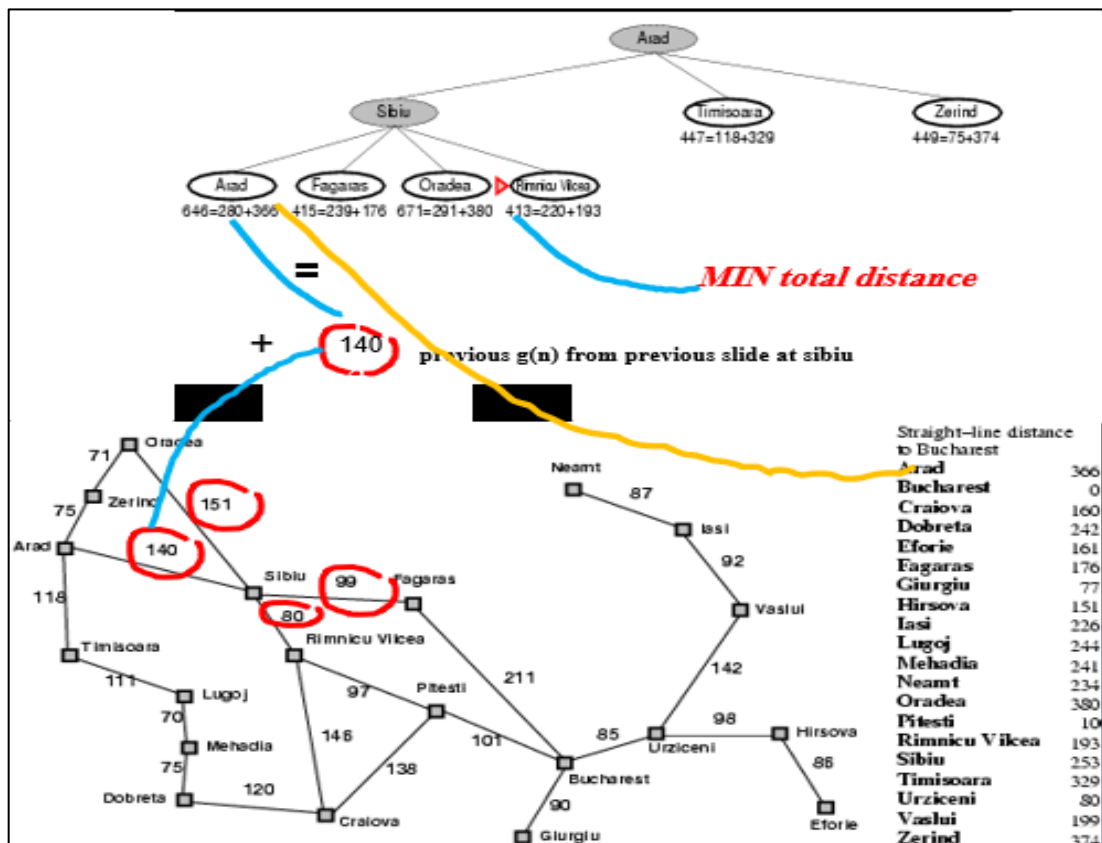
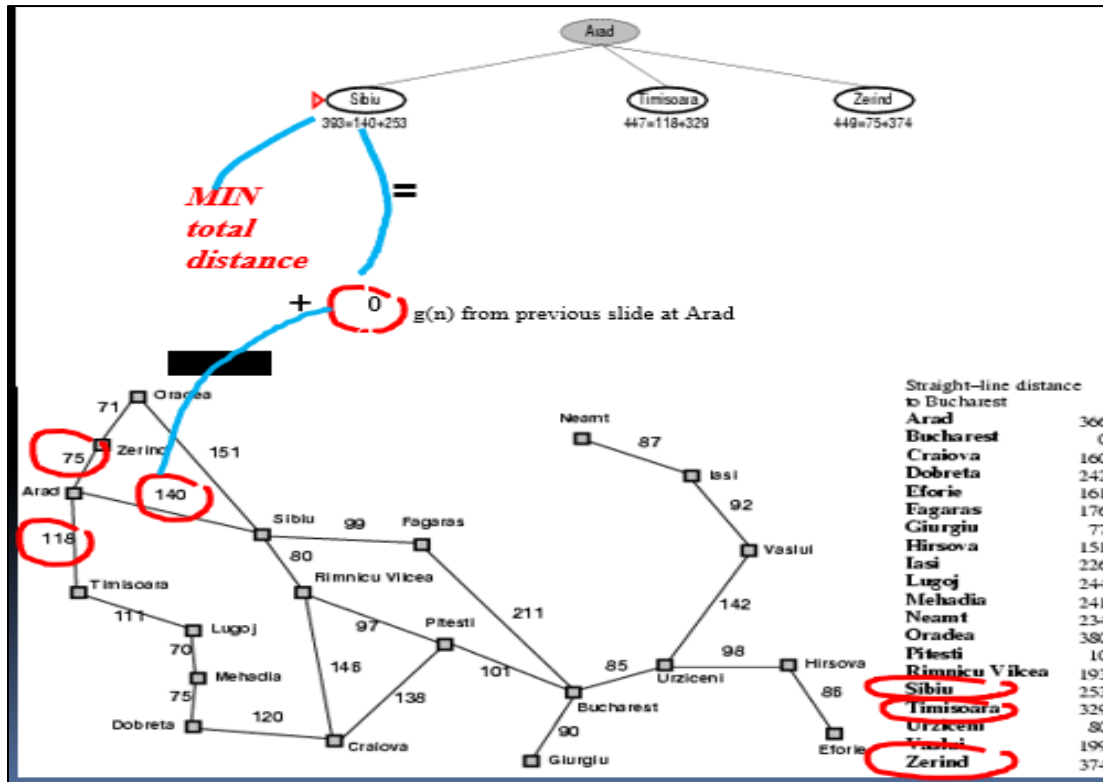
- Complete? No – can get stuck in loops, e.g., Iasi → Neamt → Iasi → Neamt →
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? No

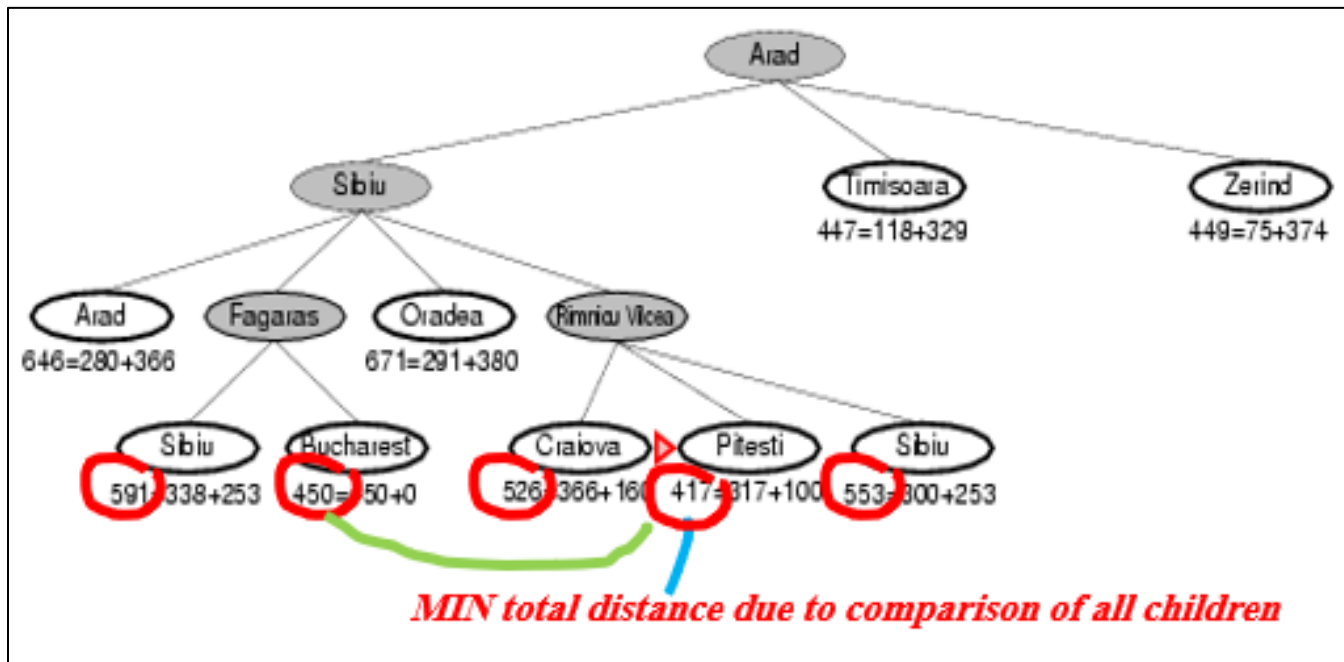
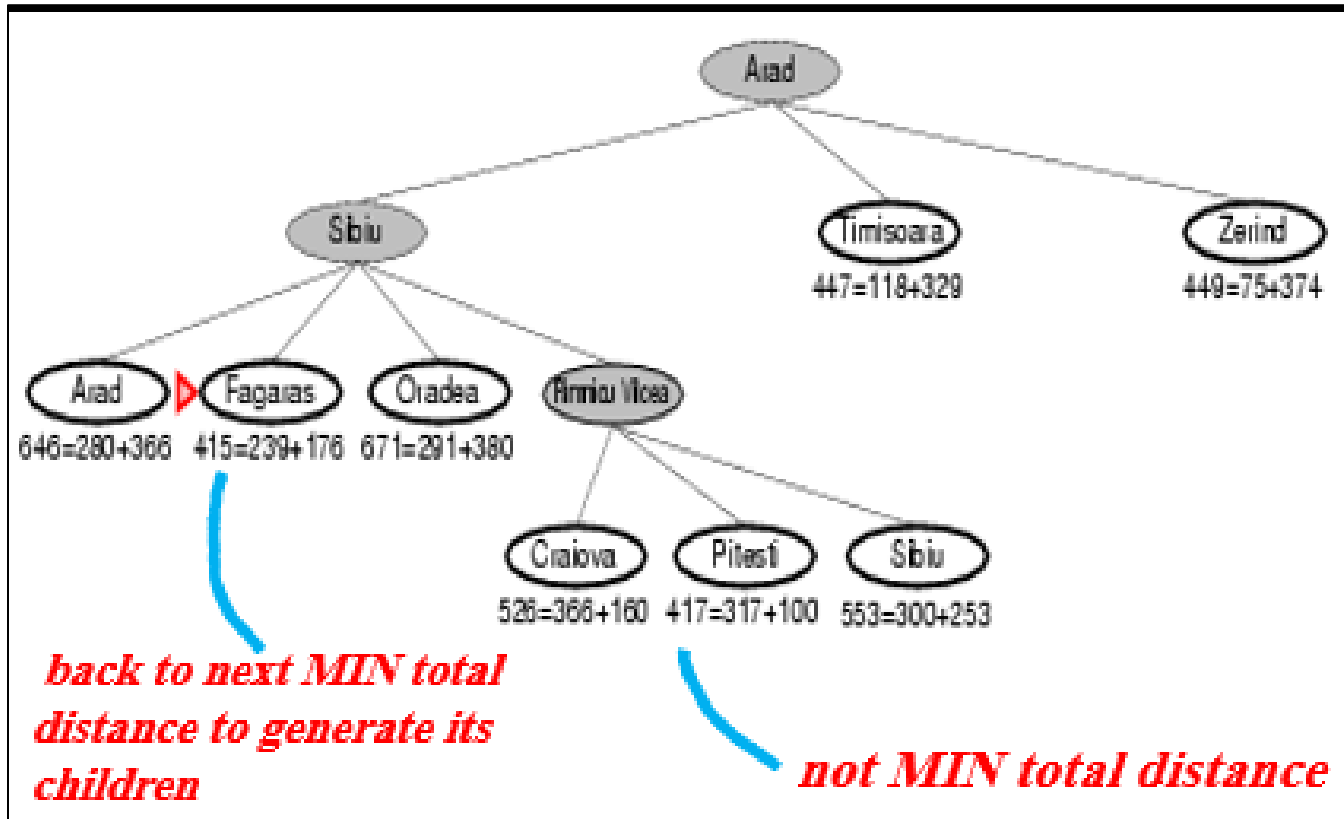
4.7.3 A* search (Enhanced Greedy Best First Search)

- Idea: avoid expanding paths that are already expensive
- Evaluation function $f(n) = g(n) + h(n)$ where:
 - $g(n)$ = cost so far to reach n (accumulative)
 - $h(n)$ = estimated cost from n to goal
 - $f(n)$ = estimated total cost of path through n to goal

A* search example





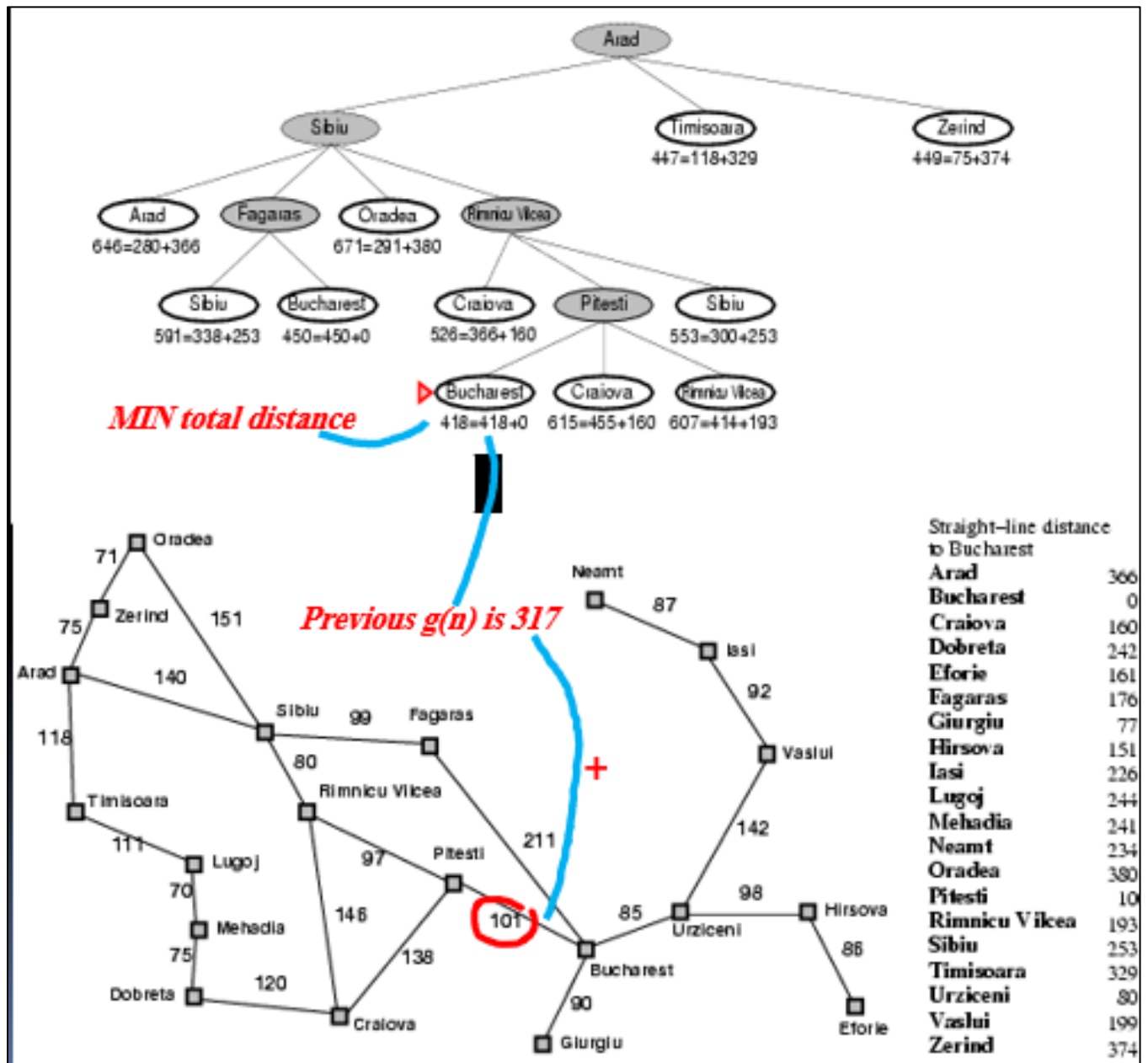


Straight line distance = 366

Distance on map= 140 +80 + 97 + 101 = 418 < 450

We lost 51 km

We saved 32 km



Properties of A*

- Complete? Yes
- Time? : Exponential
- Space? : Keeps all nodes in memory
- Optimal? : Yes

4.8 Constraint Satisfaction Problems (CSP)

In artificial intelligence and operations research, constraint satisfaction is the process of finding a solution to a set of constraints that impose conditions that the variables must satisfy. A solution is therefore, a vector of variables that satisfies all constraints. The techniques used in constraint satisfaction depend on the kind of constraints being considered. Such problems are usually solved via:

- search, in particular a form of backtracking
- local search
- Constraint propagation
- Constraint propagation methods are also used in conjunction with search to make a given problem simpler to solve.

Constraint satisfaction originated in the field of artificial intelligence in the 1970s. During the 1980s and 1990s, insert of constraints into a programming language were developed. Languages often used for constraint programming are Prolog and C++.

4.8.1 Constraint Satisfaction Problems Basic Concepts (CSP)

Mathematical problems where one must find states or objects that satisfy a number of constraints or criteria. CSPs are the subject of intense research in both artificial intelligence and operations research. Many CSPs require a combination of heuristics and combinatorial search methods to be solved in a reasonable time. Examples of problems that can be modeled as a CSP:

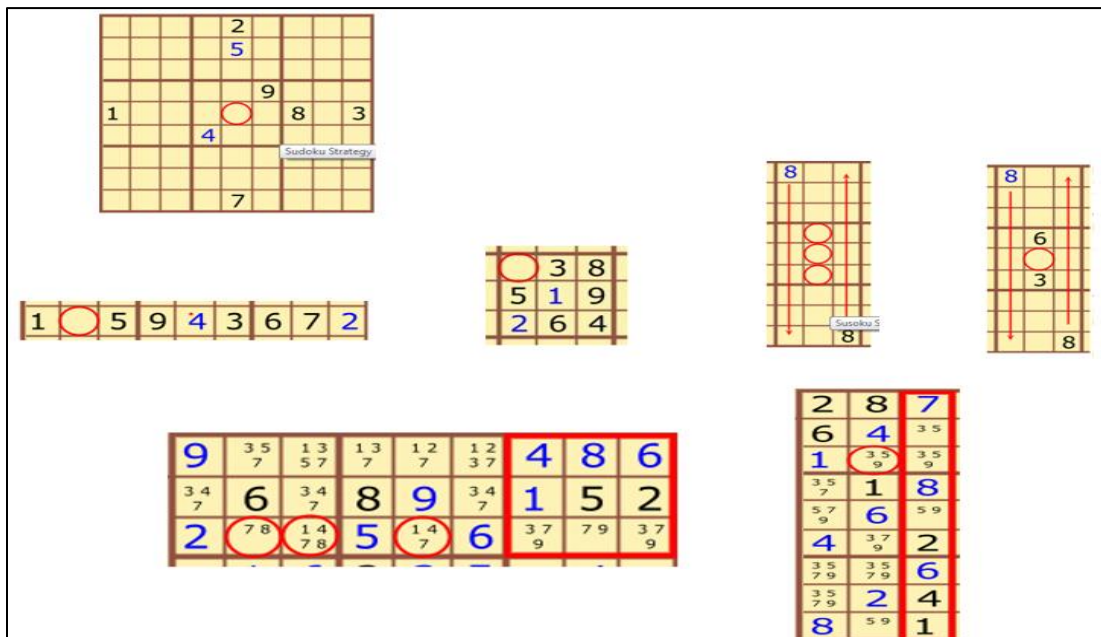
- Eight queens puzzle

- Map coloring problem
- Sudoku
- Boolean satisfiability

Algorithms used for solving CSP include:

- AC-3 algorithm
- Backtracking
- min-conflicts algorithm

CSP Example 1 (Sudoku Strategy)



CSP formal math definition

Formally, a constraint satisfaction problem is defined as a triple (X, D, C) , where:

X is a set of variables

D is a domain of values

C is a set of constraints

Every constraint is in turn, is a pair (t, R) ; where:

t is a tuple of variables

R is a set of tuples of values; all these tuples having the same number of elements; as a result, R is a relation.

An evaluation of the variables is a function from variables X to domains D . A solution is an evaluation that satisfies all constraints.

CSP as a search problem

Standard search problem:

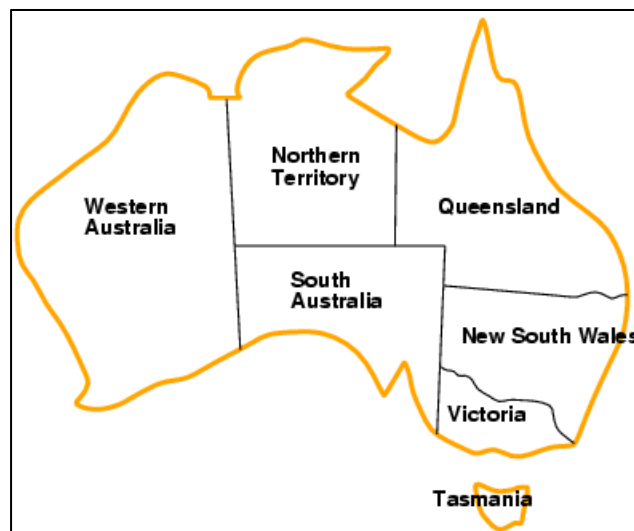
- state is a “black box” – any data structure that supports successor function, heuristic function, and goal test

CSP:

- State is defined by variables X_i with values from domain D_i
- Goal test is a set of constraints specifying allowable combinations of values for subsets of variables.

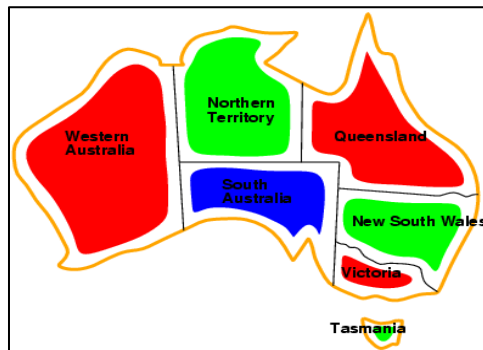
Simple example of a formal representation language, allows useful general-purpose algorithms with more power than standard search algorithms.

CSP Example 2 (Map-Coloring)



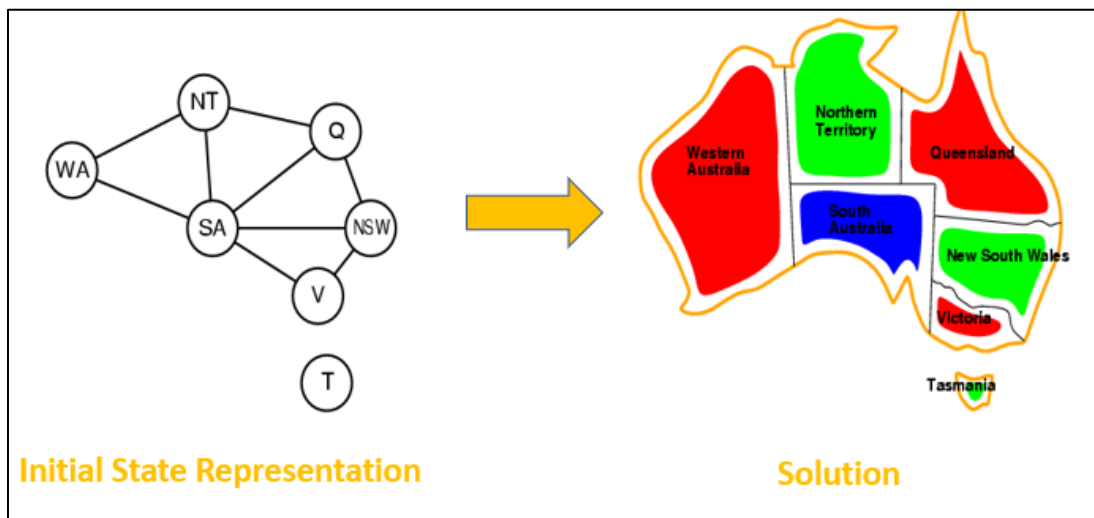
- Variables WA, NT, Q, NSW, V, SA, T

- Domains $D_i = \{\text{red, green, blue}\}$
- Constraints: adjacent regions must have different colors
e.g., $WA \neq NT$, or (WA, NT) in:
 $\{(\text{red, green}), (\text{red, blue}), (\text{green, red}), (\text{green, blue}), (\text{blue, red}), (\text{blue, green})\}$
- Solutions are:
 - complete
 - consistent assignments,
 e.g., $WA = \text{red}, NT = \text{green}, Q = \text{red}, NSW = \text{green}, V = \text{red}, SA = \text{blue}, T = \text{green}$



CSP Example 2 -Constraint graph

- Binary CSP: each constraint relates two variables(nodes)
- Constraint graph: nodes are variables, arcs are constraints



Varieties of CSP's variables

- Discrete variables

- finite domains:
 - n variables, domain size $d \rightarrow O(d^n)$ complete assignments
 - e.g., Boolean CSPs
- infinite domains:
 - integers, strings
 - e.g., job scheduling, variables are start/end days for each job
 - need a constraint language, e.g., $StartJob_1 + 5 \leq StartJob_3$

➤ **Continuous variables**

- e.g., start/end times for Hubble Space Telescope observations
- linear constraints solvable in polynomial time by linear programming

Varieties of CSP's constraints

- **Unary** constraints involve a single variable, (e.g., $SA \neq \text{green}$)
- **Binary** constraints involve pairs of variables, (e.g., $SA \neq WA$)
- **Higher-order** constraints involve 3 or more variables, (e.g., crypt arithmetic column constraints (*discussed next*))

CSP Example 3: Crypt arithmetic

It is a type of mathematical game consisting of a mathematical equation among unknown numbers, whose digits are equation among unknown numbers, whose digits are represented by letters. The goal is to identify the value of each letter.

The name can be extended to puzzles that use non-alphabetic symbols instead of letters. The equation is typically a basic operation of arithmetic, such as addition, multiplication, or division.

The classic example, published in the July 1924 issue of Strand Magazine by Henry Dudeney is:

The diagram illustrates the SEND + MORE = MONEY cryptarithm puzzle. On the left, the puzzle is written in red capital letters: SEND + MORE = MONEY. A blue oval highlights the word 'MORE' and a blue arrow points to the right. On the right, the numerical solution is shown on a yellow background: 9567 + 1085 = 10652. The numbers are aligned by their digits: 9567, 1085, and 10652.

The final solution to this puzzle is:

M = 1, O = 0, Y = 2, E = 5, N = 6, D = 7, R = 8, and S = 9.

Traditionally, each letter should represent a different digit, and the leading digit (D, E, Y) of a multi-digit number must not be zero (Constraint 1). A good puzzle should have a unique solution and the letters should make up a cute phrase (Constraint 2). Crypt arithmetic can be useful as a motivation in the teaching of algebra and basics of cryptic.

Solving crypt arithmetic example

Solving a cryptarithm by hand usually involves a mix of deductions and exhaustive tests of possibilities. For instance, the following sequence of deductions solves SEND + MORE = MONEY puzzle above (columns are numbered from right to left):

1. From column 5, M = 1 since it is the only carry-over possible from the sum of two single digit numbers in column 4.
2. To produce a carry from column 4 to column 5, S + M is at least 9 (in case of carry from previous addition) or more than 9 (in case of no carry from previous addition), so S is 8 (in

case of carry from previous addition) or 9 (in case of no carry from previous addition), so $S + M$ is 9 or 10, and so O is 0 or 1. But $M = 1$, so $O = 0$.

3. If there were a carry from column 3 to column 4 then $E = 9$ and so $N = 0$. But $O = 0$, so there is no carry from column 3 to column 4, and $S = 9$.

4. If there were no carry, from column 2 to column 3; then $E = N$, which is impossible. Therefore there is a carry from column 2 to column 3 and $N = E + 1$.

5. If there were no carry from column 1 to column 2, then $N + R = E \bmod 10$, and $N = E + 1$, so $E + 1 + R = E \bmod 10$, so $R = 9$. But $S = 9$ (rejected hypothesis), so there must be a carry from column 1 to column 2 and $R = 8$.

6. To produce a carry from column 1 to column 2, we must have $D + E = 10 + Y$. As Y cannot be 0 or 1, $D + E$ is at least 12. As D is at most 7, then E is at least 5. Also, N is at most 7, and $N = E + 1$. So E is 5 or 6.

7. If E were 6 then to make $D + E$ at least 12, D would have to be 7. But $N = E + 1$, so N would also be 7, which is impossible. Therefore $E = 5$ and $N = 6$.

8. To make $D + E$ at least 12 we must have $D = 7$, so $Y = 2$.

Algorithm Constraint Satisfaction

Propagate available constraints. To do this first set OPEN to set of all objects that must have values assigned to them in a complete solution. Then do until an inconsistency is detected or until OPEN is empty:

- a. Select an object OB from OPEN. Strengthen as much as possible the set of constraints that apply to OB.
- b. If this set is different from the set that was assigned the last time OB was examined or if this is the first time OB has been examined, then add to OPEN all objects that share any constraints with OB.
- c. Remove OB from OPEN.

If the union of the constraints discovered above defines a solution, then quit and report the solution.

If the union of the constraints discovered above defines a contradiction, then return the failure.

If neither of the above occurs, then it is necessary to make a guess at something in order to proceed. To do this loop until a solution is found or all possible solutions have been eliminated:

- Select an object whose value is not yet determined and select a way of strengthening the constraints on that object
- Recursively invoke constraint satisfaction with the current set of constraints augmented by the strengthening constraint just selected.

Exercise 1

$$\begin{array}{r}
 \text{E A T} \\
 + \quad \text{T H A T} \\
 \hline
 \text{A P P L E}
 \end{array}$$

- Since every four-digit number is less than 10,000 and every three-digit number is less than 1,000, the sum of two such numbers is necessarily less than 11,000.
- This sum, though, is a five-digit number, hence is greater than 10,000. Consequently, A must be 1 and P must be 0.
- Further, we can conclude that $T = 9$. Otherwise, we would be adding a number less than 1,000 to one less than 9,000, leaving us short of the requisite total.
- The units column then produces $E = 8$ while generating a carryover of 1 into the tens column. Together with the previously found value of A, we learn from the tens column that $L = 3$.
- Finally, the hundreds column yields the equation $E + H = P + 10$, where the "10" is required to accommodate the needed carryover into the thousands column.
- When the values of E and P are substituted into this relationship, we get $8 + H = 10$, from which it follows that $H = 2$.

Therefore, the unique solution of the puzzle turns out to be $819 + 9219 = 10038$.

Exercise 2

$$\begin{array}{r}
 \text{N O} \\
 \text{G U N} \\
 + \quad \text{N O} \\
 \hline
 \text{H U N T}
 \end{array}$$

- Obviously $H = 1$.
- From the NUNN column we must have "carry 1," so $G = 9$, $U = \text{zero}$.

- Since we have "carry" zero or 1 or 2 from the ONOT column, correspondingly we have $N + U = 10$ or 9 or 8.
- But duplication is not allowed, so $N = 8$ with "carry 2" from ONOT.
- Hence, $O + O = T + 20 - 8 = T + 12$.
- Testing for $T = 2, 4$ or 6, we find only $T = 2$ acceptable, $O = 7$.
- So we have $87 + 908 + 87 = 1082$

CSP Real-world Applications

- Assignment problems (e.g., who teaches what class)
- Timetabling problems (e.g., which class is offered when and where?)
- Transportation scheduling
- Factory scheduling

4.8.2 Solving of CSPs

Constraint satisfaction problems (on finite domains) are typically solved using a form of search. The most used techniques are variants of:

- backtracking (without and with heuristics)
- constraint propagation
- local search

4.8.2.1 Backtracking Search (without heuristics)

Variable assignments are commutative, i.e,

[WA = red then NT = green] same as [NT = green then WA = red]

- Only need to consider assignments to a single variable at each node $\rightarrow b = d$
- Depth-first search for CSPs with single-variable assignments is called backtracking search. i.e. Backtracking = DFS
- Backtracking search is the basic uninformed algorithm for CSPs
- Can solve n-queens for $n \approx 25$

It is a recursive algorithm. It maintains a partial assignment of the variables. Initially, all variables are unassigned. At each step, a variable is chosen, and all possible values are assigned to it in turn. For each value,

The consistency of the partial assignment with the constraints is checked; in case of consistency, a recursive call is performed. When all values have been tried, the algorithm backtracks. Consistency is defined as the satisfaction of all constraints whose variables are all assigned.

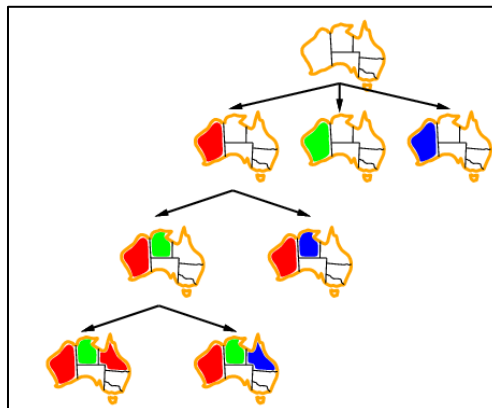
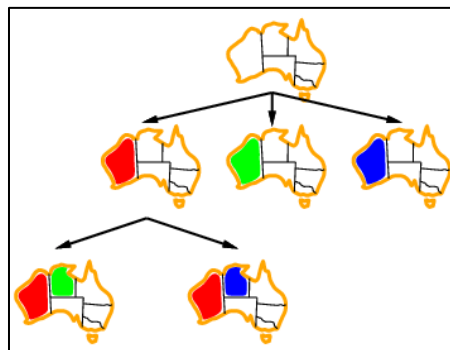
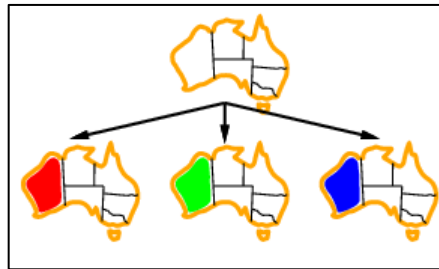
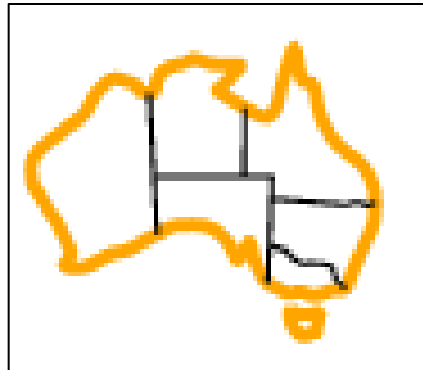
Backtracking Search Algorithm

```

function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return RECURSIVE-BACKTRACKING( $\{\}$ , csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(Variables[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment according to Constraints[csp] then
      add { var = value } to assignment
      result  $\leftarrow$  RECURSIVE-BACKTRACKING(assignment, csp)
      if result  $\neq$  failure then return result
      remove { var = value } from assignment
  return failure
  
```

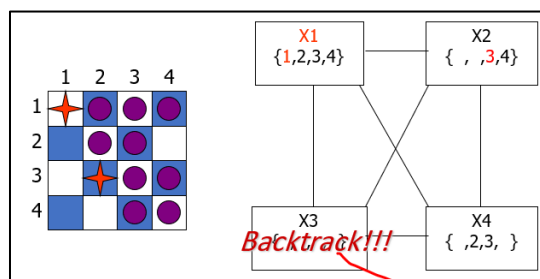
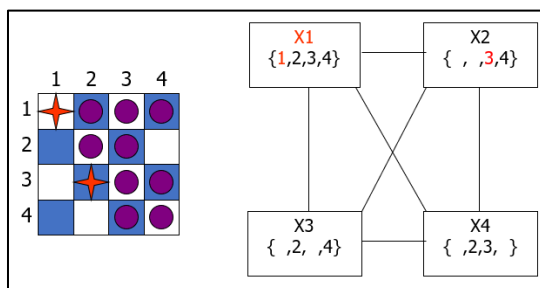
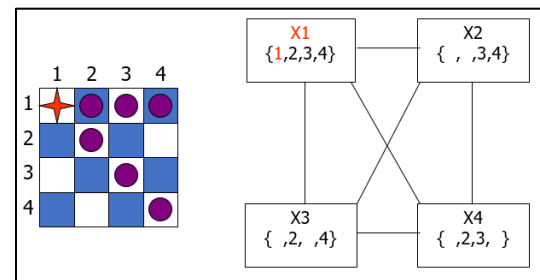
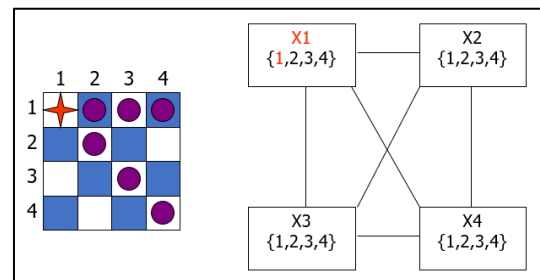
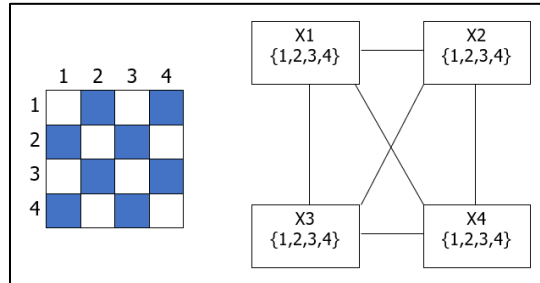
Example 3: Backtracking search



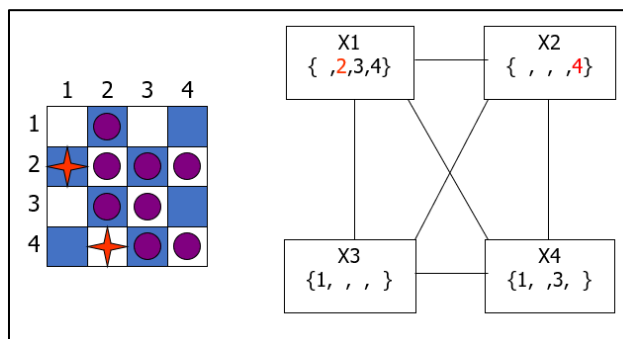
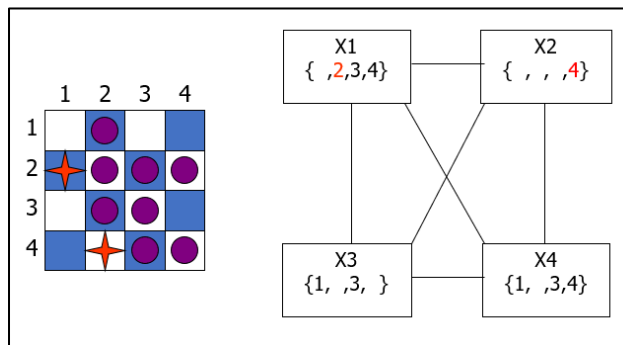
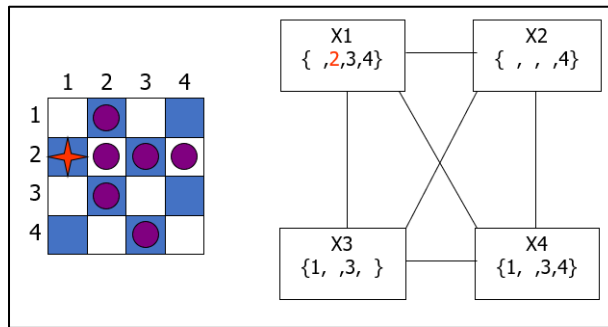
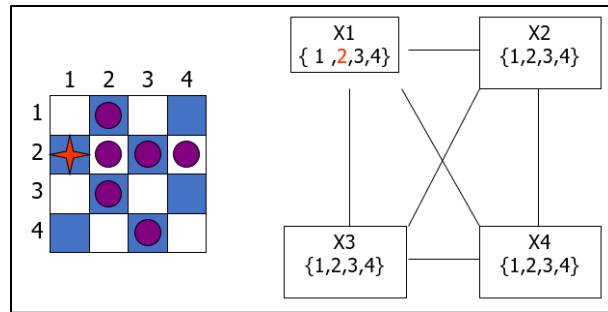
Continue, backtrack if blocked

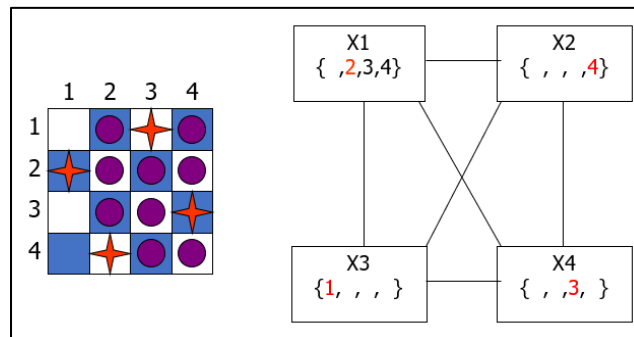
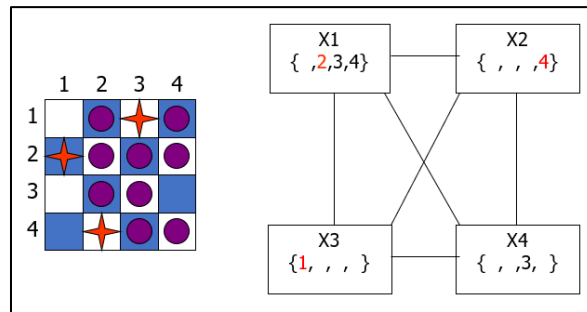
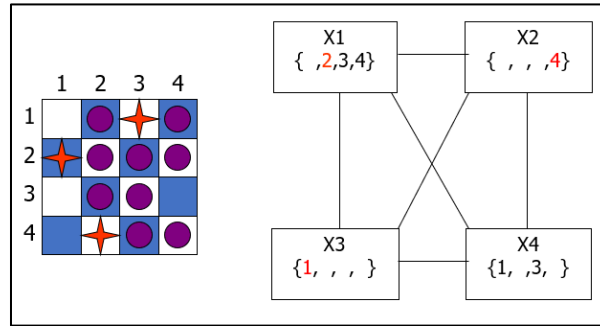
Example 4: 4-Queens Problem

(No row or diagonals, columns waits for final consistency without backtracking)



The previous steps are not logic because it blocks column 3, backtrack and take the next choice until exhausting





All assignments satisfy all constraints, no backtracking; the previous steps are logic and this is a final solution.

4.8.2.2 Backtracking Search with Heuristics

General-purpose methods can give huge gains in speed, but:

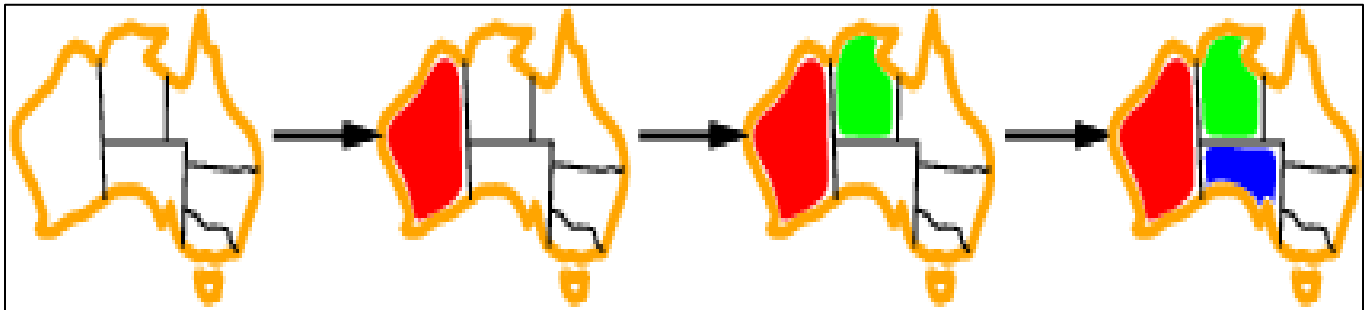
- Which variable should be assigned next?
- In what order should its values be tried?
- Can we detect predictable failure early?

Possible Heuristics techniques:

1. Most constrained variable
2. Most constraining variable
3. Least constraining value
4. Forward checking

1. Most constrained variable minimum remaining values (MRV) heuristic

Choose the variable (WA, NT, Q, NSW, V, SA, T) with the fewest legal values

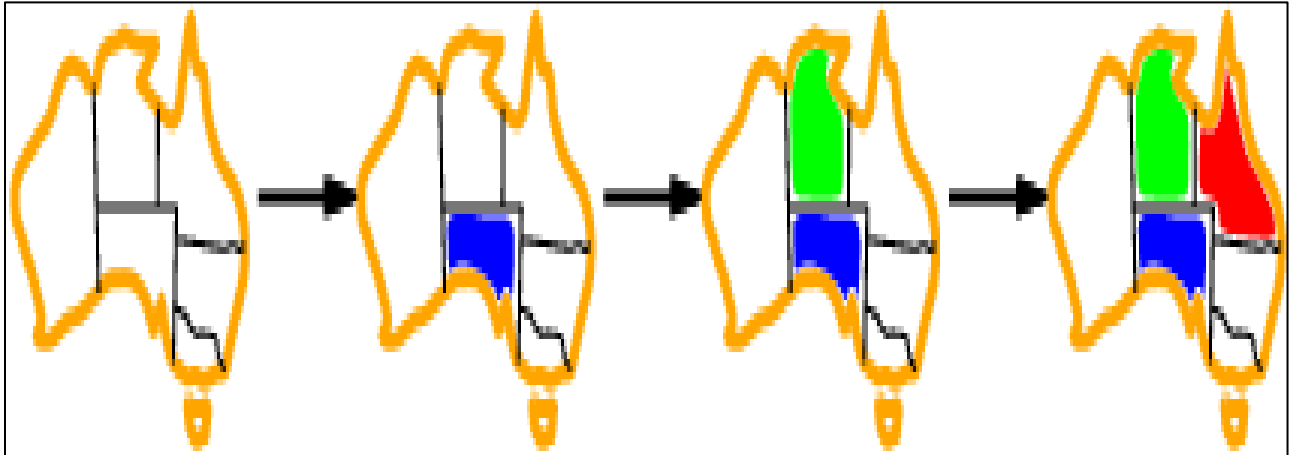


- Initially, assign to WA which have 2 borders; any legal value.
- Then NT, which have 3 borders; has 2 left legal values.
- SA, which have 4 borders; has the last legal value....etc

2. Most constraining variable

For the case of: Tie-breaker among most constrained variables. Most constraining variable:

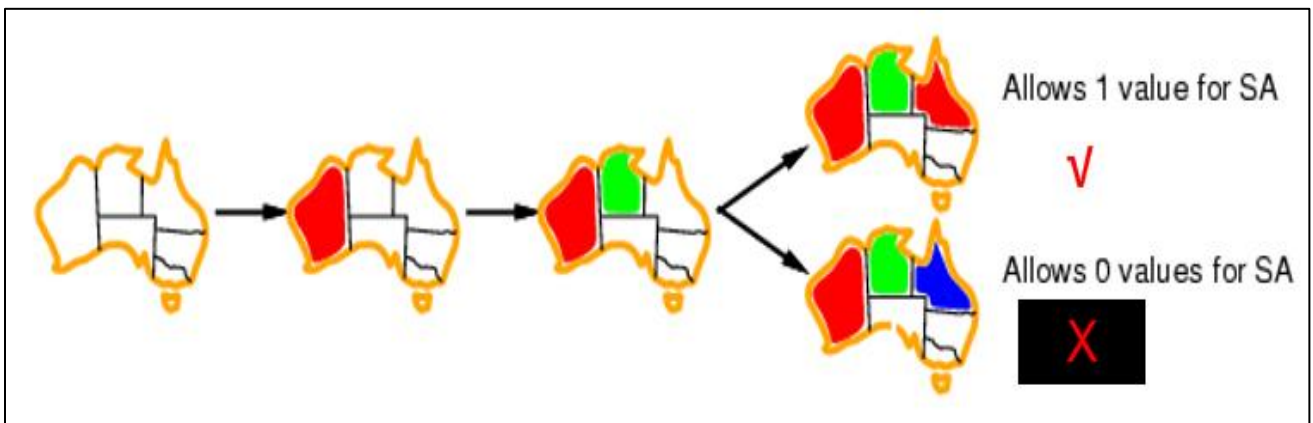
- Choose the variable with the most constraints on remaining variables
 - SA has 5 borders, assign any color to it.
 - NT, which have 3 borders; can have 2 choices
 - Q, which have 3 borders; can have only one choice left..etc



3. Least constraining value

Given a variable, choose the least constraining value:

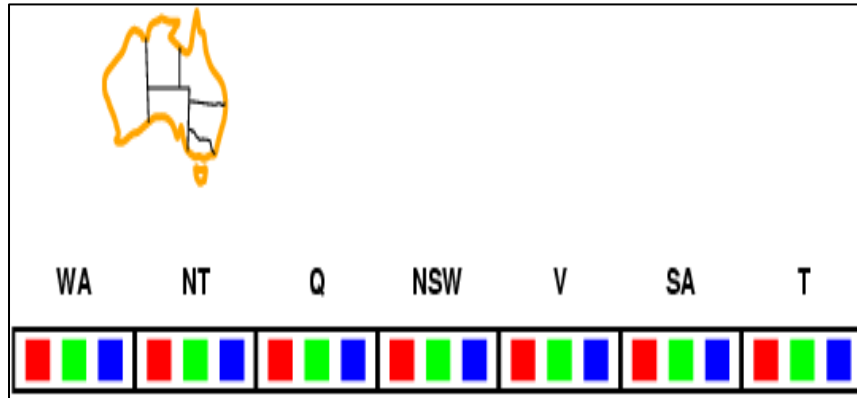
- the one that rules out the fewest values in the remaining variables



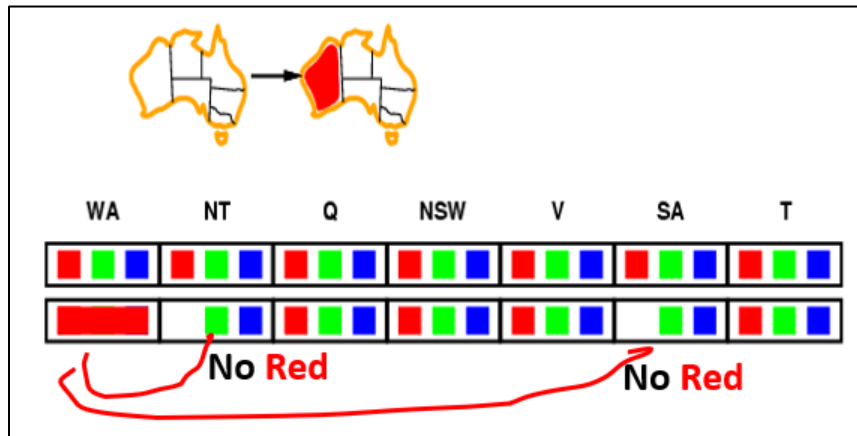
4. Forward checking

Idea:

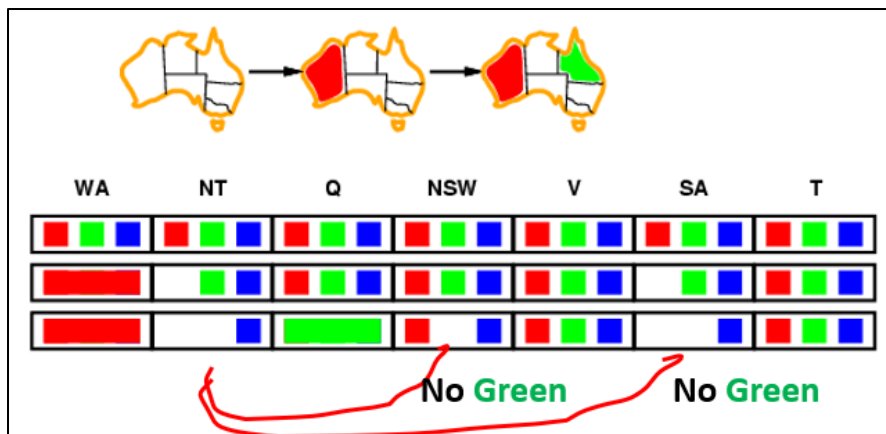
- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values
- Initially, all the 3 choices are available for all the 7 variables



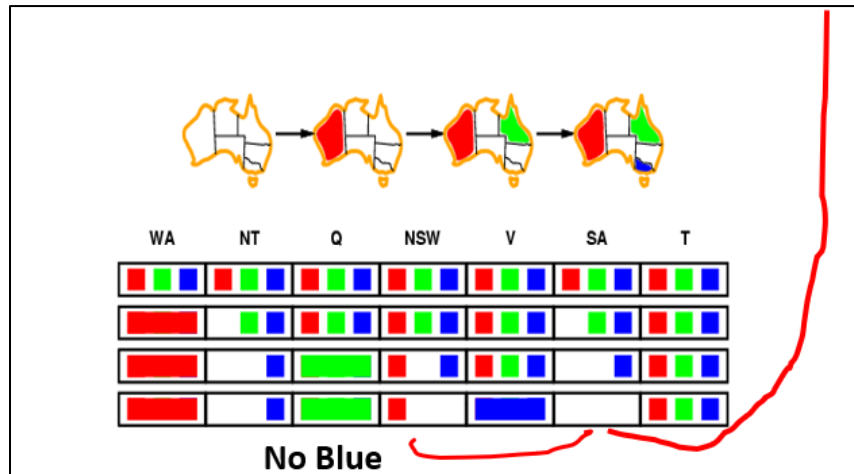
- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values



- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values

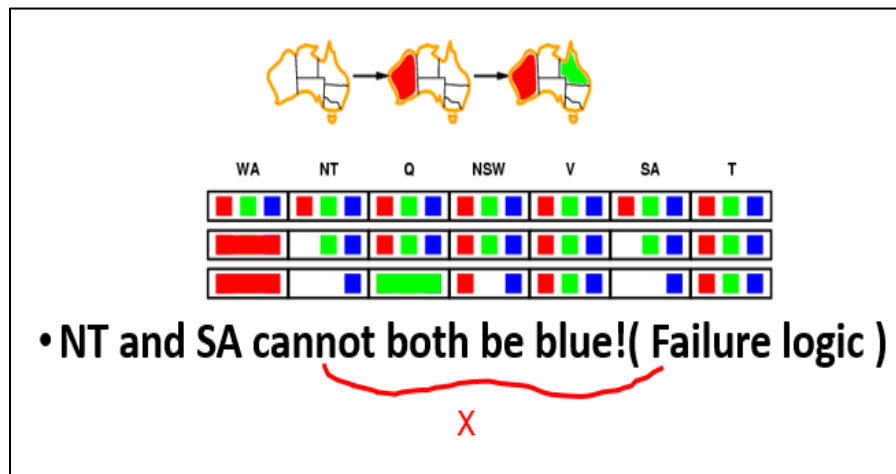


- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values left
- This path is not acceptable solution



Disadvantage: Forward checking, as a heuristic type of backtracking search; propagates information from assigned to unassigned variables. But it doesn't provide early detection for all failures (Constraint propagation in 5.3.2 will do)

- NT and SA cannot both be blue! (Failure logic)
- NT and SA cannot both be blue! (Failure logic)



4.9 Applicability of search techniques

- Searching algorithms are methods of finding a specific element or value in a collection of data, such as an array, a list, a tree, or a graph.
- Searching algorithms can have different approaches and complexities depending on the structure, size, and order of the data.
- Some examples of searching algorithms are linear search, binary search, depth-first search, breadth-first search, and hash table search.
- Searching algorithms can help you locate data for retrieval, modification, or verification. For example, you can use a searching algorithm to find a name in a phone book, a number in a sorted array, a path in a maze, or a keyword in a document.

Revision Sheet # 4**Assignment # 8**

- a) Given that is: $b = 10$, $m = 9$, $d = 5$, $l = 3$

Complete the next table, arrange the three types of search algorithms decently for both the space and time complexity.

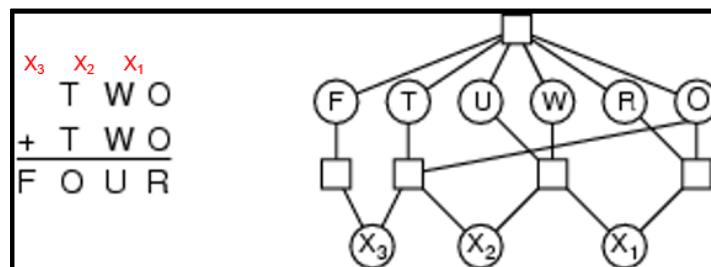
- b) Re-compute the same table, if the values changed to ($b = 5$, $m = 9$, $d = 10$, $l = 3$).
 c) Give your comments, comparing between your computations in (a) and (b)

	Depth First	Depth Limited	Iterative Deepening
Space Complexity			
Time Complexity			

Assignment # 9

Prepare a survey on **Communication** including:

definitions ,theories, applications ,problems and research areas and inter-relations with the AI Main Topics (reasoning, knowledge, planning , learning, communication, perception , ability to move and manipulate object).

Assignment # 10

- Given the Domains: $\{0,1,2,3,4,5,6,7,8,9\}$
- Constraints: $Alldiff(F,T,U,W,R,O)$

- find the Variables: $F T U W R O X_1 X_2 X_3$

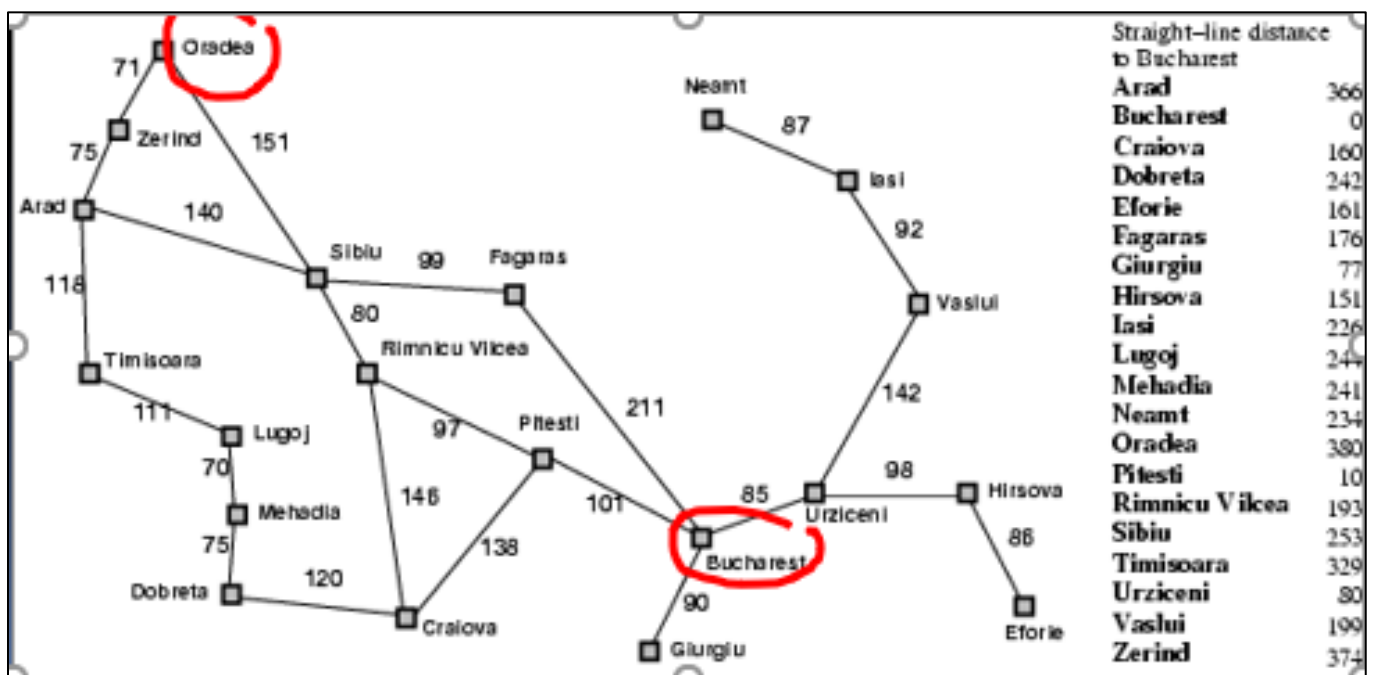
Assignment # 11

Prepare a survey on **communication** including:

definitions ,theories, applications ,problems and research areas and inter-relations with the AI Main Topics (reasoning, knowledge, planning , learning, communication, perception , ability to move and manipulate object).

Assignment # 12

- Use the greedy best first search algorithm to reach from Oradea to Bucharest
- Use A* search algorithm to reach from Oradea to Bucharest
- Compare between the two paths in (a), (b) and give your comment



CHAPTER 5

Learning

5.1 Definition of the concept of learning

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences. The ability to learn is possessed by humans, animals, and some machines; there is also evidence for some kind of learning in certain plants. Some learning is immediate, induced by a single event (e.g. being burned by a hot stove), but much skill and knowledge accumulate from repeated experiences. The changes made by learning often last a lifetime, and it is hard to distinguish learned material that seems to be "lost" from that which cannot be retrieved.

5.1.1 Factors affecting learning

- **External factors**

1. Inheritance.
2. Status of students.
3. Physical environment.

- **Internal factors**

There are several internal factors that affect learning. They are:

1. Goals or purposes.
2. Motivational behavior.
3. Interest.
4. Attention.

5. Drill or practice.
6. Fatigue.
7. Aptitude.
8. Attitude.
9. Emotional conditions.
10. Speed, Accuracy and retention.
11. Learning activities.
12. Testing.
13. Guidance.

5.2 Types of Learning

Any component of an agent can be improved by learning from data. The improvements, and the techniques used to make them, depend on four major factors:

- Which component is to be improved?
- What prior knowledge the agent already has?
- What representation is used for the data and the component?
- What feedback is available to learn from?

Components to be learned

The components of these agents include:

1. A direct mapping from conditions on the current state to actions.
2. A means to infer relevant properties of the world from the percept sequence.
3. Information about the way the world evolves and about the results of possible actions the agent can take.
4. Utility information indicating the desirability of world states.
5. Action-value information indicating the desirability of actions.

6. Goals that describe classes of states whose achievement maximizes the agent's utility.

Representation and prior knowledge

We have seen several examples of representations for agent components: propositional and first-order logical sentences for the components in a logical agent; Bayesian networks for the inferential components of a decision-theoretic agent, and so on. Effective learning algorithms have been devised for all of these representations.

There is another way to look at the various types of learning. We say that learning a general function or rule from specific input–output pairs is called inductive learning. We can also do analytical or deductive LEARNING: going from a known general rule to a new rule that is logically entailed, but is useful because it allows more efficient processing.

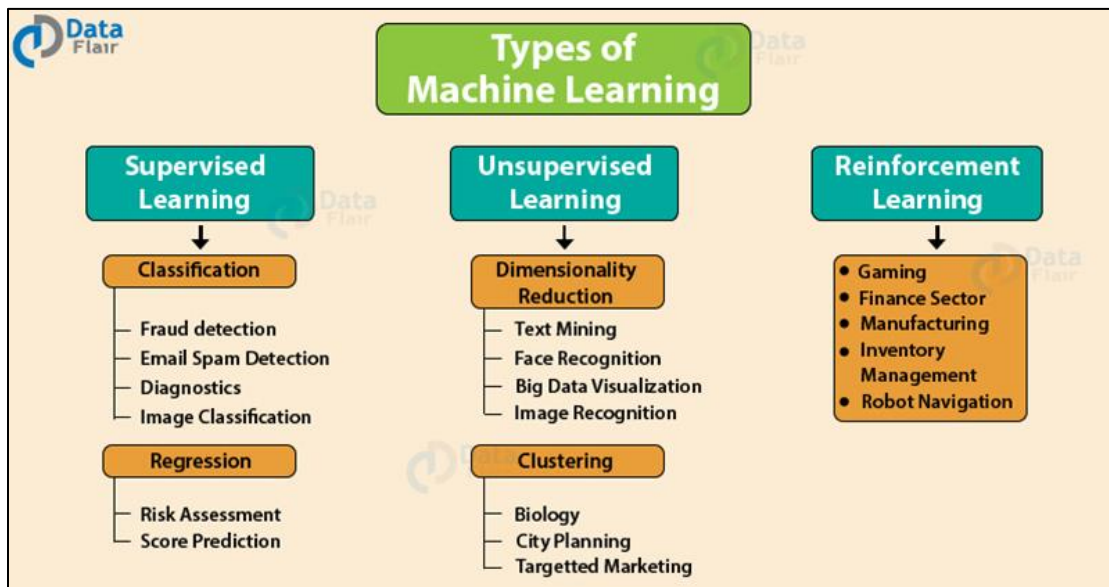
Feedback to learn from

There are three types of feedback that determine the three main types of learning:

- In **unsupervised learning** the agent learns patterns in the input even though no explicit feedback is supplied. The most common unsupervised learning task is **clustering**: detecting potentially useful clusters of input examples. For example, a taxi agent might gradually develop a concept of “good traffic days” and “bad traffic days” without ever being given labeled examples of each by a teacher.
- In **reinforcement learning** the agent learns from a series of reinforcements—rewards or punishments. For example, the lack of a tip at the end of the journey gives the taxi agent an indication that it did something wrong. The two points for a win at the end of a chess game tells the agent it did something

right. It is up to the agent to decide which of the actions prior to the reinforcement were most responsible for it.

- In **supervised learning** the agent observes some example input–output pairs and learns a function that maps from input to output.



5.2.1 Learning Algorithms

There are three components of learning algorithm as illustrated in the following figure.

Representation	Evaluation	Optimization
Instances	Accuracy/Error rate	Combinatorial optimization
<i>K</i> -nearest neighbor	Precision and recall	Greedy search
Support vector machines	Squared error	Beam search
Hyperplanes	Likelihood	Branch-and-bound
Naive Bayes	Posterior probability	Continuous optimization
Logistic regression	Information gain	Unconstrained
Decision trees	K-L divergence	Gradient descent
Sets of rules	Cost/Utility	Conjugate gradient
Propositional rules	Margin	Quasi-Newton methods
Logic programs		Constrained
Neural networks		Linear programming
Graphical models		Quadratic programming
Bayesian networks		
Conditional random fields		

5.3 Learning Decision Trees

Decision tree induction is one of the simplest and yet most successful forms of machine learning. We first describe the representation—the hypothesis space—and then show how to learn a good hypothesis.

5.3.1 The decision tree representation

A decision tree represents a function that takes as input a vector of attribute values and returns a “decision”—a single output value. The input and output values can be discrete or continuous. For now we will concentrate on problems where the inputs have discrete values and the output has exactly two possible values; this is Boolean classification, where each example input will be classified as true (a positive example) or false (a negative example).

A decision tree reaches its decision by performing a sequence of tests. Each internal node in the tree corresponds to a test of the value of one of the input attributes, A_i , and the branches from the node are labeled with the possible values of the attribute, $A_i = v_{ik}$.

Each leaf node in the tree specifies a value to be returned by the function. The decision tree representation is natural for humans; indeed, many “How To” manuals (e.g., for car repair) are written entirely as a single decision tree stretching over hundreds of pages.

As an example, we will build a decision tree to decide whether to wait for a table at a restaurant. The aim here is to learn a definition for the goal predicate **WillWait**. First we list the attributes that we will consider as part of the input:

1. Alternate: whether there is a suitable alternative restaurant nearby.
2. Bar : whether the restaurant has a comfortable bar area to wait in.
3. Fri/Sat: true on Fridays and Saturdays.

4. Hungry: whether we are hungry.
5. Patrons: how many people are in the restaurant (values are None, Some, and Full)?
6. Price: the restaurant's price range (\$, \$\$, \$\$\$).
7. Raining: whether it is raining outside.
8. **Reservation**: whether we made a reservation.
9. **Type**: the kind of restaurant (French, Italian, Thai, or burger).
10. **WaitEstimate**: the wait estimated by the host (0–10 minutes, 10–30, 30–60, or >60).

5.3.2 Expressiveness of decision trees

A Boolean decision tree is logically equivalent to the assertion that the goal attribute is true if and only if the input attributes satisfy one of the paths leading to a leaf with value true.

Writing this out in propositional logic, we have

$$\text{Goal} \Leftrightarrow (\text{Path1} \vee \text{Path2} \vee \cdots \vee \text{Path}_n),$$

Where each Path is a conjunction of attribute-value tests required to follow that path. Thus, the whole expression is equivalent to disjunctive normal form, which means that any function in propositional logic can be expressed as a decision tree.

As an example, the rightmost path is:

$$\text{Path} = (\text{Patrons} = \text{Full} \wedge \text{WaitEstimate} = 0\text{--}10)$$

We can show this in a general way. Consider the set of all Boolean functions on n attributes. How many different functions are in this set?

This is just the number of different truth tables that we can write down, because the function is defined by its truth table. A truth table over n attributes has 2^n rows, one for each combination of values of the attributes.

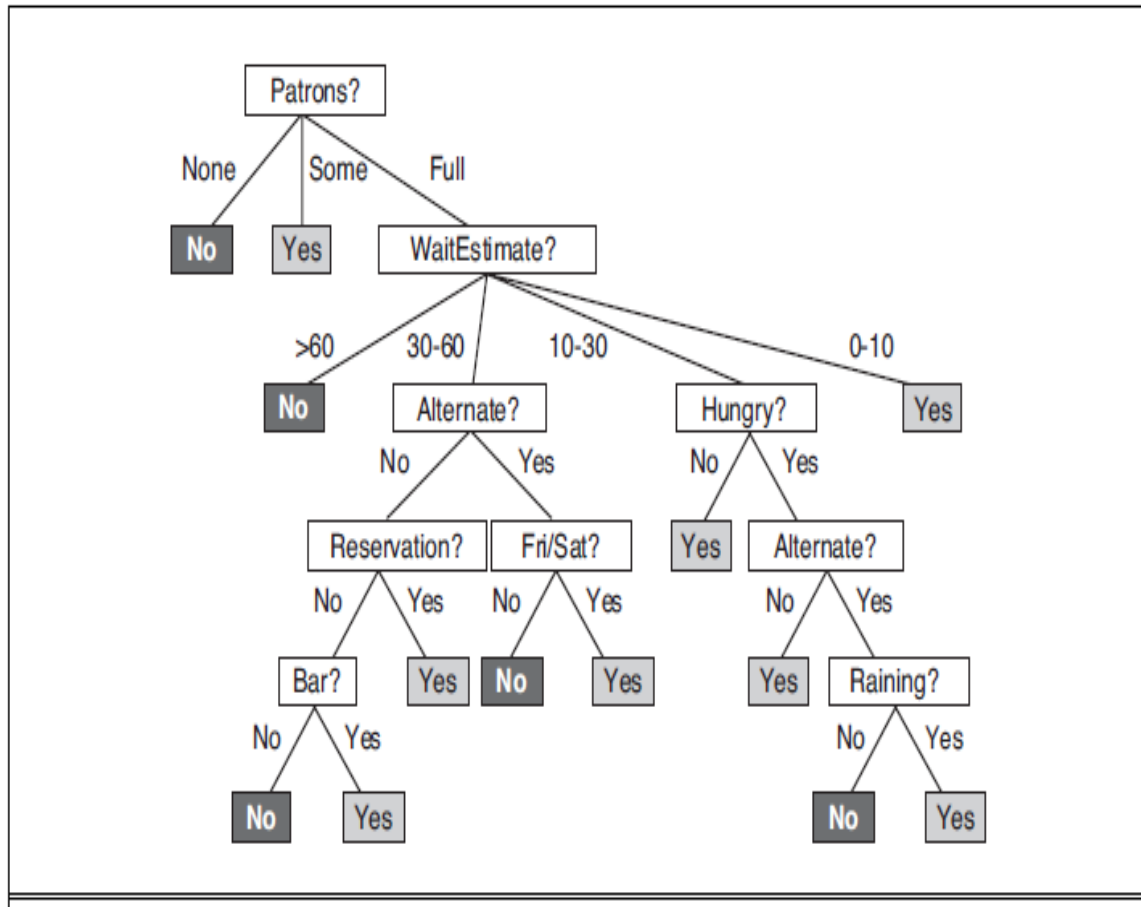
Consider the set of all Boolean functions on n attributes. How many different functions are in this set? This is just the number of different truth tables that we can write down, because the function is defined by its truth table. A truth table over n attributes has 2^n rows, one for each combination of values of the attributes. We can consider the “answer” column of the table as a 2^n -bit number that defines the function. That means there are $(2 \text{ power } 2^n)$ different functions.

For example, with just the ten Boolean attributes of our restaurant problem there are 2^{1024} or about 10^{308} different functions to choose from, and for 20 attributes there are over $10^{300,000}$. We will need some ingenious algorithms to find good hypotheses in such a large space.

5.3.3 Making decision trees from examples

An example for a Boolean decision tree consists of an (x, y) pair, where x is a vector of values for the input attributes, and y is a single Boolean output value.

A training set of 12 examples is shown in the table below. The positive examples are the ones in which the goal *WillWait* is true ($x_1, x_3, \dots$); the negative examples are the ones in which it is false ($x_2, x_5, \dots$).



Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	$y_1 = \text{Yes}$
x_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	$y_2 = \text{No}$
x_3	No	Yes	No	No	Some	\$	No	No	Burger	0-10	$y_3 = \text{Yes}$
x_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	$y_4 = \text{Yes}$
x_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	$y_5 = \text{No}$
x_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	$y_6 = \text{Yes}$
x_7	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	$y_7 = \text{No}$
x_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	$y_8 = \text{Yes}$
x_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	$y_9 = \text{No}$
x_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	$y_{10} = \text{No}$
x_{11}	No	No	No	No	None	\$	No	No	Thai	0-10	$y_{11} = \text{No}$
x_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	$y_{12} = \text{Yes}$

Examples for the restaurant domain

We want a tree that is consistent with the examples and is as small as possible. Unfortunately, no matter how we measure size, it is an intractable problem to find the smallest consistent tree; there is no way to efficiently search through the 2^{2^n} trees. With some simple heuristics, however, we can find a good approximate solution: a small (but not smallest) consistent tree.

The DECISION-TREE-LEARNING algorithm adopts a greedy divide-and-conquer strategy: always test the most important attribute first. This test divides the problem up into smaller sub-problems that can then be solved recursively.

By “most important attribute,” we mean the one that makes the most difference to the classification of an example. That way, we hope to get to the correct classification with a small number of tests, meaning that all paths in the tree will be short and the tree as a whole will be shallow.

The figure below shows that Type is a poor attribute, because it leaves us with four possible outcomes, each of which has the same number of positive as negative examples. On the other hand, in (b) we see that Patrons is a fairly important attribute, because if the value is None or Some, then we are left with example sets for which we can answer definitively (No and Yes, respectively).

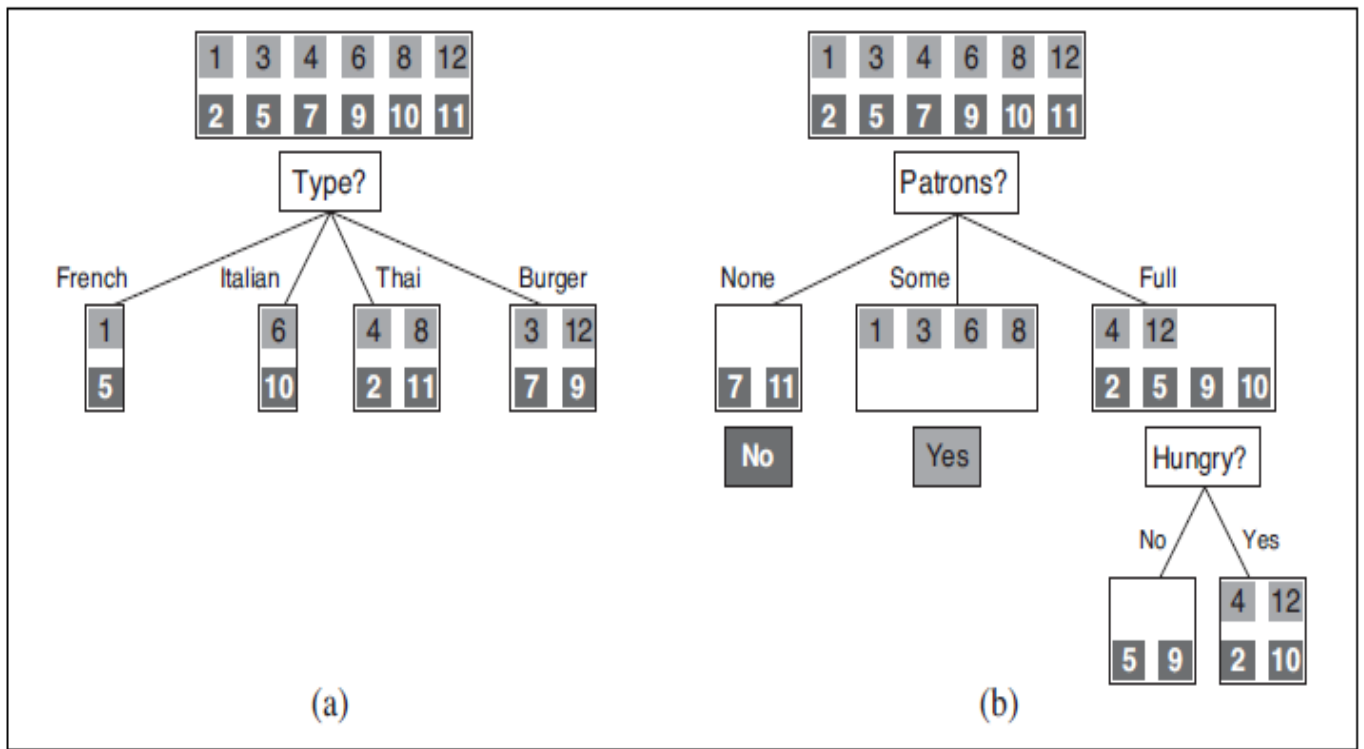
If the value is full, we are left with a mixed set of examples. In general, after the first attribute test splits up the examples, each outcome is a new decision tree learning problem in itself, with fewer examples and one less attribute. There are four cases to consider for these recursive problems:

1. If the remaining examples are all positive (or all negative), then we are done: we can answer Yes or No. Figure (b) shows examples of this happening in the None and Some branches.

2. If there are some positive and some negative examples, then choose the best attribute to split them. Figure (b) shows Hungry being used to split the remaining examples.

3. If there are no examples left, it means that no example has been observed for this combination of attribute values, and we return a default value calculated from the plurality classification of all the examples that were used in constructing the node's parent. These are passed along in the variable parent examples.

4. If there are no attributes left, but both positive and negative examples, it means that these examples have exactly the same description, but different classifications. This can happen because there is an error or **noise** in the data; because the domain is nondeterministic; or because we can't observe an attribute that would distinguish the examples. The best we can do is return the plurality classification of the remaining examples. The best we can do is return the plurality classification of the remaining examples.



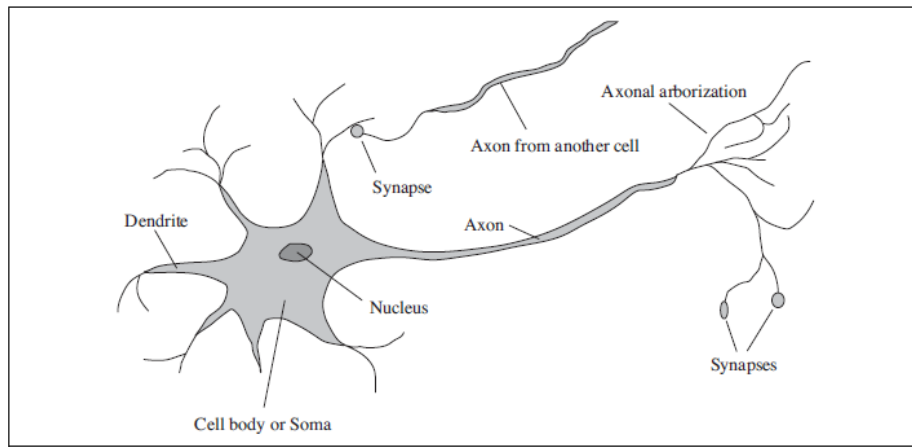
Splitting the examples by testing on attributes. At each node we show the positive (light boxes) and negative (dark boxes) examples remaining.

- (a) Splitting on *Type* brings us no nearer to distinguishing between positive and negative examples.
- (b) Splitting on *Patrons* does a good job of separating positive and negative examples? After splitting on *Patrons*, *Hungry* is a fairly good second test.

5.4 Artificial Neural Networks

We turn now to what seems to be a somewhat unrelated topic: the brain. In fact, as we will see, the technical ideas we have discussed so far in this chapter turn out to be useful in building mathematical models of the brain's activity; conversely, thinking about the brain has helped in extending the scope of the technical ideas.

In neuroscience, the hypothesis that mental activity consists primarily of electrochemical activity in networks of brain cells is called **neurons**. The figure below shows a schematic diagram of a typical neuron.)



The parts of a nerve cell or neuron. Each neuron consists of a cell body, or soma, that contains a cell nucleus. Branching out from the cell body are a number of fibers called dendrites and a single long fiber called the axon. The axon stretches out for a long distance, much longer than the scale in this diagram indicates.

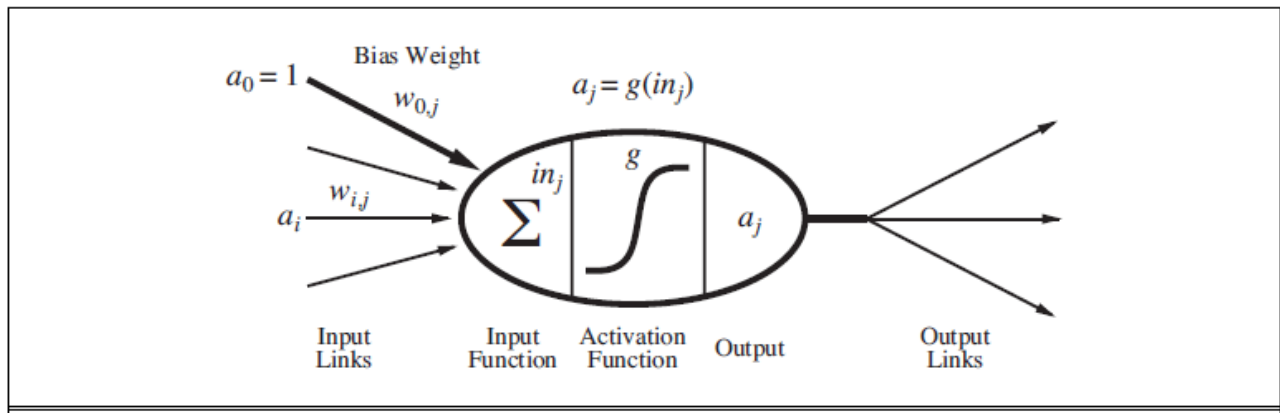
Typically, an axon is 1 cm long (100 times the diameter of the cell body), but can reach up to 1 meter. A neuron makes connections with 10 to 100,000 other neurons at junctions called synapses. Signals are propagated from neuron to neuron by a complicated electrochemical reaction. The signals control brain activity in the short term and also enable long-term changes in the connectivity of neurons.

These mechanisms are thought to form the basis for learning in the brain. Most information processing goes on in the cerebral cortex, the outer layer of the brain. The basic organizational unit appears to be a column of tissue about 0.5 mm in diameter, containing about 20,000 neurons and extending the full depth of the cortex about 4 mm in humans).

Inspired by this hypothesis, some of the earliest AI work aimed to create artificial **neural networks**. (Other names for the field include **connectionism**, **parallel distributed processing**, and **neural computation**.)

The figure below shows a simple mathematical model of the neuron devised by McCulloch and Pitts (1943). Roughly speaking, it “fires” when a linear combination of its inputs exceeds some (hard or soft) threshold—that is, it implements a linear classifier of the kind described in the preceding section. A neural network is just a collection of units connected together; the properties of the network are determined by its topology and the properties of the “neurons.” Since 1943, much more detailed and realistic models have been developed, both for neurons and for larger systems in the brain, leading to the modern field of **computational neuroscience**. On the other hand, researchers in AI and statistics became interested in the more abstract properties of neural networks, such as their ability to perform distributed computation, to tolerate noisy inputs, and to learn.

Although we understand now that other kinds of systems—including Bayesian networks—have these properties, neural networks remain one of the most popular and effective forms of learning system and are worthy of study in their own right.



A simple mathematical model for a neuron. The unit's output activation is

$$a_j = g\left(\sum_{i=0}^n w_{i,j} a_i\right)$$

Where a_i is the output activation of unit i and $w_{i,j}$ is the weight on the link from unit i to this unit.

5.4.1 Neural network structures

Neural networks are composed of nodes or units (see previous figure) connected by directed links. A link from unit i to unit j serves to propagate the activation a_i LINK from i to j . Each link also has a numeric weight $w_{i,j}$ associated with it, which determines the strength and sign of the connection. Just as in linear regression models, each unit has a dummy input $a_0 = 1$ with an associated weight $w_{0,j}$. Each unit j first computes a weighted sum of its inputs:

$$in_j = \sum_{i=0}^n w_{i,j} a_i$$

Then it applies an **activation function** g to this sum to derive the output:

$$a_j = g(in_j) = g\left(\sum_{i=0}^n w_{i,j} a_i\right)$$

The activation function g is typically either a hard threshold in which case the unit is called a perceptron, or PERCEPTRON a logistic function, in which case the term sigmoid perceptron is sometimes used.

Having decided on the mathematical model for individual “neurons,” the next task is to connect them together to form a network. There are two fundamentally distinct ways to do this. A feed-forward network has connections only in one direction—that is, it forms a directed acyclic graph. Every node receives input from “upstream” nodes and delivers output to “downstream” nodes; there are no loops.

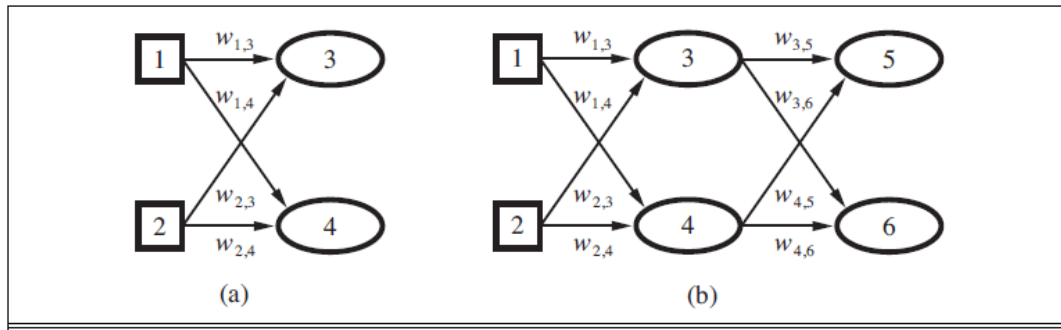
A feed-forward network represents a function of its current input; thus, it has no internal state other than the weights themselves. A recurrent network, on the other hand, feeds its outputs back into its own inputs. This means that the activation levels of the network form a dynamical system that may reach a stable state or exhibit oscillations or even chaotic behavior. Moreover, the response of the network to a given input depends on its initial state, which may depend on previous inputs.

Feed-forward networks are usually arranged in layers, such that each unit receives input only from units in the immediately preceding layer. In the next two subsections, we will look at single-layer networks, in which every unit connects directly from the network’s inputs to its outputs, and multilayer networks, which have one or more layers of hidden units that are not connected to the outputs of the network. So far in this chapter, we have considered only learning problems with a single output variable y , but neural networks are often used in cases where multiple outputs are appropriate.

For example, if we want to train a network to add two input bits, each a 0 or a 1, we will need one output for the sum bit and one for the carry bit. Also, when the learning problem involves classification into more than two classes—for example, when learning to categorize images of handwritten digits—it is common to use one output unit for each class.

5.4.2 Single-layer feed-forward neural networks (perceptrons)

A network with all the inputs connected directly to the outputs is called a single-layer neural PERCEPTRON network, or a perceptron network. The figure below shows a simple two-input, two-output perceptron network.



(a) A perceptron network with two inputs and two output units.

(b) A neural network with two inputs, one hidden layer of two units, and

With such a network, we might hope to learn the two-bit adder function, for example. Here are all the training data we will need:

x_1	x_2	y_3 (carry)	y_4 (sum)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

The first thing to notice is that a perceptron network with m outputs is really m separate networks, because each weight affects only one of the outputs. Thus, there will be m separate training processes.

The linearly separable functions constitute just a small fraction of all Boolean functions; the inability of perceptrons to learn even such simple functions as XOR was a significant setback to the promising neural network, community in the 1960s. Perceptrons are far from useless, that logistic regression (i.e., training a sigmoid

perceptron) is even today a very popular and effective tool. Moreover, a perceptron can represent some quite “complex” Boolean functions very compactly.

For example, the **majority function**, which outputs a **1** only if more than half of its n inputs are **1**, can be represented by a perceptron with each $w_i = 1$ and with $W_0 = -n/2$. A decision tree would need exponentially many nodes to represent this function.

As a conclusion, the limitations of the single layer network has led to the development of multi-layer feed-forward networks with one or more hidden layers, called multi-layer perceptron (MLP) networks.

5.4.3 Multilayer feed-forward neural networks

(McCulloch and Pitts, 1943) were well aware that a single threshold unit would not solve all their problems. In fact, their paper proves that such a unit can represent the basic Boolean functions AND, OR, and NOT and then goes on to argue that any desired functionality can be obtained by connecting large numbers of units into (possibly recurrent) networks of arbitrary depth. The problem was that nobody knew how to train such networks. This turns out to be an easy problem if we think of a network the right way: as a function $h_w(\mathbf{x})$ parameterized by the weights $\mathbf{w}$.

Consider the simple network which has two input units, two hidden units, and two output unit. (In addition, each unit has a dummy input fixed at 1.) Given an input vector $\mathbf{x}=(x_1, x_2)$, the activations of the input units are set to $(a_1, a_2) = (x_1, x_2)$.

The output at unit 5 is given by:

$$\begin{aligned} a_5 &= g(w_{0,5} + w_{3,5} a_3 + w_{4,5} a_4) \\ &= g(w_{0,5} + w_{3,5} g(w_{0,3} + w_{1,3} a_1 + w_{2,3} a_2) + w_{4,5} g(w_{0,4} + w_{1,4} a_1 + w_{2,4} a_2)) \\ &= g(w_{0,5} + w_{3,5} g(w_{0,3} + w_{1,3} x_1 + w_{2,3} x_2) + w_{4,5} g(w_{0,4} + w_{1,4} x_1 + w_{2,4} x_2)). \end{aligned}$$

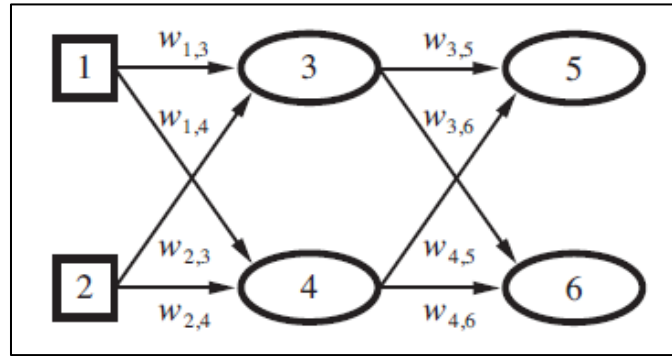
Thus, we have the output expressed as a function of the inputs and the weights. A similar expression holds for unit 6. As long as we can calculate the derivatives of such expressions with respect to the weights, we can use the gradient-descent loss-minimization method to train the network.

Before investigating into learning rules, let us look at the ways in which networks generate complicated functions. First, remember that each unit in a sigmoid network represents a soft threshold in its input space. With one hidden layer and one output layer, each output unit computes a soft-thresholded linear combination of several such functions.

With a single, sufficiently large hidden layer, it is possible to represent any continuous function of the inputs with arbitrary accuracy; with two layers, even discontinuous functions can be represented. Unfortunately, for any particular network structure, it is harder to characterize exactly which functions can be represented and which ones cannot.

5.4.4 Learning in multilayer networks

First, let us dispense with one minor complication arising in multilayer networks: interactions among the learning problems when the network has multiple outputs. In such cases, we should think of the network as implementing a vector function h_w rather than a scalar function h_w ; for example, the network in the figure below returns a vector $[a_5, a_6]$.



Similarly, the target output will be a vector y . Whereas a perceptron network decomposes into m separate learning problems for an m -output problem, this decomposition fails in a multilayer network. For example, both a_5 and a_6 depend on all of the input-layer weights, so updates to those weights will depend on errors in both a_5 and a_6 . Fortunately, this dependency is very simple in the case of any loss function that is additive across the components of the error vector $y - h_w(x)$. For the L_2 loss, we have, for any weight w ,

$$\frac{\partial}{\partial w} \text{Loss}(\mathbf{w}) = \frac{\partial}{\partial w} |\mathbf{y} - \mathbf{h}_w(\mathbf{x})|^2 = \frac{\partial}{\partial w} \sum_k (y_k - a_k)^2 = \sum_k \frac{\partial}{\partial w} (y_k - a_k)^2$$

Where the index k ranges over nodes in the output layer. Each term in the final summation is just the gradient of the loss for the k th output, computed as if the other outputs did not exist. Hence, we can decompose an m -output learning problem into m learning problems, provided we remember to add up the gradient contributions from each of them when updating the weights.

The major complication comes from the addition of hidden layers to the network. Whereas the error $y - h_w$ at the output layer is clear, the error at the hidden layers seems mysterious because the training data do not say what value the hidden nodes should have. Fortunately, it turns out that we can back-propagate **BACK-PROPAGATION** the error from the output layer to the hidden layers. The back-propagation process emerges directly from a derivation of the overall error

gradient. First, we will describe the process with an intuitive justification; then, we will show the derivation.

At the output layer, we have multiple output units, so let Err_k be the k th component of the error vector $y - h_w$. We will also find it convenient to define a modified error $\Delta_k = \text{Err}_k \times g'(\text{ink})$, so that the weight update rule becomes:

$$\mathbf{w}_{j,k} \leftarrow \mathbf{w}_{j,k} + \alpha \times \mathbf{a}_j \times \Delta_k$$

To update the connections between the input units and the hidden units, we need to define a quantity analogous to the error term for output nodes. Here is where we do the error backpropagation.

The idea is that hidden node j is “responsible” for some fraction of the error Δ_k in each of the output nodes to which it connects. Thus, the Δ_k values are divided according to the strength of the connection between the hidden node and the output node and are propagated back to provide the Δ_j values for the hidden layer. The propagation rule for the Δ values is the following:

$$\Delta_j = g'(in_j) \sum_k w_{j,k} \Delta_k .$$

Now the weight-update rule for the weights between the inputs and the hidden layer is essentially identical to the update rule for the output layer:

$$\mathbf{w}_{i,j} \leftarrow \mathbf{w}_{i,j} + \alpha \times \mathbf{a}_i \times \Delta_j$$

The back-propagation process can be summarized as follows:

- Compute the Δ values for the output units, using the observed error.
- Starting with output layer, repeat the following for each layer in the network, until the earliest hidden layer is reached:
 - Propagate the Δ values back to the previous layer.

- Update the weights between the two layers.

The detailed algorithm is shown in the following figure:

```

function BACK-PROP-LEARNING(examples, network) returns a neural network
  inputs: examples, a set of examples, each with input vector x and output vector y
           network, a multilayer network with L layers, weights  $w_{i,j}$ , activation function g
  local variables:  $\Delta$ , a vector of errors, indexed by network node

  repeat
    for each weight  $w_{i,j}$  in network do
       $w_{i,j} \leftarrow$  a small random number
    for each example (x, y) in examples do
      /* Propagate the inputs forward to compute the outputs */
      for each node i in the input layer do
         $a_i \leftarrow x_i$ 
      for  $\ell = 2$  to L do
        for each node j in layer  $\ell$  do
           $in_j \leftarrow \sum_i w_{i,j} a_i$ 
           $a_j \leftarrow g(in_j)$ 
      /* Propagate deltas backward from output layer to input layer */
      for each node j in the output layer do
         $\Delta[j] \leftarrow g'(in_j) \times (y_j - a_j)$ 
      for  $\ell = L - 1$  to 1 do
        for each node i in layer  $\ell$  do
           $\Delta[i] \leftarrow g'(in_i) \sum_j w_{i,j} \Delta[j]$ 
      /* Update every weight in network using deltas */
      for each weight  $w_{i,j}$  in network do
         $w_{i,j} \leftarrow w_{i,j} + \alpha \times a_i \times \Delta[j]$ 
  until some stopping criterion is satisfied
  return network

```

5.4.5 Applicability of Neural Networks in learning

1. Social Media

As we have studied earlier, Artificial Neural Network is a great tool for processing images. It captures the detailing of the features and acknowledges them quite

effectively. This feature allows ANN to find extensive use in the area of social networking. Major social networking applications make use of ANN extensively.

As an example, let us consider the “People you might know” element on the leading networking sites. As you may have experienced, social networking sites suggest people have accounts on the network based on the pictures that you post online. This feature is nothing but the application of Artificial Neural Networks in real life.

With the help of facial recognition, the algorithm is able to suggest people that you are friends with. It also helps you tag the people based on their facial features.

The algorithm analyzes your profile, interests as well as friend list to suggest to you a list of people that you might know who have a social presence on the network. It uses data points in the images and identifies a person on the basis of the previously stored database. It tends to look for 100 reference points on the image and then matches it with the database information. This helps industries in providing efficient services to their users using ANNs.

2. Marketing and Sales

Product recommendation and image search are some of the most common applications of ANNs in the field of Marketing and Sales. Whenever you try searching for some products online, the e-commerce sites automatically start suggesting products that are similar to your previous interests or things that may compliment the product that you have bought previously. The algorithm stores the information about your interests on the database and the trained models then identify a pattern in your shopping style based on ANN algorithms.

Not only e-commerce sites but food delivery services have also started adopting this model for providing effective services to their customers. The tech giants not only suggest you with similar products on their sites, they even pester you with the same on social networking sites. The collected database of the algorithm helps the neural

network in finding effective marketing strategies. This is what paves the way for a new-age marketing schema and is done by implementing personalized marketing.

Artificial Neural Network proves beneficial in identifying the customer's likes, interests, and needs based on previous order history and search results. This helps industries in identifying the surge in demand for a certain product and tailoring effective marketing strategies accordingly.

Apart from the marketing schema, ANNs are extensively used for stock exchange prediction. The accuracy of the prediction model is commendable. Companies make use of ANN to predict the prices of the stocks by analyzing and identifying the patterns of the stock price and the environment.

The trained ANN model finds the relationship between stock prices and political, marketing, and other factors that may affect the stock pricing schema. Companies like MJ Futures assured a wholesome of 199.2% of returns over a time span of 2 years. All because of ANN, consumers, as well as stockholders, are able to enjoy the immense benefits from the predictions.

3. Facial Recognition

To date, biometric authentication is one of the most secure methods for user authentication. ANN is proving to do a significant job in recognizing faces on the basis of the images we feed to the system. Well-trained ANN models are also capable of classifying images on the basis of the faces it recognizes through the dataset.

First, it takes in the image as input, and pre-processing begins on the image. This processed image then goes to the next stage for dimensionality reduction. Furthermore, this reduced image is then used for the recognition of the various faces in the image. The two networks that we use while developing this network are fully connected feed forward neural network and PCA for dimensionality reduction.

4. Healthcare

Trained ANN models are an emerging star in the field of biological research. ANNs capture standardized datasets and try to find the pattern in the growth of symptoms and the viruses. The models approach the functioning of the biological clusters in a very effective manner. Apart from these, ANNs find a wide range of applications in the cardiology sector.

ANNs are extensively used for diagnosis, medical imaging, analysis, and radiology in various disciplines of medicines. Applications of ANN are quite effective in pharmacoepidemiology and medical data mining. Apart from these, Artificial Neural Networks also find their applications in the discipline of Oncology. The ANN models are trained so that they can identify and detect cancerous cells at microscopic levels with accuracy.

Facial analysis is used widely for studying the effects of various rare diseases that may manifest in physical characteristics. So, extensive implementation of ANNs in the medical discipline helps in accurate diagnostics abilities and leads to medical evolution.

5. Text Classification and Categorization

Text classification is an important aspect of many applications like web searching, information filtering, readability, language identification, and sentiment analysis. In an attempt to build a CNN on top of word2vec, Convolutional Neural Networks for Sentence Classification was presented by **Yoon Kim**. This suggested model had the task of filtering negative as well as positive comments on a sentimental basis.

The results of the large number of tests that were conducted revealed that when you tune the hyperparameters to a smaller extent, the model depicts amazing suggestions that are used by the pre-trained vectors. The results were quite commendable for a thesaurus as well. The model showed a requirement of 99.96% of training and depicted 98.40% accuracy.

In a study conducted as an **Amazon Review data set**, the researchers tried to construct a sentiment polarity dataset with two negative as well as positive datasets. The results were a mix of 97.57% of training accuracy as well as 95.07% of testing accuracy.

One more advantage of this network is paraphrasing the given sentences. While developing quizzical applications involving questions and answers, it becomes very important for the developer to analyze if two different sentences are having the same meaning. Paraphrasing helps the machine in understanding the similarity between the sentences by rearranging the words to form the same sentence. ANN is a crucial part of the development of this process.

6. Part of Speech Tagging

POS or Part-of-Speech tagging is used in many disciplines like parsing, text-to-speech conversion, information extraction, and many others. In the prescribed model Part-of-Speech Tagging with Bidirectional Long Short-Term Memory, the testing by **Wall Street Journal data from Penn Treebank III** showed a 97.40% tagging accuracy.

7. Aerospace

Due to the effective ability to establish a non-linear relationship between input and output, ANNs find huge applications in Aerospace Engineering. You can train ANN models to perform fault diagnosis, high-performance auto-piloting, securing aircraft controls, and modeling key dynamic simulations.

8. Image Compression

The main idea behind image compression is equalizing the size of the input and output layers. The intermediate layer while producing the output is comparatively smaller. Hence, the compression ratio is the comparison between the input layer and the intermediate layer.

9. Compression Ratio= Input Layer/ Intermediate Layer

The main focus of image compression is to store, encrypt and re-generate the actual image again. Hence, in such networks, the input image is itself used for model training.

10. Speech Recognition

From times unknown, speech has been a key factor of communication for humans. So, as a matter of fact, humans would expect it from the machines as well to be capable enough to connect with them with the help of speech. Despite the technical advancements, humans still feel the need for a sophisticated high-level language to communicate with their machines. In order to help machines have more humane behavior, we have to help them communicate through various spoken languages.

ANN is playing a major part in the development of these speeches recognizing techniques for the machines. ANNs like Multilayer Networks, Kohonen Self-Organizing Maps, and Multilayer Network with Recurrent connections play a significant role in achieving the expected results.

11. Handwriting Recognition

In the introduction of ANN in the previous article, we came across the application of ANN in handwriting recognition. Handwriting recognition is a milestone in the field of fraud detection. The algorithm takes in the bitmap pattern of handwritten characters and outputs the correct letter or digit. The two most widely used applications of handwriting recognition are

Optical character recognition for data entry

Validation of signature on a bank cheque

The algorithm accurately analyzes the handwritten characters and thus provides a surge in the security system. The model is trained in such a way that we first feed the

input to the first layer. The first layer handles the input queries while the last layer handles the output. The middle layers are the processing layers and have no connecting link to the outside world. The flow for data is unidirectional and thus we call it a feed-forward network. There is no link between the perceptrons of the same layer.

Apart from this, handwriting recognition is also an important part of authenticating users for any system. To date, signatures are a crucial part of the authentication of users into the system. This technique provides a high level of security since you cannot breach the system easily.

12. Personal Assistance

Though Machine Learning has tremendously left its impact on a variety of disciplines, the most fascinating one amongst them has to be the way personal assistants work.

Amazon Alexa, Google Home, and Siri are some of the most heard of names in the arena of Personal Assistants. These assistants can help you in the making of plethora works more convenient and accountable. Their work can range from helping you find something on the Internet over just a voice command to helping you look for your holiday destinations.

Personal Assistants can take care of all these in just a jiffy. The major factor working behind these assistants is Natural Language Processing (NLP). NLP makes use of Artificial Neural Networks to attain results with high accuracy and reliability.

13. Weather Forecasting

Conventionally, the meteorological department made forecasts that were never accurate before artificial intelligence came into force. Weather Forecasting is primarily undertaken to anticipate the upcoming weather conditions with prior

knowledge of the weather by observing the past weather patterns. In the times of Machine Learning, weather forecasts are even used to predict the possibilities of natural disasters.

Other Fields of Applications

Apart from the various disciplines that we have mentioned above, we use ANN in a plethora of other disciplines as well. Using the Multilayer Perceptron Network, we can deploy models for Machine Diagnostics and Medical Diagnostics. They use the tan sigmoid function as the activation function for processing.

Classification Supervised Algorithms along with ANNs are a perfect combination for Portfolio Management. Like Medical Diagnostics, Portfolio Management also uses the tan sigmoid function as the activation function for processing. The list of the applications of ANN goes on with the addition of fields like Fraud Detection, Credit Rating, Targeted Marketing, and much more.

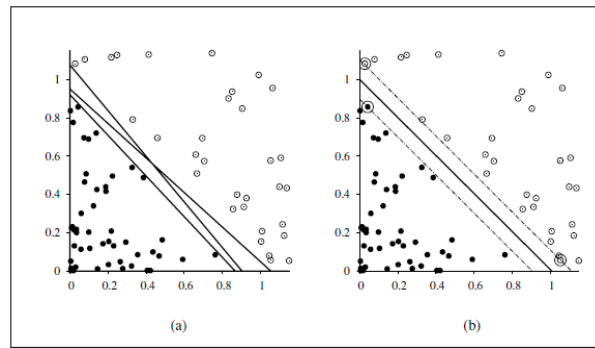
5.5 Support Vector Machines

The support vector machine or SVM SUPPORT VECTOR framework is currently the most popular approach for MACHINE “off-the-shelf” supervised learning: if you don’t have any specialized prior knowledge about a domain, then the SVM is an excellent method to try first. There are three properties that make SVMs attractive:

1. SVMs construct a maximum margin separator—a decision boundary with the largest possible distance to example points. This helps them generalize well.
2. SVMs create a linear separating hyperplane, but they have the ability to embed the data into a higher-dimensional space, using the so-called kernel trick.
3. SVMs are a nonparametric method—they retain training examples and potentially need to store them all. On the other hand, in practice they often end up retaining only

a small fraction of the number of examples—sometimes as few as a small constant time the number of dimensions. Thus, SVMs combine the advantages of nonparametric and parametric models: they have the flexibility to represent complex functions, but they are resistant to overfitting.

You could say that SVMs are successful because of one key insight and one neat trick. We will cover each in turn.



Support vector machine classification:

(a) Two classes of points (black and white circles) and three candidate linear separators.

(b) The maximum margin separator (heavy line), is at the midpoint of the **margin** (area between dashed lines).

The **support vectors** (points with large circles) are the examples closest to the separator.

The examples, so from the point of view of 0/1 loss, each would be equally good. Logistic regression would find some separating line; the exact location of the line depends on all the example points. The key insight of SVMs is that some examples are more important than others, and that paying attention to them can lead to better generalization.

Consider the lowest of the three separating lines in (a). It comes very close to 5 of the black examples. Although it classifies all the examples correctly, and thus minimizes loss, it should make you nervous that so many examples are close to the line; it may be that other black examples will turn out to fall on the other side of the line.

SVMs address this issue: Instead of minimizing expected empirical loss on the training data, SVMs attempt to minimize expected generalization loss. We don't know where the as-yet-unseen points may fall, but under the probabilistic assumption that they are drawn from the same distribution as the previously seen examples, there are some arguments from computational learning theory suggesting that we minimize generalization loss by choosing the separator that is farthest away from the examples we have seen so far. We call this separator, shown in the previous figure MAXIMUM MARGIN (b) the maximum margin separator. The margin SEPARATOR MARGIN is the width of the area bounded by dashed lines in the figure—twice the distance from the separator to the nearest example point.

Now, how do we find this separator? Before showing the equations, some notation:

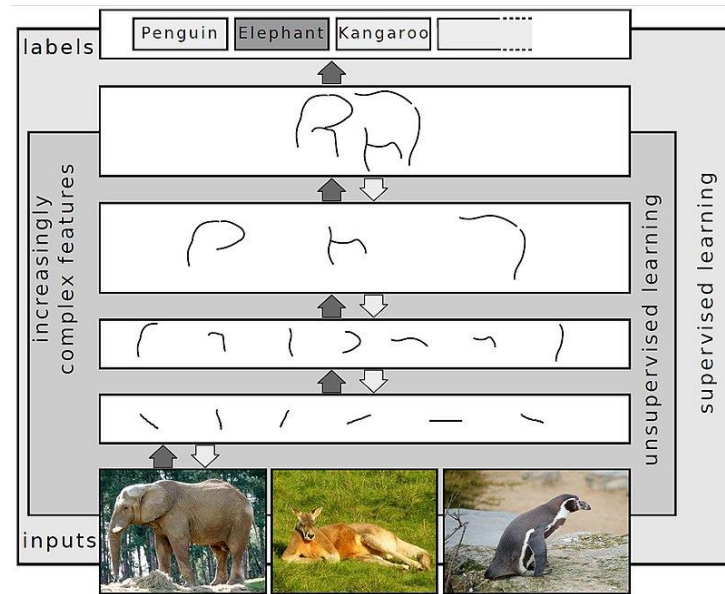
Traditionally SVMs use the convention that class labels are +1 and -1, instead of the +1 and 0 we have been using so far. Also, where we put the intercept into the weight vector w (and a corresponding dummy 1 value into x_j , 0), SVMs do not do that; they keep the intercept as a separate parameter, b . With that in mind, the separator is defined as the set of points

$\{x: w \cdot x + b = 0\}$. We could search the space of w and b with gradient descent to find the parameters that maximize the margin while correctly classifying all the examples.

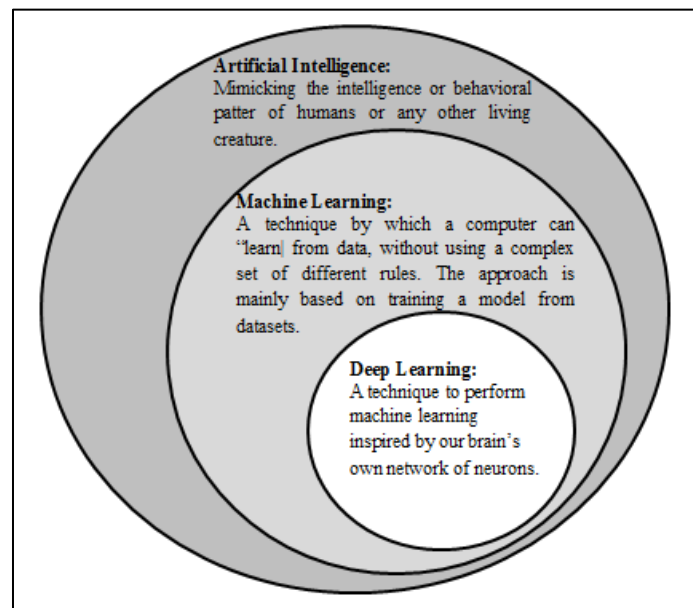
However, it turns out there is another approach to solving this problem.

5.6 Deep Learning

Deep learning is a class of machine learning algorithms that uses multiple layers to progressively extract higher-level features from the raw input. For example, in image processing, lower layers may identify edges, while higher layers may identify the concepts relevant to a human such as digits or letters or faces.



Deep learning (also known as deep structured learning) is part of a broader family of machine learning methods based on artificial neural networks with representation learning. Learning can be supervised, semi-supervised or unsupervised.



Deep-learning architectures such as deep neural networks, deep belief networks, recurrent neural networks and convolutional neural networks have been applied to fields including computer vision, machine vision, speech recognition, natural language processing, audio recognition, social network filtering, machine

translation, bioinformatics, drug design, medical image analysis, material inspection and board game programs, where they have produced results comparable to and in some cases surpassing human expert performance.

Artificial neural networks (ANNs) were inspired by information processing and distributed communication nodes in biological systems. ANNs have various differences from biological brains. Specifically, neural networks tend to be static and symbolic, while the biological brain of most living organisms is dynamic (plastic) and analog.

The adjective "deep" in deep learning refers to the use of multiple layers in the network. Early work showed that a linear perceptron cannot be a universal classifier, and then that a network with a nonpolynomial activation function with one hidden layer of unbounded width can on the other hand so be. Deep learning is a modern variation which is concerned with an unbounded number of layers of bounded size, which permits practical application and optimized implementation, while retaining theoretical universality under mild conditions. In deep learning the layers are also permitted to be heterogeneous and to deviate widely from biologically informed connectionist models, for the sake of efficiency, trainability and understandability, whence the "structured" part.

5.6.1 Deep Learning & Natural Language Processing

Neural networks have been used for implementing language models since the early 2000s. LSTM helped to improve machine translation and language modeling.

Other key techniques in this field are negative sampling and word embedding. Word embedding, such as word2vec, can be thought of as a representational layer in a deep learning architecture that transforms an atomic word into a positional representation of the word relative to other words in the dataset; the position is represented as a point in a vector space. Using word embedding as an RNN input layer allows the network to parse sentences and phrases using an effective compositional vector

grammar. A compositional vector grammar can be thought of as probabilistic context free grammar (PCFG) implemented by an RNN.

Recursive auto-encoders built atop word embeddings can assess sentence similarity and detect paraphrasing. Deep neural architectures provide the best results for constituency parsing, sentiment analysis, information retrieval, spoken language understanding, machine translation, contextual entity linking, writing style recognition, Text classification and others. Recent developments generalize word embedding to sentence embedding.

Google Translate (GT) uses a large end-to-end long short-term memory network. Google Neural Machine Translation (GNMT) uses an example-based machine translation method in which the system "learns from millions of examples." It translates "whole sentences at a time, rather than pieces. Google Translate supports over one hundred languages. The network encodes the "semantics of the sentence rather than simply memorizing phrase-to-phrase translations". GT uses English as an intermediate between most language pairs.

5.6.2 Deep Learning & Commercial activity

Facebook's AI lab performs tasks such as automatically tagging uploaded pictures with the names of the people in them.

Google's DeepMind Technologies developed a system capable of learning how to play Atari video games using only pixels as data input. In 2015 they demonstrated their AlphaGo system, which learned the game of Go well enough to beat a professional Go player. Google Translate uses a neural network to translate between more than 100 languages.

In 2015, Blippar demonstrated a mobile augmented reality application that uses deep learning to recognize objects in real time.

In 2017, Covariant.ai was launched, which focuses on integrating deep learning into factories.

As of 2008, researchers at The University of Texas at Austin (UT) developed a machine learning framework called Training an Agent Manually via Evaluative Reinforcement, or TAMER, which proposed new methods for robots or computer programs to learn how to perform tasks by interacting with a human instructor. First developed as TAMER, a new algorithm called Deep TAMER was later introduced in 2018 during a collaboration between U.S. Army Research Laboratory (ARL) and UT researchers. Deep TAMER used deep learning to provide a robot the ability to learn new tasks through observation. Using Deep TAMER, a robot learned a task with a human trainer, watching video streams or observing a human perform a task in-person. The robot later practiced the task with the help of some coaching from the trainer, who provided feedback such as “good job” and “bad job.”

Revision Sheet # 5

Assignment # 13

Prepare a survey on **perception** including:

definitions ,theories, applications ,problems and research areas and inter-relations with the AI Main Topics (reasoning, knowledge, planning , learning, communication, perception , ability to move and manipulate object).

Assignment # 14

Consider the problem faced by an infant learning to speak and understand a language.

- (a) Explain how this process fits into the general learning model. Describe the percepts and actions of the infant, and the types of learning the infant must do. Describe the subfunctions the infant is trying to learn in terms of inputs and outputs, and available example data.

- (b)** Repeat (a) for the case of learning to play tennis (or some other sport with which you are familiar).

Is this supervised learning or reinforcement learning?

CHAPTER 6

Planning

6.1 Definition of Planning Problem

Planning in organizations and public policy is both:

1. The organizational process of creating and maintaining a plan;
2. The psychological process of thinking about the activities required to create a desired goal on some scale.

As such, it is a fundamental property of intelligent behavior. This thought process is essential to the creation and refinement of a plan, or integration of it with other plans, that is, it combines forecasting of developments with the preparation of scenarios of how to react to them.

6.2 Types of plans

- Business Plan
- Enterprise Architecture Planning
- Event Planning and Production
- Family Planning
- Financial Plan
- Land use planning
- Lesson Plan
- Marketing Plan
- Network Resource Planning
- Strategic Planning

6.3 Applicability of Planning

- Mobile robots
 - An initial motivator, and still being developed

- Simulated environments
 - Goal-directed agents for training or games
- Web and grid environments
 - Composing queries or services
 - Workflows on a computational grid
- Managing crisis situations
 - e.g. oil-spill, forest fires, urban evacuation, in factories, ...
- And many more.....
 - Factory automation, flying autonomous spacecraft, playing bridge, military planning, ...

Generating Plans Problem

- Given:
 - A way to describe the world
 - An initial state of the world
 - A goal description
 - A set of possible actions to change the world.
- Required to find:
 - A prescription for actions to change the initial state into one that satisfies the goal.

Generating plans steps

Planning is not done off hand. It is prepared after careful and extensive research. For a comprehensive business plan, management has to:

1. Clearly define the target / goal in writing:
 - a. It should be set by a person having authority
 - b. The goal should be realistic
 - c. It should be specific

- d. Acceptability
 - e. Easily measurable
2. Identify all the main issues which need to be addressed
 3. Review past performance
 4. Decide budgetary requirement.
 5. Focus on matters of strategic importance.
 6. What are requirements and how will they be met.
 7. What will be the likely length of the plan and its structure?
 8. Identify shortcomings in the concept and gaps.
 9. Use Strategies for implementation.
 10. Review periodically.

6.4 Planning Algorithms

- STRIPS Stanford Research Institute Problem Solver
- ABSTRIPS Abstraction STRIPS
- ABTWEAK Abstraction TWEAK
- PEST Planning and Execution System Test bed
- RAPs Reactive Action Packages
- CAPlan Computer assisted Planning
- MESS Multiple-Event Stream Simulator
- TRAINS Mixed-initiative planning
- STAN State Analysis Planner
- CPR Core Plan Representation

STRIPS

In artificial intelligence, STRIPS (Stanford Research Institute Problem Solver) is an automated planner developed by Richard Fikes and Nils Nilsson in 1971. The same name was later used to refer to the formal language of the inputs to this planner. This language is the base for most of the languages for expressing

automated planning problem instances in use today. As an example, STRIPS problem:

“A monkey is in a lab. The monkey wants some bananas. There are three locations in the lab – locations A, B and C. The monkey is at location A. There is a box in location C. There are some bananas in location B, but they are hanging from the ceiling. The monkey needs the box to get to the bananas. A STRIPS instance is composed of:

- An initial state;
- The specification of the goal states – situations which the planner is trying to reach;
- A set of actions. For each action, the following are included:
 - preconditions (what must be established before the action is performed);
 - postconditions (what is established after the action is performed).

Generic STRIPS Algorithm

STRIPS(initstate, goals, ops)

Let currentstate be initstate;

For each goal in goals do

 If goal cannot be proven in current state

 Pick an operator instance, op, s.t. goal $\hat{=}$ adds(op);

 /* Solve preconditions */

 STRIPS(currentstate, preconds(op), ops);

 /* Apply operator */

 currentstate := currentstate + adds(op) dels(ops);

 /* Patch any clobbered goals */

Let rgoals be anyrandum goals which are not provable in currentstate;

STRIPS(currentstate, rgoals, ops).

6.5 Planning Examples

Example 1: Monkey plan

- Initial state: At(A), Level(low), BoxAt (C), BananasAt (B)
- Goal state: Have (Bananas)
- Actions: `_Move(X, Y)_`
//move from X to Y
- Preconditions: At(X), Level(low).
- Postconditions: not At(X), At(Y)

`_ClimbUp(Location)_` //climb up on the box .

- Preconditions: At(Location), BoxAt(Location), Level(low)
- Preconditions: At(Location), BoxAt(Location), Level(low)
- Postconditions: Level(high), not Level(low)

`_ClimbDown(Location)_`

//climb down from the box

- Preconditions: At(Location), BoxAt(Location), Level(high)
- Postconditions: Level(low), not Level(high)

`_MoveBox(X, Y)_`

//move the box from X to Y -- And the monkey moves, too

- Preconditions: At(X), BoxAt(X), Level(low) Postconditions:
- BoxAt(Y), not BoxAt(X), At(Y), not At(X)

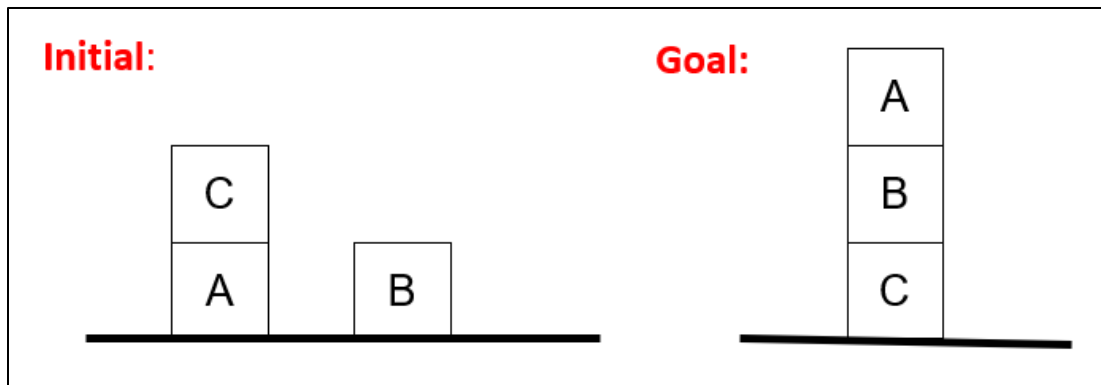
`_TakeBananas(Location)_`

//take the bananas

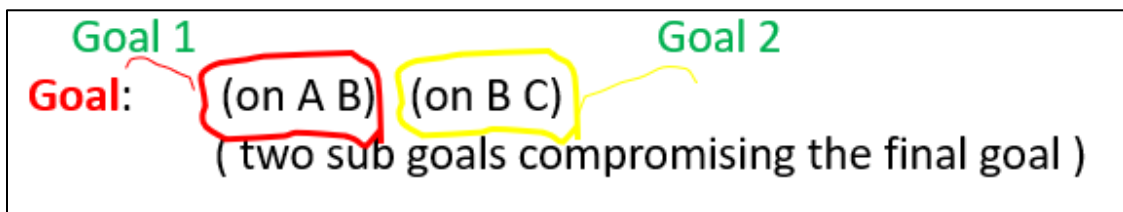
- Preconditions: At(Location), BananasAt(Location), Level(high)
- Postcondition: Have(bananas)

Blocks World (Sussman Anomaly)

Goals are interleaved, they are not separate to each other. The Sussman anomaly is a problem in artificial intelligence, first described by Gerald Sussman, that illustrates a weakness of non-interleaved planning algorithms which were prominent in the early 1970s.

Example 2: blocks world (Sussman Anomaly)

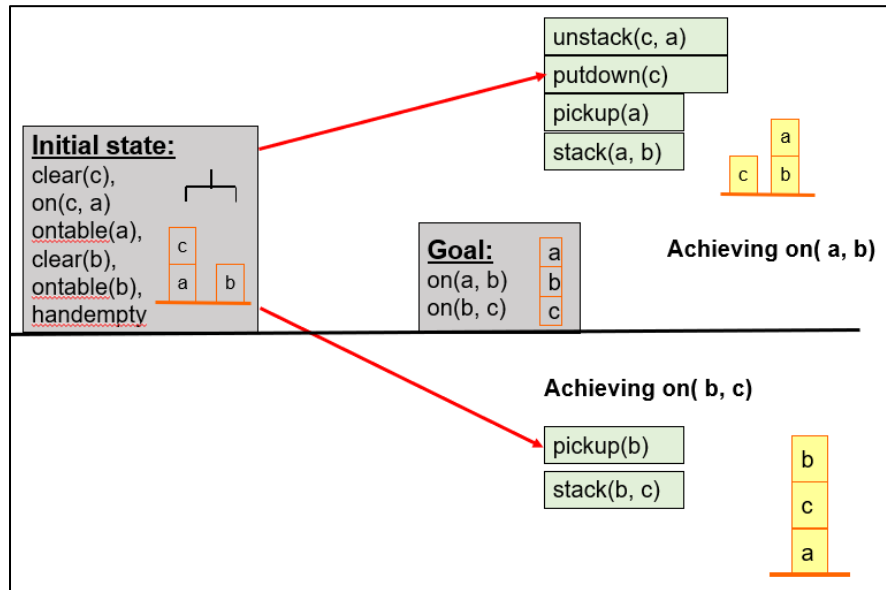
State I: (on-table A) (on C A) (on-table B) (clear B) (clear C)

**Planning is non-trivial**

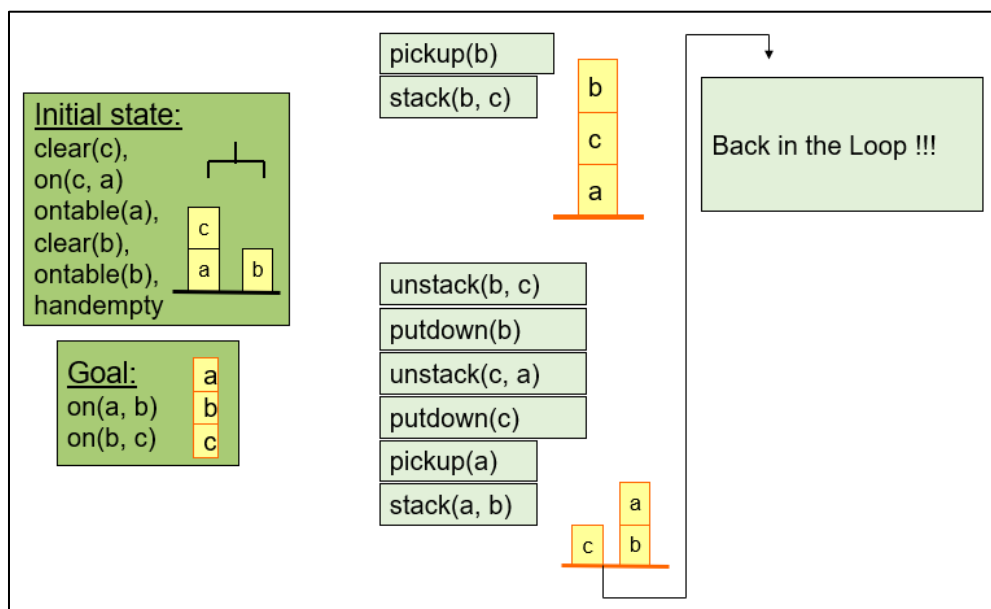
Suppose the planner, starts by pursuing Goal 1 (on A B). The straightforward solution is to move C out of the way, then move A on top B. But while this sequence accomplishes Goal 1, the agent cannot now pursue Goal 2 (on B C) ; without undoing Goal 1 since both A and B must be moved on top C.

If instead, the planner starts with Goal 2 (on B C); the most efficient solution is to move B. But again, the planner cannot pursue Goal 1 (on A B) without undoing Goal 2. So, starting of either goals does not assure success in achieving other goals. This is due to the contradiction relation between the two goals. Planning is non-trivial!

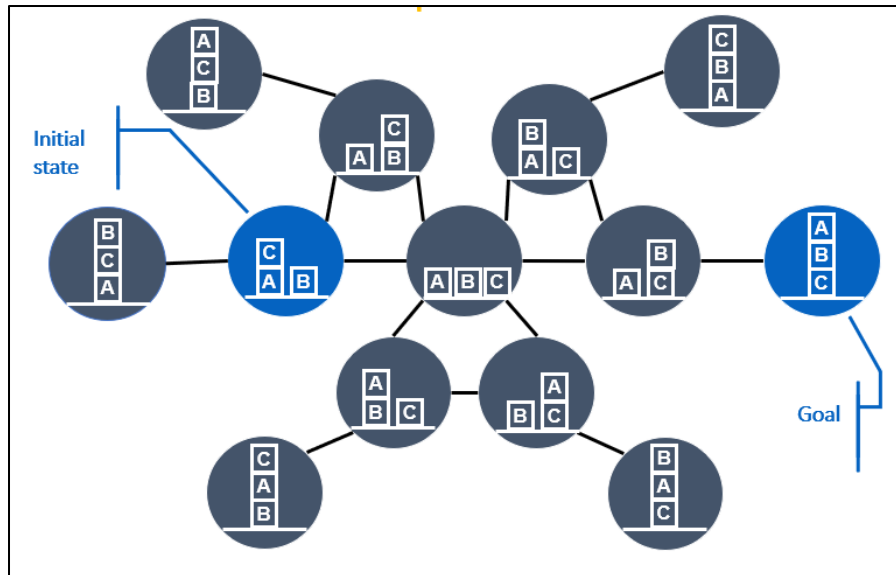
Sussman Anomaly again (using assistant hand)



Sussman Anomaly continued: Undoing a Goal



State-Space Search



Example 3: Towers of Hanoi

It is a mathematical game or puzzle. It consists of three rods, and a number of disks of different sizes which can slide onto any rod. The puzzle starts with the disks neatly stacked in order of size on one rod, the smallest at the top, thus making a conical shape. The objective of the puzzle is to move the entire stack to another rod, obeying the following rules:

1. Only one disk may be moved at a time.
2. Each move consists of taking the upper disk from one of the rods and sliding it onto another rod, on top of the other disks that may already be present on that rod.
3. No disk may be placed on top of a smaller disk.

Simple Solution

The following solution is a simple solution for the puzzle:

1. Alternate moves between the smallest piece and a non-smallest piece.
2. When moving the smallest piece, always move it in the same direction (either to the left or to the right but be consistent).

3. If there is no tower in the chosen direction, move it to the opposite end.
4. When the turn is to move the non-smallest piece, there is only one legal move.

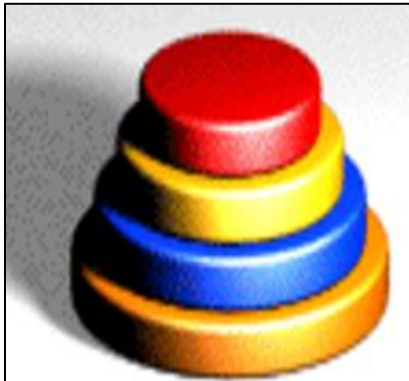
Solution for the Tower of Hanoi 64

If we were able to move 64 disks at a rate of one per second, using the smallest number of moves, it would take them $2^{64}-1$ seconds or roughly 600 billion years where operation taking place is:

- It would take 18,446,744,073,709,551,615 turns to finish

$$\frac{2^{64} - 1}{60 \times 60 \times 24 \times 365.2425}$$

Animated solution for Towers of Hanoi 4



It needs $2^4-1 = 16-1 = 15$ steps

Tower of Hanoi Applications

1. It is frequently used in psychological research on problem solving.
2. It is also used as Backup rotation scheme when performing computer data Backups where multiple tapes/media are involved.
3. The Tower of Hanoi is also used as a memory test by neuropsychologists trying to evaluate amnesia (loosing memory capabilities after collision)

Revision Sheet # 6

Assignment # 15

Prepare a survey on uncertainty concept including:

definitions ,theories, applications ,problems and research areas and inter-relations with the AI Main Topics (reasoning, knowledge, planning , learning, communication, perception , ability to move and manipulate object).

Assignment # 16

- Assume you have 3 rings, (ring1, ring2, ring3) placed on rod A from top to bottom such that; diameter of ring 1 is $<$ diameter of ring 2 $<$ diameter of ring 3.
 - Assume you have another two empty rods: rod_B and rod_C
 - Put your plan, obeying the rules mentioned in slide # 20 and using the three rods; to put the three rings in the same order on rod_C
- (Hint: each step in your plan, must include preconditions, postconditions and action)
- How many steps are needed for your plan?

CHAPTER 7

Logic Programming

7.1 Logic Programming Tools

Logic programming is a programming, database and knowledge-representation and reasoning paradigm which is based on formal logic. A program, database or knowledge base in a logic programming language is a set of sentences in logical form, expressing facts and rules about some problem domain. Major logic programming language families include Prolog, Answer Set Programming (ASP) and Datalog. In all of these languages, rules are written in the form of clauses:

$A :- B_1, \dots, B_n.$

and are read as declarative sentences in logical form:

A if B₁ and ... and B_n.

A is called the head of the rule, B₁, ..., B_n is called the body, and the B_i are called literals or conditions. When $n = 0$, the rule is called a fact and is written in the simplified form:

A.

Queries (or goals) have the same syntax as the bodies of rules and are commonly written in the form:

$?- B_1, \dots, B_n.$

In the simplest case of Horn clauses (or "definite" clauses), all of the A, B₁, ..., B_n are atomic formulae of the form $p(t_1, \dots, t_m)$, where p is a predicate symbol naming a relation, like "motherhood", and the t_i are terms naming objects (or individuals). Terms include both constant symbols, like "charles", and variables, such as X, which start with an upper-case letter.

Consider, for example, the following Horn clause program:

```
mother_child(elizabeth, charles).
```

```
father_child(charles, william).
```

```
father_child(charles, harry).
```

```
parent_child(X, Y) :-
```

```
    mother_child(X, Y).
```

```
parent_child(X, Y) :-
```

```
    father_child(X, Y).
```

```
grandparent_child(X, Y) :-
```

```
    parent_child(X, Z),
```

```
    parent_child(Z, Y).
```

The program can be queried both to generate grandparents and to generate grandchildren. It can even be used to generate all pairs of grandchildren and grandparents:

```
?- grandparent_child(X, william).
```

```
X = elizabeth.
```

```
?- grandparent_child(elizabeth, Y).
```

```
Y = william;
```

```
Y = harry.
```

```
?- grandparent_child(X, Y).
```

X = elizabeth

Y = william.

X = elizabeth

Y = harry.

Although Horn clause logic programs are Turing complete,[1][2] for most practical applications, Horn clause programs need to be extended to "normal" logic programs with negative conditions. For example, the definition of sibling uses a negative condition, where the predicate = is defined by the clause $X = X$:

sibling(X, Y) :-

 parent_child(Z, X),

 parent_child(Z, Y),

 not(X = Y).

Logic programming languages that include negative conditions have the knowledge representation capabilities of a non-monotonic logic.

In ASP and Datalog, logic programs have only a declarative reading, and their execution is performed by means of a proof procedure or model generator whose behaviour is not meant to be controlled by the programmer. However, in the Prolog family of languages, logic programs also have a procedural interpretation as goal-reduction procedures:

to solve A, solve B1, and ... and solve Bn.

Negative conditions in the bodies of clauses also have a procedural interpretation, known as negation as failure: A negative literal `not B` is deemed to hold if and only if the positive literal `B` fails to hold.

Much of the research in the field of logic programming has been concerned with trying to develop a logical semantics for negation as failure and with developing other semantics and other implementations for negation. These developments have been important, in turn, for supporting the development of formal methods for logic-based program verification and program transformation.

Variants and extensions

Prolog

The SLD resolution rule of inference is neutral about the order in which sub goals in the bodies of clauses can be *selected* for solution. For the sake of efficiency, Prolog restricts this order to the order in which the sub goals are written. SLD is also neutral about the strategy for searching the space of SLD proofs. Prolog searches this space, top-down, depth-first, trying different clauses for solving the same (sub)goal in the order in which the clauses are written.

This search strategy has the advantage that the current branch of the tree can be represented efficiently by a stack. When a goal clause at the top of the stack is reduced to a new goal clause, the new goal clause is pushed onto the top of the stack. When the selected sub goal in the goal clause at the top of the stack cannot be solved, the search strategy *backtracks*, removing the goal clause from the top of the stack, and retrying the attempted solution of the selected sub goal in the previous goal clause using the next clause that matches the selected sub goal.

Backtracking can be restricted by using a sub goal, called *cut*, written as `!`, which always succeeds but cannot be backtracked. Cut can be used to improve efficiency, but can also interfere with the logical meaning of clauses. In many cases, the use of cut can be replaced by negation as failure. In fact, negation as failure can be defined in Prolog, by using cut, together with any literal, say *fail*, that unifies with the head of no clause:

```
not(P) :- P, !, fail.
```

```
not(P).
```

This definition illustrates the homoiconic property of Prolog, which means that programs and program components (such as P in the definition of $\text{not}(P)$) can be treated as data. This is a powerful feature which has many applications, including its use for implementing met interpreters.

Prolog has also contributed to the development of other programming languages, including ALF, Fril, Gödel, Mercury, Oz, Ciao, Visual Prolog, XSB, and λ Prolog.

Constraint Logic Programming

Constraint logic programming (CLP) combines Horn clause logic programming with constraint solving. It extends Horn clauses by allowing some predicates, declared as constraint predicates, to occur as literals in the body of a clause. Constraint predicates are not defined by the facts and rules in the program, but are predefined by some domain-specific model-theoretic structure or theory.

Procedurally, sub goals whose predicates are defined by the program are solved by goal-reduction, as in ordinary logic programming, but constraints are simplified and checked for satisfiability by a domain-specific constraint-solver, which implements the semantics of the constraint predicates. An initial problem is solved by reducing it to a satisfiable conjunction of constraints.

Interestingly, the first version of Prolog already included a constraint predicate $\text{dif}(\text{term1}, \text{term2})$, from Philippe Roussel's 1972 PhD thesis, which succeeds if both of its arguments are different terms, but which is delayed if either of the terms contains a variable.

The following constraint logic program represents a toy temporal database of John's history as a teacher:

$\text{teaches}(\text{john}, \text{hardware}, T) :- 1990 \leq T, T < 1999.$

$\text{teaches}(\text{john}, \text{software}, T) :- 1999 \leq T, T < 2005.$

$\text{teaches}(\text{john}, \text{logic}, T) :- 2005 \leq T, T \leq 2012.$

$\text{rank}(\text{john}, \text{instructor}, T) :- 1990 \leq T, T < 2010.$

$\text{rank}(\text{john}, \text{professor}, T) :- 2010 \leq T, T < 2014.$

Here $\leq$ and $<$ are constraint predicates, with their usual intended semantics. The following goal clause queries the database to find out when John both taught logic and was a professor:

$?- \text{teaches}(\text{john}, \text{logic}, T), \text{rank}(\text{john}, \text{professor}, T).$

The solution $2010 \leq T, T \leq 2012$ results from simplifying the constraints $2005 \leq T, T \leq 2012$, $2010 \leq T, T < 2014$.

Constraint logic programming has been used to solve problems in such fields as civil engineering, mechanical engineering, digital circuit verification, automated timetabling, air traffic control, and finance. It is closely related to abductive logic programming.

Datalog

Datalog is a database definition language, which combines a relational view of data, as in relational databases, with a logical view, as in logic programming.

Relational databases use a relational calculus or relational algebra, with relational operations, such as *union*, *intersection*, *set difference* and *cartesian product* to specify queries, which access a database. Datalog uses logical connectives, such as *or*, *and* and *not* in the bodies of rules to define relations as part of the database itself.

It was recognized early in the development of relational databases that recursive queries cannot be expressed in either relational algebra or relational calculus, and that this deficiency can be remedied by introducing a least-fixed-point operator. In contrast, recursive relations can be defined naturally by rules in logic programs, without the need for any new logical connectives or operators.

Datalog differs from more general logic programming by having only constants and variables as terms. Moreover, all facts are variable-free, and rules are restricted, so that if they are executed bottom-up, then the derived facts are also variable-free.

For example, consider the family database:

`mother_child(elizabeth, charles).`

`father_child(charles, william).`

`father_child(charles, harry).`

`parent_child(X, Y) :-`

`mother_child(X, Y).`

`parent_child(X, Y) :-`

```
father_child(X, Y).
```

```
ancestor_descendant(X, Y) :-
```

```
    parent_child(X, X).
```

```
ancestor_descendant(X, Y) :-
```

```
    ancestor_descendant(X, Z),
```

```
    ancestor_descendant(Z, Y).
```

Bottom-up execution derives the following set of additional facts and terminates:

```
parent_child(elizabeth, charles).
```

```
parent_child(charles, william).
```

```
parent_child(charles, harry).
```

```
ancestor_descendant(elizabeth, charles).
```

```
ancestor_descendant(charles, william).
```

```
ancestor_descendant(charles, harry).
```

```
ancestor_descendant(elizabeth, william).
```

```
ancestor_descendant(elizabeth, harry).
```

Top-down execution derives the same answers to the query:

```
?- ancestor_descendant(X, Y).
```

But then it goes into an infinite loop. However, top-down execution with tabling gives the same answers and terminates without looping.

Answer set programming

Like Datalog, Answer Set programming (ASP) is not Turing-complete. Moreover, instead of separating goals (or queries) from the program to be used in solving the goals, ASP treats the whole program as a goal, and solves the goal by generating a stable model that makes the goal true. For this purpose, it uses the stable model semantics, according to which a logic program can have zero, one or more intended models. For example, the following program represents a degenerate variant of the map coloring problem of coloring two countries red or green:

```
country(oz).
country(iz).
adjacent(oz, iz).
colour(C, red) :- country(C), not(colour(C, green)).
colour(C, green) :- country(C), not(colour(C, red)).
```

The problem has four solutions represented by four stable models:

```
country(oz). country(iz). adjacent(oz, iz). color(oz, red).  color(iz, red).

country(oz). country(iz). adjacent(oz, iz). color(oz, green). color(iz, green).

country(oz). country(iz). adjacent(oz, iz). color(oz, red).  color(iz, green).

country(oz). country(iz). adjacent(oz, iz). color(oz, green). color(iz, red).
```

To represent the standard version of the map coloring problem, we need to add a constraint that two adjacent countries cannot be colored the same color. In ASP, this constraint can be written as a clause of the form:

```
:- country(C1), country(C2), adjacent(C1, C2), color(C1, X), color(C2, X).
```

With the addition of this constraint, the problem now has only two solutions:

```
country(oz). country(iz). adjacent(oz, iz). color(oz, red). color(iz, green).
```

```
country(oz). country(iz). adjacent(oz, iz). colour(oz, green). colour(iz, red).
```

The addition of constraints of the form :- Body. eliminates models in which Body is true.

Confusingly, *constraints in ASP* are different from *constraints in CLP*. Constraints in CLP are predicates that qualify answers to queries (and solutions of goals). Constraints in ASP are clauses that eliminate models that would otherwise satisfy goals. Constraints in ASP are like integrity constraints in databases.

This combination of ordinary logic programming clauses and constraint clauses illustrates the generate-and-test methodology of problem solving in ASP: The ordinary clauses define a search space of possible solutions, and the constraints filter out unwanted solutions.

Most implementations of ASP proceed in two steps: First they instantiate the program in all possible ways, reducing it to a propositional logic program (known as *grounding*).

Abductive logic programming

Abductive logic programming (ALP), like CLP, extends normal logic programming by allowing the bodies of clauses to contain literals whose predicates are not defined by clauses. In ALP, these predicates are declared as *assumable*, and are used as in abductive reasoning to explain observations, or more generally to add new facts to the program (as assumptions) to solve goals.

For example, suppose we are given an initial state in which a red block is on a green block on a table at time 0:

```
holds(on(green_block, table), 0).
```

```
holds(on(red_block, green_block), 0).
```

Suppose we are also given the goal:

```
?- holds(on(green_block, red_block), 3), holds(on(red_block, table), 3).
```

The goal can represent an observation, in which case a solution is an explanation of the observation. Or the goal can represent a desired future state of affairs, in which case a solution is a plan for achieving the goal.

We can use the rules for cause and effect presented earlier to solve the goal, by treating the `happens` predicate as assumable:

`holds(Fact, Time2) :-`

`happens(Event, Time1),`

`Time2 is Time1 + 1,`

`initiates(Event, Fact).`

`holds(Fact, Time2) :-`

`happens(Event, Time1),`

`Time2 is Time1 + 1,`

`holds(Fact, Time1),`

`not(terminates(Event, Fact)).`

`initiates(move(Object, Place), on(Object, Place)).`

`terminates(move(Object, Place2), on(Object, Place1)).`

ALP solves the goal by reasoning backwards and adding assumptions to the program, to solve abducible sub goals. In this case there are many alternative solutions, including:

`happens(move(red_block, table), 0).`

`happens(tick, 1).`

`happens(move(green_block, red_block), 2).`

```
happens(tick,0).
```

```
happens(move(red_block, table), 1).
```

```
happens(move(green_block, red_block), 2).
```

```
happens(move(red_block, table), 0).
```

```
happens(move(green_block, red_block), 1).
```

```
happens(tick, 2).
```

Here `tick` is an event that marks the passage of time without initiating or terminating any fluent.

There are also solutions in which the two `move` events happen at the same time. For example:

```
happens(move(red_block, table), 0).
```

```
happens(move(green_block, red_block), 0).
```

```
happens(tick, 1).
```

```
happens(tick, 2).
```

Such solutions, if not desired, can be removed by adding an integrity constraint, which is like a constraint clause in ASP:

```
:- happens(move(Block1, Place), Time), happens(move(Block2, Block1), Time).
```

Abductive logic programming has been used for fault diagnosis, planning, natural language processing and machine learning. It has also been used to interpret negation as failure as a form of abductive reasoning. [\[53\]](#)

Inductive logic programming

Inductive logic programming (ILP) is an approach to machine learning that induces logic programs as hypothetical generalizations of positive and negative examples. Given a logic program representing background knowledge and positive examples together with constraints representing negative examples, an ILP system induces a logic program that generalizes the positive examples while excluding the negative examples.

ILP is similar to ALP, in that both can be viewed as generating hypotheses to explain observations, and as employing constraints to exclude undesirable hypotheses. But in ALP the hypotheses are variable-free facts, and in ILP the hypotheses are general rules.

For example, given only background knowledge of the `mother_child` and `father_child` relations, and suitable examples of the `grandparent_child` relation, current ILP systems can generate the definition of `grandparent_child`, inventing an auxiliary predicate, which can be interpreted as the `parent_child` relation:

```
grandparent_child(X, Y):- auxiliary(X, Z), auxiliary(Z, Y).
```

```
auxiliary(X, Y):- mother_child(X, Y).
```

```
auxiliary(X, Y):- father_child(X, Y).
```

Stuart Russell has referred to such invention of new concepts as the most important step needed for reaching human-level AI.

Concurrent logic programming

Concurrent logic programming integrates concepts of logic programming with concurrent programming. Its development was given a big impetus in the 1980s by its choice for the systems programming language of the Japanese Fifth Generation Project (FGCS).

A concurrent logic program is a set of guarded Horn clauses of the form:

$$H :- G_1, \dots, G_n \mid B_1, \dots, B_n.$$

The conjunction $G_1, \dots, G_n$ is called the guard of the clause, and $\mid$ is the commitment operator. Declaratively, guarded Horn clauses are read as ordinary logical implications:

$$H \text{ if } G_1 \text{ and } \dots \text{ and } G_n \text{ and } B_1 \text{ and } \dots \text{ and } B_n.$$

However, procedurally, when there are several clauses whose heads H match a given goal, then all of the clauses are executed in parallel, checking whether their guards $G_1, \dots, G_n$ hold. If the guards of more than one clause hold, then a committed choice is made to one of the clauses, and execution proceeds with the sub goals $B_1, \dots, B_n$ of the chosen clause. These sub goals can also be executed in parallel. Thus, concurrent logic programming implements a form of "don't care nondeterminism", rather than "don't know nondeterminism".

For example, the following concurrent logic program defines a predicate `shuffle(Left, Right, Merge)`, which can be used to shuffle two lists `Left` and `Right`, combining them into a single list `Merge` that preserves the ordering of the two lists `Left` and `Right`:

```
shuffle([], [], []).
```

```
shuffle(Left, Right, Merge) :-
```

```
    Left = [First | Rest] |
```

```
    Merge = [First | ShortMerge],
```

```
    shuffle(Rest, Right, ShortMerge).
```

```
shuffle(Left, Right, Merge) :-
```

```
    Right = [First | Rest] |
```

```
    Merge = [First | ShortMerge],
```

```
    shuffle(Left, Rest, ShortMerge).
```

Here, `[]` represents the empty list, and `[Head | Tail]` represents a list with first element `Head` followed by list `Tail`, as in Prolog. (Notice that the first occurrence of `|` in the second and third clauses is the list constructor, whereas the second occurrence of `|` is the commitment operator.) The program can be used, for example, to shuffle the lists `[ace, queen, king]` and `[1, 4, 2]` by invoking the goal clause:

```
shuffle([ace, queen, king], [1, 4, 2], Merge).
```

The program will non-deterministically generate a single solution, for example `Merge = [ace, queen, 1, king, 4, 2]`.

Concurrent constraint logic programming

Concurrent constraint logic programming combines concurrent logic programming and constraint logic programming, using constraints to control concurrency. A clause can contain a guard, which is a set of constraints that may block the applicability of the clause. When the guards of several

clauses are satisfied, concurrent constraint logic programming makes a committed choice to use only one.

Higher-order logic programming

Several researchers have extended logic programming with higher-order programming features derived from higher-order logic, such as predicate variables. Such languages include the Prolog extensions HiLog and λ Prolog.

Linear logic programming

Basing logic programming within linear logic has resulted in the design of logic programming languages that are considerably more expressive than those based on classical logic. Horn clause programs can only represent state change by the change in arguments to predicates. In linear logic programming, one can use the ambient linear logic to support state change. Some early designs of logic programming languages based on linear logic include LO, Lolli, ACL, and Forum. Forum provides a goal-directed interpretation of all linear logic.

Object-oriented logic programming

F-logic extends logic programming with objects and the frame syntax.

Logtalk extends the Prolog programming language with support for objects, protocols, and other OOP concepts. It supports most standard-compliant Prolog systems as backend compilers.

Transaction logic programming

Transaction logic is an extension of logic programming with a logical theory of state-modifying updates. It has both a model-theoretic semantics and a procedural one. An implementation of a subset of Transaction logic is available in the Flora-2 system.

7.2 logic & Resolution proof

Already, from the early 1950s, as soon as the first digital computers became available, research was initiated on using logic for problem solving by means of the computer. This research was undertaken from different points of view.

Several researchers were primarily interested in the mechanization of mathematical proofs: the efficient automated generation of such proofs was their main objective.

One of them was M. Davis who, already in 1954, developed a computer program which was capable of proving several theorems from number theory. The greatest triumph of the program was its proof that the sum of two even numbers is even.

Other researchers, however, were more interested in the study of human problem solving, more in particular in heuristics. For these researchers, mathematical reasoning served as a point of departure for the study of heuristics, and logic seemed to capture the essence of mathematics; they used logic merely as a convenient language for the formal representation of human reasoning.

The classical example of this approach to the area of theorem proving is a program developed by A. Newell, J.C. Shaw and H.A. Simon in 1955, called the Logic Theory Machine. This program was capable of proving several theorems from the Principia Mathematica of A.N.

Whitehead and B. Russell. As early as 1961, J. McCarthy, amongst others, pointed out that theorem proving could also be used for solving non-mathematical problems. This idea was elaborated by many authors. Well known is the early work on so-called question-answering systems by J.R. Slagle and the later work in this field by C.C. Green and B. Raphael.

After some initial success, it soon became apparent that the inference rules known at that time were not as suitable for application in digital computers as hoped for. Many AI researchers lost interest in applying logic, and shifted their attention towards the development of other formalisms for a more efficient representation and manipulation of information.

The breakthrough came thanks to the development of an efficient and flexible inference rule in 1965, named resolution, that allowed applying logic for automated problem solving by the computer, and theorem proving finally gained an established position in artificial intelligence and, more recently, in the computer science as a whole as well.

Logic can directly be used as a knowledge-representation formalism for building knowledge systems; currently however, this is done only on a small scale. But then, the clear semantics of logic makes the formalism eminently suitable as a point of departure for understanding what the other knowledge-representation formalisms are all about.

As an example:

Proof with Resolution:

If it rains, Joe brings his umbrella ($r \rightarrow u$) If Joe has an umbrella, he doesn't get wet ($u \rightarrow \text{NOT } w$) If it doesn't rain, Joe doesn't get wet ($\text{NOT } r \rightarrow \text{NOT } w$)

7.3 Using of logic concept in theorem proving

Proof assistants are gaining in popularity. From helping to teach students how to write mathematical proofs¹ to the large-scale projects like checking compiler correctness, they are seeing more use in the computing world.

Proof assistants are programming languages that correspond with a formal logic, and come in different shapes and forms; Coq has a particular focus on the theorem proving aspect where proofs can be written with intuitive tactics, whereas the language Agda is first and foremost a functional programming language, like Haskell.

Under the hood, theorem provers ensure proof correctness by a strong type system. The mainstream languages are all based on so-called intuitionistic type theory. They all capitalize on the astonishing fact that functions in a functional programming language correspond to proofs in intuitionistic logic; and the types of these functions correspond to the propositions that are derived by said proofs.

Under this correspondence, type-checking a function is the same operation as checking a proof of a proposition. Intuitionism inescapably ties these languages to

the fact that they are unable to prove a simple logical notion; that any proposition is either true or false. This is known as the law of the excluded middle ((lem)), and it characterizes the logic we call classical logic.

Intuitionistic logic rejects this notion, and instead is based on the idea that any proof must be constructive; so, a proof of ‘A or B; must be constructed from a proof of either A or B – it is not enough to prove it can’t be the case that ‘A or B’ is not true. This notion of constructive is strongly tied with computability, and it was believed that a proof had a correspondence with a function only if the proof is constructive.

Until the 80s, it was believed that classical logic did not have a computational counterpart. This belief was challenged when Griffin discovered that a control operator, similar to the call/cc function in Scheme, corresponded directly with an axiom of classical logic; double negation elimination, which states that a proposition being ‘not not true’ is the same as it being ‘true’. This is another axiom that characterizes classical logic. This spawned a new area of research – calculi that correspond with classical logic

Revision Sheet # 7

1. Can you explain De Morgan's laws in the context of programming logic?
2. How do you use logical operators in programming?
3. Describe a situation where you used truth tables in programming logic.
4. How would you define and use predicate logic in programming?
5. Can you explain the concept of idempotent operations? Give an example.
6. What is short-circuit evaluation in logic programming and how does it impact performance?
7. Explain how a ternary operator works and provide a scenario where it would be useful.
8. What is a logic error in programming and how do you identify and fix it?
9. How would you use recursion in logic programming? Can you write a brief code snippet?
10. What is the difference between procedural and declarative logic in programming?
11. Can you explain the Pigeonhole Principle and its relevance in logic programming?
12. What role does Boolean Algebra play in the field of logic programming?
13. Can you explain the concept of First-Order Logic and give an example of its use in programming?
14. Discuss how logical programming differs from other programming paradigms.
15. How do you interpret and manipulate logical expressions and statements in a high-level language?

CHAPTER 8

Natural Language Processing

Natural language processing (NLP) is an interdisciplinary subfield of computer science and linguistics. It is primarily concerned with giving computers the ability to support and manipulate human language. Illustration of the field by a brain and a microchip interacting via language, knowledge representation, signal processing and programming is presented in the following figure.



It involves processing natural language datasets, such as text corpora or speech corpora, using either rule-based or probabilistic (i.e. statistical and, most recently, neural network-based) machine learning approaches.

The goal is a computer capable of "understanding" the contents of documents, including the contextual nuances of the language within them. The technology can then accurately extract information and insights contained in the documents as well as categorize and organize the documents themselves.

Challenges in natural language processing frequently involve speech recognition, natural-language understanding, and natural-language generation.

Neural NLP (present)

In 2003, word n-gram model, at the time the best statistical algorithm, was overperformed by a multi-layer perceptron (with a single hidden layer and context length of several words trained on up to 14 million of words with a CPU cluster in language modelling) by Yoshua Bengio with co-authors.

In 2010, Tomáš Mikolov (then a PhD student at Brno University of Technology) with co-authors applied a simple recurrent neural network with a single hidden layer to language modelling, and in the following years he went on to develop Word2vec. In the 2010s, representation learning and deep neural network-style (featuring many hidden layers) machine learning methods became widespread in natural language processing.

That popularity was due partly to a flurry of results showing that such techniques can achieve state-of-the-art results in many natural language tasks, e.g., in language modeling and parsing. This is increasingly important in medicine and healthcare, where NLP helps analyze notes and text in electronic health records that would otherwise be inaccessible for study when seeking to improve care or protect patient privacy.

NLP Approaches:

Symbolic, statistical, neural networks

Symbolic approach

i.e., the hand-coding of a set of rules for manipulating symbols, coupled with a dictionary lookup, was historically the first approach used both by AI in general and by NLP in particular: such as by writing grammars or devising heuristic rules for stemming.

Machine learning approaches, which include both statistical and neural networks, on the other hand, have many advantages over the symbolic approach:

- both statistical and neural networks methods can focus more on the most common cases extracted from a corpus of texts, whereas the rule-based approach needs to provide rules for both rare cases and common ones equally.
- language models, produced by either statistical or neural networks methods, are more robust to both unfamiliar (e.g. containing words or structures that have not been seen before) and erroneous input (e.g. with misspelled words or words accidentally omitted) in comparison to the rule-based systems, which are also expensive to produce.
- the larger such a (probabilistic) language model is, the more accurate it becomes, in contrast to rule-based systems that can gain accuracy only by increasing the amount and complexity of the rules leading to intractability problems.

Although rule-based systems for manipulating symbols were still in use in 2020, they have become mostly obsolete with the advance of LLMs in 2023.

Before that they were commonly used:

- when the amount of training data is insufficient to successfully apply machine learning methods, e.g., for the machine translation of low-resource languages such as provided by the Apertium system,
- for preprocessing in NLP pipelines, e.g., tokenization, or
- for postprocessing and transforming the output of NLP pipelines, e.g., for knowledge extraction from syntactic parses.

Statistical approach

In the late 1980s and mid-1990s, the statistical approach ended a period of AI winter, which was caused by the inefficiencies of the rule-based approaches.

The earliest decision trees, producing systems of hard if-then rules, were still very similar to the old rule-based approaches. Only the introduction of hidden Markov

models, applied to part-of-speech tagging, announced the end of the old rule-based approach.

Neural networks

A major drawback of statistical methods is that they require elaborate feature engineering. Since 2015, the statistical approach was replaced by neural networks approach, using word embeddings to capture semantic properties of words.

Intermediate tasks (e.g., part-of-speech tagging and dependency parsing) have not been needed anymore.

Neural machine translation, based on then-newly-invented sequence-to-sequence transformations, made obsolete the intermediate steps, such as word alignment, previously necessary for statistical machine translation.

8.1 Natural Language Processing in Search engines

The earliest search engines were primarily keyword driven, gleaning their results by matching a specific query with a webpage or document that included those keywords. This was an inexact science, at best, and could be wildly inaccurate and frustrating for early *internet users*.

That's because keyword search is not a straightforward process. At least, not for humans. In a recent webinar, Vincent Bernard, Director of R&D at Coveo, spoke with three NLP experts about this exciting new technology to better understand the hype around this latest search trend.

“Often, we don’t even know the exact word we should search for because we’re not aware of the content of the document that we are going to search in,” explained Hanieh Deilamsalehy, Machine Learning Researcher at Adobe.

“For human beings, the concept of keyword matching is not intuitive. When we search for something in a document, we expect the search engine to understand what we mean, to look for the concept rather than the keyword.”

With natural language processing (NLP), modern search promises a much more intuitive process for humans. NLP-enabled search engines are designed to understand a searcher’s natural language query and the context around it. This enables the search engine to provide more relevant results — culminating in natural language search.

“We’re no longer looking at keyword matching, but at the semantic meaning of the search query and then trying to retrieve the relevant results,” said Deilamsalehy.

A subset of artificial intelligence (AI), NLP goes (way) beyond keyword matching by using natural language understanding to unveil the semantic meaning of a search query. It attempts to retrieve the most relevant results based on context and searcher intent.

NLP algorithms can parse the nuances and subtleties of human communication in a way that traditional keyword-based search engines cannot.

“They use a mix of analytical signals,” said Eric Immermann, Practice Director of Search and Content at Perficient. “Who searches for what, who clicks on what, how do people interact? These are content-understanding signals that use natural language processing, knowledge graph, and other technologies to help the search platform or search engine better understand the content that’s being searched for.”

Some search engines go even further to bridge the gap of search intent and search results by searching video and images using NLP.

You can type something in Google like “fluffy cats” and Google’s results will include relevant websites, videos, and images featuring fluffy cats. It’s using language and NLP to search for a relevant result across different mediums, which is a key difference between today’s modern search and yesterday’s keyword-driven search.

Unlike traditional search technology, NLP moves away from a straight index approach towards something more conceptual. Using the term “fluffy cat” as an example, here are the three main steps inherent in NLP-driven search, as defined by Kurt Cagle, Managing Editor of Data Science Central.

- It uses query permutations: NLP first tries to figure out what the user means by “cat.” It does this by looking at different permutations of the term. Then, it looks at the word “fluffy” and this term’s associated permutations. The NLP then identifies the commonalities that occur with this word combination. It’s these commonalities that produce the conceptual search results. “In essence, you’re moving through a breadth of information in order to be able to find what the most likely meaning is from that query,” explained Cagle.
- It’s an iterative process: Another focus of NLP — and a key difference versus traditional search — is that we’re moving into search as an iterative process rather than simply a single access point into a system. Said Cagle, “When we talk about search, it’s not just the initial search, it’s that I’ve selected something that tells the system that this is kind of what I’m looking for. That changes the context of what I’m looking for in the future, as part of the overall process.”
- It’s contextual and conversational: An iterative process that considers context and applies what it’s learned from a user’s past searches is more conversational. That’s the essential nature of an NLP search engine and why it’s better suited to the way we, as human beings, search for information.

One key difference between the traditional machine learning (ML) used by legacy search engines and using NLP techniques is that the latter is made possible by deep learning.

With traditional ML, the algorithm performance gets better and better as we introduce more data, but eventually it reaches a plateau. That is, fundamentally, the model stops learning. With deep learning, the model performance keeps getting better and better as we introduce new data to the model.

“Much of our progress in NLP is due to the progress in deep learning and the introduction of transformers and large language models,” explained Deilamsalehy. “Advances in large language models have revolutionized the world of NLP.”

8.2 XML

What is XML?

Extensible Markup Language (XML) lets you define and store data in a shareable manner. XML supports information exchange between computer systems such as websites, databases, and third-party applications. Predefined rules make it easy to transmit data as XML files over any network because the recipient can use those rules to read the data accurately and efficiently.

Why is XML important?

Extensible Markup Language (XML) is a markup language that provides rules to define any data. Unlike other programming languages, XML cannot perform computing operations by itself. Instead, any programming language or software can be implemented for structured data management.

For example, consider a text document with comments on it. The comments might give suggestions like these:

Make the title bold

This sentence is a header

This word is the author

Such comments improve the document’s usability without affecting its content. Similarly, XML uses markup symbols to provide more information about any data. Other software, like browsers and data processing applications, use this information to process structured data more efficiently.

XML tags

You use markup symbols, called tags in XML, to define data. For example, to represent data for a bookstore, you can create tags such as <book>, <title>, and <author>. Your XML document for a single book would have content like this:

```
<book>  
<title> Learning Amazon Web Services </title>  
<author> Mark Wilkins </author>  
</book>
```

Tags bring sophisticated data coding to integrate information flows across different systems.

What are the benefits of using XML?

Support interbusiness transactions

When a company sells a good or service to another company, the two businesses need to exchange information like cost, specifications, and delivery schedules. With Extensible Markup Language (XML), they can share all the necessary information electronically and close complex deals automatically, without any human intervention.

Maintain data integrity

XML lets you transfer data along with the data's description, preventing the loss of data integrity. You can use this descriptive information to do the following:

- Verify data accuracy

- Automatically customize data presentation for different users

- Store data consistently across multiple platforms

Improve search efficiency

Computer programs like search engines can sort and categorize XML files more efficiently and precisely than other types of documents. For example, the word *mark* can be either a noun or a verb. Based on XML tags, search engines can accurately categorize *mark* for relevant search results. Thus, XML helps computers to interpret natural language more efficiently.

Design flexible applications

With XML, you can conveniently upgrade or modify your application design. Many technologies, especially newer ones, come with built-in XML support. They can automatically read and process XML data files so that you can make changes without having to reformat your entire database.

What are the applications of XML?

Extensible Markup Language (XML) is the underlying technology in thousands of applications, ranging from common productivity tools like word processing to book publishing software and even complex application configuration systems.

Data transfer

You can use XML to transfer data between two systems that store the same data in different formats. For example, your website stores dates in MM/DD/YYYY format, but your accounting system stores dates in DD/MM/YYYY format. You can transfer the data from the website to the accounting system by using XML. Your developers can write code that automatically converts the following:

Website data to XML format

XML data to accounting system data

Accounting system data back to XML format

XML data back to website data

Web applications

XML gives structure to the data that you see on webpages. Other website technologies, like HTML, work with XML to present consistent and relevant data to website visitors. For example, consider an e-commerce website that sells clothes. Instead of showing all clothes to all visitors, the website uses XML to create customized webpages based on user preferences. It shows products from specific brands by filtering the <brand> tag.

Documentation

You can use XML to specify the structural information of any technical document. Other programs then process the document structure to present it flexibly. For example, there are XML tags for a paragraph, an item in a numbered list, and a heading. Using these tags, other types of software automatically prepare the document for uses such as printing and webpage publication.

Data type

Many programming languages support XML as a data type. With this support, you can easily write programs in other languages that work directly with XML files.

What are the components of an XML file?

An Extensible Markup Language (XML) file is a text-based document that you can save with the .xml extension. You can write XML similar to other text files. To create or edit an XML file, you can use any of the following:

Text editors like Notepad or Notepad++

Online XML editors

Web browsers

Any XML file includes the following components.

XML document

The <xml></xml> tags are used to mark the beginning and end of an XML file. The content within these tags is also called an XML document. It is the first tag that any software will look for to process XML code.

XML declaration

An XML document begins with some information about XML itself. For example, it might mention the XML version that it follows. This opening is called an XML declaration. Here's an example.

```
<?xml version="1.0" encoding="UTF-8"?>
```

XML elements

All the other tags you create within an XML document are called XML elements.

XML elements can contain these features:

Text

Attributes

Other elements

All XML documents begin with a primary tag, which is called the root element.

For example, consider the XML file below.

```
<InvitationList>
```

```
<family>
```

```
<aunt>
  <name>Christine</name>
  <name>Stephanie</name>
</aunt>
</family>
</InvitationList>
```

<InvitationList> is the root element; *family* and *aunt* are other element names.

XML attributes

XML elements can have other descriptors called attributes. You can define your own attribute names and write the attribute values within quotation marks as shown below.

```
<person age="22">
```

XML content

The data in XML files is also called XML content. For example, in the XML file, you might see data like this.

```
<friend>
  <name>Charlie</name>
  <name>Steve</name>
</friend>
```

The data values *Charlie* and *Steve* are the content.

What is an XML schema?

An Extensible Markup Language (XML) schema is a document that describes some rules or limits on the structure of an XML file. You can describe these constraints in several different ways, like these:

Grammatical rules to determine the order of elements

Yes or No conditions that the content must satisfy

Data types for the content in XML files

Constraints for data integrity

For example, an XML schema for bookstores might impose constraints like these:

A book element will have the attributes *title* and *author*.

The book element will be nested under a category element with an attribute name.

The price of a book will be a separate element nested under *book*.

To meet these constraints, we will write the XML file as shown below.

```
<category name="Technology">  
<book title="Learning Amazon Web Services", author="Mark Wilkins">  
<price>$20</price>  
</book>  
</category>
```

XML schemas enforce consistency in how different software applications create and use XML files. Some industries implement XML schemas that are specific to their operations to reduce complexity in writing XML code for interbusiness data transfer. For example, Scalable Vector Graphics (SVG) is an XML specification for describing computer graphics-related data. Software developers write XML files so that they meet such industry specifications.

What is an XML parser?

An Extensible Markup Language (XML) parser is software that can process or read XML documents to extract the data within them. XML parsers also check the syntax or rules of the XML file and can validate it against a particular XML schema. Because XML is a strict markup language, the parsers will not process the file if there are any validation or syntax errors. For example, the XML parser will give errors if any of these conditions are true:

A closing tag or end tag is missing

Attribute values don't have quotation marks

A schema condition has not been met

Software applications use XML parsers to transform XML files into native data types. They can thus focus on the application logic without having to go into the details of the XML itself.

How is XML different from HTML?

HyperText Markup Language (HTML) is the language used in most webpages. A web browser processes the HTML documents and displays them as a multimedia page. The World Wide Web Consortium (W3C) is the international community that develops protocols and guidelines to ensure the long-term growth of the web. W3C established both the HTML and Extensible Markup Language (XML) standards that website developers implement for consistency and quality.

XML vs. HTML

While HTML and XML files look very similar, there are some key differences.

Purpose

The purpose of HTML is to present and display data. However, XML stores and transports data.

Tags

HTML has predefined tags, but users can create and define their own tags in XML.

Syntax rules

There are some minor yet important differences between HTML and XML syntax. For example, XML is case sensitive, but HTML is not. XML parsers will give errors if you write a tag as <Book> instead of <book>.

How do AWS services support XML?

All AWS data integration services can process Extensible Markup Language (XML) files. We list some examples below.

AWS Glue is a serverless data integration service that you can use to discover, prepare, and combine data for analytics, machine learning, and application development. AWS Glue DataBrew is a visual data preparation tool that you can use to prepare data with an interactive, point-and-click visual interface without writing code. DataBrew can input all types of file formats, including XML.

Amazon Simple Queue Service (SQS) is a fully managed message queuing service that you can use to send, store, and receive messages between software components at any volume. Amazon SQS messages can contain up to 256 KB of text data, including XML, JSON, and unformatted text.

Amazon Kinesis makes it easy to collect, process, and analyze real-time, streaming data so you can get timely insights and react quickly to new information. With the key capabilities of Kinesis, you can process streaming data cost effectively at any scale. You also gain the flexibility to choose tools that suit the requirements of your application. Stream, transform, and analyze XML data in real time with Kinesis. Get started with data integration by creating an AWS account today.

8.3 RDF

The Resource Description Framework (RDF) is a World Wide Web Consortium (W3C) standard originally designed as a data model for metadata. It has come to be used as a general method for description and exchange of graph data. RDF provides a variety of syntax notations and data serialization formats, with Turtle (Terse RDF Triple Language) currently being the most widely used notation.

RDF is a directed graph composed of triple statements. An RDF graph statement is represented by: 1) a node for the subject, 2) an arc that goes from a subject to an object for the predicate, and 3) a node for the object. Each of the three parts of the statement can be identified by a Uniform Resource Identifier (URI). An object can also be a literal value. This simple, flexible data model has a lot of expressive power to represent complex situations, relationships, and other things of interest, while also being appropriately abstract.

RDF was adopted as a W3C recommendation in 1999. The RDF 1.0 specification was published in 2004, the RDF 1.1 specification in 2014. SPARQL is a standard query language for RDF graphs. RDF Schema (RDFS), Web Ontology Language (OWL) and SHACL (Shapes Constraint Language) are ontology languages that are used to describe RDF data.

Overview

The RDF data model is similar to classical conceptual modeling approaches (such as entity–relationship or class diagrams). It is based on the idea of making statements about resources (in particular web resources) in expressions of the form *subject–predicate–object*, known as triples. The *subject* denotes the resource, and the *predicate* denotes traits or aspects of the resource, and expresses a relationship between the *subject* and the *object*.

For example, one way to represent the notion "The sky has the color blue" in RDF is as the triple: a subject denoting "the sky", a predicate denoting "has the color", and an object denoting "blue". Therefore, RDF uses *subject* instead of *object* (or *entity*) in contrast to the typical approach of an entity–attribute–value model in object-oriented design: entity (sky), attribute (color), and value (blue).

RDF is an abstract model with several serialization formats (being essentially specialized file formats). In addition, the particular encoding for resources or triples can vary from format to format.

This mechanism for describing resources is a major component in the W3C's Semantic Web activity: an evolutionary stage of the World Wide Web in which automated software can store, exchange, and use machine-readable information distributed throughout the Web, in turn enabling users to deal with the information with greater efficiency and certainty. RDF's simple data model and ability to model disparate, abstract concepts has also led to its increasing use in knowledge management applications unrelated to Semantic Web activity.

A collection of RDF statements intrinsically represents a labeled, directed multigraph. This makes an RDF data model better suited to

certain kinds of knowledge representation than are other relational or ontological models.

As RDFS, OWL and SHACL demonstrate, one can build additional ontology languages upon RDF.

8.4 Ontology for NLP interface

an ontology is a formal representation of knowledge, while an NLP model is a machine learning-based system designed to process and understand human language. They serve different purposes and are used in knowledge representation and natural language processing in other contexts.

At its core, ontology is the study of what is. To make this a little more concrete, one could also say ontology is the study of what exists or of what is real. “Does God exist?” “Are my feelings real?”, “What is 'nothing,' and does it exist?” are all examples of ontological.

The aim of the Semantic Web is to provide well-defined and understandable information not only by humans, but also by computers, allowing these last ones to automate, integrate and reuse high-quality information across different applications. Ontologies are considered a technology which supports all aforementioned tasks.

In the context of computer and information sciences, an ontology defines a set of representational primitives allowing to model a domain of knowledge or discourse . Nowadays, data stored in ontology-based knowledge bases has significantly grown, becoming in an important component in enhancing the Web intelligence and in supporting data integration. Indeed, ontologies are being applied to different domains such as biomedicine , finance , innovation management , cloud computing , medicine , and human perception , among others. At the same time, Wikipedia, a free-access and free content Internet encyclopedia, has become one of the most popular Web-

based knowledge sources, maintained by thousands of collaborators. DBpedia has emerged as a community effort to structure Wikipedia's content and make it available on the Web. Currently, the English version of DBpedia describes 4.22 million of things in a consistent ontology, including persons, places, creative works, organizations, species and diseases. The most known formal query language for information retrieval from semantic knowledge bases is SPARQL (Simple Protocol and RDF Query Language). The use of SPARQL demands a high level of knowledge and expertise about technologies such as RDF (Resource Description Framework) and query language expressions. Because of this, several research efforts have been carried out in order to make accessible the Semantic Web information to all kind of users. From this perspective, NLI (Natural Language Interface) is deemed to be a very intuitive approach from a user point of view in order to address the gap between knowledge bases systems and end-users. An NLI allows users to access information stored in any repository by formulating requests in natural language .

8.5 Semantic Web and NLP

They are two separate subject areas but they do overlap in some places. Because documents, regardless of their format are made up of heterogeneous syntax and semantics, the goal is to represent information that is understandable to a machine and not just a human being. This is a common goal of the Semantic Web and Natural Language Processing.

Semantic Web

The semantic web is based on two fundamental ideas:

- Associating meta-information with Internet-based resources. Metadata is pieces of information about other data which can be added explicitly or implicitly.
- The ability to reason about the meta-information. For example, a machine should be able to recognize that a picture of balloon is not an animal, even if it is shaped

like one. This idea of reasoning and inference on textual data is still very experimental, however, showing considerable success. There is a range of techniques to query such information such as SPARQL, Machine Learning (a pre-annotated corpus), and other statistical techniques.

The use of ontologies is becoming ever more important in this domain. Description Logic provides the mathematical foundation for knowledge representation systems and can be used to reason with the information.

Natural Language Processing

Whereas Natural Language Processing is an important and ongoing research area in theoretical computer science and artificial intelligence, it can look beyond the web and process anything from text in PDFs to speaking to your phone. Wikipedia has highlighted some of the key areas such as:

1. Automatic (Abstractive & Extractive) Summarization
2. Coreference Resolution
3. Discourse Analysis
4. Language Translation
5. Morphological Segmentation
6. Named Entity Recognition
7. Natural Language Generation
8. Optical Character Recognition
9. Parsing
10. Question Answering
11. Relationship (Semantics) Extraction
12. Speech Segmentation
13. Speech & Voice Recognition
14. Topic Segmentation
15. Word Sense Disambiguation (WSD)

16. Information Retrieval

17. Information Extraction

18. Text Simplification

19. Spelling Correction

Both subject areas have been heavily researched into the syntactics of language, both research fields aim to understand language, notably text. However, in recent times the use of semantics has had a lot of time and investment put into it. But in essence, how to represent relationships in text and miscellaneous structures is a top priority of both fields of thought. As a conclusion, Semantic Web is mostly annotated with RDF, OWL, etc., whereas NLP really focuses on freeform Text.

Revision Sheet # 8

1. What is Naive Bayes algorithm, when we can use this algorithm in NLP?
2. Explain Dependency Parsing in NLP?
3. What is text Summarization?
4. What is NLTK? How is it different from Spacy?
5. What is information extraction?
6. What is Bag of Words?
7. What is Pragmatic Ambiguity in NLP?
8. What is Masked Language Model?
9. What is the difference between NLP and CI (Conversational Interface)?
10. What are the best NLP Tools?